

İçindekiler

Bakım ve İş Emri Yönetim Sistemi — Teknoloji Kararları, Kavramlar ve Yapım Planı

Giriş — neden stack değişti

Kararların dört kutusu

BÖLÜM 0 — Sistem nasıl çalışıyor

İki ayrı bilgisayar

Bir isteğin yolculuğu

Her teknoloji hangi adımda — tek tabloda

Sık karıştırılan üç çift

Tek cümlelik özet

BÖLÜM A — Hızlı eşleme tablosu

A.1 Ödevin zorunlu teknolojileri → benim karşılıklarım

A.2 Ödevde adı geçmeyen ama eklediğim parçalar

A.3 Çalışmanın bölümleri → hangi teknolojiyle karşılanıyor

BÖLÜM B — Sistemin yapması istenen on iki şey

1) Kuruma ait lokasyonlar yönetilebilmelidir

2) Lokasyonlara bağlı varlıklar yönetilebilmelidir

3) Varlıklar için arıza, bakım veya kontrol talepleri oluşturulabilmelidir

4) Oluşturulan talepler iş emrine dönüştürülebilmelidir

5) İş emirleri uygun personele atanabilmelidir

6) İş emirlerinin durumları takip edilebilmelidir

7) İş emirlerine yorum ve işlem kaydı eklenebilmelidir

8) İş emri geçmişi görüntülenebilmelidir

9) İş emirleri için SLA süreleri hesaplanabilmelidir

10) Yaklaşan ve geçen SLA süreleri sistem tarafından takip edilebilmelidir

11) Kullanıcılara sistem içi bildirim oluşturulabilmelidir

12) Yönetim ekranında temel operasyon istatistikleri gösterilebilmelidir

BÖLÜM C — Teknoloji kartları

C.1 NestJS

C.2 Next.js

C.3 Prisma

C.4 Zod

C.5 PostgreSQL

C.6 BullMQ + Redis

C.7 Testcontainers

C.8 dependency-cruiser

C.9 TypeScript

- C.10 Docker ve Docker Compose
- C.11 Vitest
- C.12 Playwright
- C.13 JWT ve argon2 — kimlik doğrulama
- C.14 Turborepo + pnpm workspaces
- C.15 nestjs-pino — Yapılandırılmış Loglama (Sistemin Gözü)
- C.16 nestjs-cls — İstek Bağlamı (Sistemin Hafızası)
- C.17 @nestjs/terminus — sağlık kontrolü
- C.18 Swagger / OpenAPI
- C.19 TanStack Query
- C.20 Tailwind CSS + shadcn/ui + React Hook Form
- C.21 @dbml/cli — veri modeli dokümanı (Database Markup Language)
- C.22 Renovate
- C.23 Expo — ileride mobil için

BÖLÜM E — Kavramlar, prensipler ve desenler

- E.0 Önce temel kelimeler
- E.1 Modüler monolit + Clean Architecture (§10)
- E.2 SOLID prensipleri (§8)
- E.3 DRY prensibi (§9)
- E.4 Factory Pattern ve SLA hesabı (§7)
- E.5 Durum makinesi (§6)
- E.6 Mapping kararı: neden bir mapping kütüphanesi yok (§13)
- E.7 Merkezî hata yönetimi (§19)
- E.8 Transaction ve iyimser eşzamanlılık (§20)
- E.9 Veri modeli kararları (§11, §21)
- E.10 Listeleme, filtreleme ve sayfalama (§17)
- E.11 Git akışı ve CI (§26, §27)
- E.12 Dokümantasyon: ADR ve AI_USAGE (§28, §29)
- E.13 Değerlendirilip seçilmeyen alternatifler

BÖLÜM F — Bir iş emrinin hayatı (sunucu tarafı, uçtan uca akış)

★ 1. adımın açılımı: "aynı şema sunucuda tekrar doğrular" tam olarak nerede

★ 8. adımın açılımı: version kolonu tam olarak nedir

Bu akışta hangi karar nerede görünüyor

BÖLÜM G — Bir ekranın hayatı (arayüz / UI tarafı)

- G.0 Önce sıra sorusu: arka uç (backend) mu önce, arayüz (UI) mü
- G.1 Arayüz (UI) tarafında hangi teknoloji nerede duruyor
- G.2 Bir ekranın hayatı — on adım
- G.3 Bu akışta hangi karar nerede görünüyor

BÖLÜM H — Yapım planı: adım adım ne, neyle, nereye

Bu plan nasıl kuruldu

- Adım 0 — Ortam kurulumu
- Adım 1 — PRD: sistemin ne yapacağını yazılı hâli
- Adım 2 — İskelet: boş ama çalışan sistem
- Adım 3 — Veri modeli
- Adım 4 — Kimlik doğrulama ve yetkilendirme
- Adım 5 — Lokasyon ve varlık modülleri
- Adım 6 — İş emri çekirdeği ve durum makinesi
- Adım 7 — SLA hesabı ve Factory Pattern
- Adım 8 — Arka plan işleri
- Adım 9 — Bildirimler ve yönetim istatistikleri
- Adım 10 — Arayüz (UI) temeli
- Adım 11 — İş emri listesi
- Adım 12 — İş emri detayı ve kalan ekranlar
- Adım 13 — Test tamamlama
- Adım 14 — Dokümantasyon ve sunum
- Adım 15 — Teslim paketi
- Adım 16 — Kite geri yazma

Her adımın sonunda yapılacaklar

KAPANIŞ

Bilinen teknik borçlar

Bu stack'in tek cümlelik özeti

Değişmeyen ilke

Bakım ve İş Emri Yönetim Sistemi — Teknoloji Kararları, Kavramlar ve Yapım Planı

Bu belge üç soruyu birden cevaplar:

#	Soru	Nerede
1	Hangi teknoloji seçildi ve neden ?	BÖLÜM A eşleme tablosu · BÖLÜM C 23 kart
2	O teknoloji nedir , ne işe yarar?	BÖLÜM C kartlar · BÖLÜM E kavramlar
3	Ne sırayla yapılacak?	BÖLÜM H — 17 adım

Ayrıca sistemin uçtan uca nasıl çalıştığı iki bölümde gösteriliyor: **BÖLÜM F** (sunucu tarafı) ve **BÖLÜM G** (arayüz / UI tarafı).

★ **Amaç yalnızca belgelemek değil, anlatabilmek.** Teslim sonrası canlı teknik inceleme var; buradaki her gerekçe orada savunulacak.

Her bölüm **kendi içinde yeterlidir** — bir teknolojiyi okurken başa dönmeniz gerekmez; o teknoloji orada yeniden ve tam olarak anlatılır. Kavramlar, prensipler ve tasarım desenleri Bölüm E'de ayrıca ele alınır.

Sadece "ne yapılacak" sorusunun cevabı aranıyorsa: doğrudan **BÖLÜM H — Yapım planı**'na gidilebilir. Orada her adımın ne ürettiği, hangi teknolojiyle, hangi klasöre yazıldığı ve neye bağlandığı tablo hâlinde duruyor; ayrıntı gerektiğinde ilgili bölüme işaret ediyor.

Sistemi tek bir örnek üzerinden baştan sona görmek için iki bölüm var: **BÖLÜM F** bir isteğin *sunucu* tarafındaki yolculuğunu, **BÖLÜM G** bir ekranın *arayüz (UI)* tarafındaki yolculuğunu anlatıyor. İkisi birlikte okunduğunda sistemin tamamı görülüyor.

Giriş — neden stack değişti

Teknik değerlendirme çalışmasında zorunlu teknolojiler .NET ailesinden verilmişti: ASP.NET Core Web API, Entity Framework Core, Hangfire, AutoMapper, FluentValidation, React + Vite. Kurum **"istediğin stack'i kullanabilirsin"** dediği için tamamını JavaScript/TypeScript ailesindeki karşılıklarıyla değiştirdim.

Değiştirirken tek bir kural uyguladım: **hiçbir yetenek düşmeyecek.** Çalışmanın sorduğu her şey — Factory Pattern, servis yaşam döngüleri, transaction, iyimser eşzamanlılık denetimi, mimari testler, DBML şema dokümanı — yeni stack'te de birebir karşılanıyor. **Araç değişti, beklenti değişmedi.**

Seçimlerimde üç ölçütüm vardı:

1. Piyasada yaygın ve aktif bakımda olması. Bu sistemi yıllarca başkaları da sürdürecektir. Bir kütüphane niş kalıyorsa veya yıllardır yeni sürüm çıkmıyorsa, projeyi devralacak geliştirici için risktir. Bu yüzden kütüphane seçimlerinde haftalık indirme sayısı ve son yayın tarihi **fiilen ölçüldü**; gerekçeler ilgili bölümlerde rakamlarıyla duruyor.

2. Gerçek üretim pratiği olması. İki seçenek arasında kalındığında ölçüt "*hangisi daha hızlı biter*" değil, "**gerçek kullanıcısı ve gerçek nöbetçi ekibi olan bir sistemde hangisi doğru olurdu**" oldu.

Somut örnek: entegrasyon testlerinde sahte bir veritabanı kullanmak çok daha hızlı kurulur ve testler saniyeler yerine milisaniyelerde koşar. Ama sahte veritabanı benzersizlik kısıtını, foreign key'i ve transaction'ı uygulamaz — yani testler yeşil yanar, sistem canlıda patlar. Bu yüzden testler **gerçek PostgreSQL konteynerine** karşı koşuyor (C.7). Aynı yaklaşım eşzamanlılık denetimi, mükerrer bildirim koruması ve hata yönetimi için de geçerli: kolay olan değil, **üretimde doğru olan** seçildi.

3. Devralınabilir olması. Projeyi ilk kez gören biri nereye bakacağını bilmeli — ve daha önemlisi, **yanlış yaptığında sistem ona söylemeli**. Bu yüzden mimari kurallar yorum satırı olarak değil, CI'da build'i durduran denetimler olarak kuruldu (C.8, C.21).

Kararların dört kutusu

Anlatırken her kararın hangi kutuda olduğunu söyleyeceğim:

Kutu	Anlamı	Örnek
1	Ödev istedi, zaten doğrusuydu	PostgreSQL, Docker, Factory Pattern, optimistic concurrency
2	Ödev istedi, JS eşleniğini kullandım	EF Core→Prisma, Hangfire→BullMQ, FluentValidation→Zod
3	Ödev istemedi, gerçek hayat gerektirdi	Renovate, mimari testi, correlation ID, <code>/api/v1</code>
4	Ödev istedi, yapmadım — şunu tercih ettim	AutoMapper yerine Zod+Prisma <code>select</code>

⚠ 4. kutudaki her madde **ölçüyle** desteklenir (indirme sayısı, son yayın tarihi, sürüm kısıtı) — "bence daha iyi" denmez.

BÖLÜM 0 — Sistem nasıl çalışıyor

Bu bölüm zemindir: sonraki bölümlerdeki her teknoloji, aşağıdaki altı adımdan birinde çalışıyor. Hangisinde çalıştığını bilmek, o teknolojiyi anlamanın yarısıdır.

İki ayrı bilgisayar

Her web uygulamasında iş **iki yerde** yapılır:

	İstemci (client)	Sunucu (server)
Neresi	Kullanıcının tarayıcısı / telefonu	Senin kontrolündeki, kapanmayan bilgisayar
Ne yapar	Ekranı çizer, tıklamayı alır	Karar verir, veritabanına gider
Güvenilir mi	Hayır	Evet

"İstemciye güvenilmez" ne demek: Kullanıcı tarayıcıda geliştirici araçlarını açıp gönderdiği veriyi değiştirebilir. "Ben yöneticiyim" diye veri gönderebilir. Bu yüzden "*bu kişi gerçekten yönetici mi*" kararı **her zaman sunucuda** verilir. Ekrandaki "Sil" düğmesini gizlemek bir **kolaylıktır**, güvenlik değil.

Bir isteğin yolculuğu

Kullanıcı "İş Emirleri" ekranını açtığı anda olanlar:

1. Tarayıcı "bana açık iş emirlerini ver"
↓ (internet)
2. API sunucusu "sen kimsin? yetkin var mı?"
↓
3. İş kuralları "bu kullanıcı sadece kendi lokasyonunu görebilir"
↓
4. Veritabanı SELECT ... WHERE ...
↓
5. API sunucusu sonucu JSON'a çevirir, fazla alanları kırpar
↓ (internet)
6. Tarayıcı gelen veriyi tabloya çizer

Aşağıdaki tablo her teknolojinin bu altı adımdan hangisinde çalıştığını gösteriyor.

Her teknoloji hangi adımda — tek tabloda

Teknoloji	Hangi adım	En sade tanımı	Günlük hayattan benzetme
React	6	Veriye göre ekranı çizen kütüphane	Vitrin düzenleyicisi
Next.js	6 (+1)	React'i çalışır siteye dönüştüren framework; sayfaları da sunucuda hazırlar	Vitrini kuran ve mağazayı açan ekip
TypeScript	Hepsi	JavaScript + tip kontrolü. Hatayı çalışmadan önce yakalar	Yazım denetimi
NestJS	2–5	Sunucu tarafı uygulama çatısı	Bir ofisin organizasyon şeması
Express	2	Gelen isteği karşılayan temel katman. <i>Nest zaten bunu kullanır</i>	Santral görevlisi
Prisma	4	Kod ile veritabanı arasındaki tercüman	Tercüman
PostgreSQL	4	Verinin durduğu yer	Arşiv/dolap
Zod	2 ve 5	"Gelen veri doğru biçimde mi?" kontrolü	Form denetleyicisi
JWT	2	Giriş yapan kullanıcıya verilen dijital kimlik kartı	Otel kartı
argon2	2	Şifreyi geri çevrelemez şekilde karıştırır	Kağıt öğütücü
BullMQ	—	Sonra/düzenli yapılacak işlerin listesi	Yapılacaklar defteri
Redis	—	Çok hızlı, geçici hafıza. BullMQ'nun defteri burada durur	Not kağıdı
Docker	—	Uygulamayı ihtiyaçlarıyla birlikte tek pakete koyar	Hazır yemek kabı
Vitest / Playwright	—	Testler: "hâlâ çalışıyor mu?"	Kalite kontrol
Swagger	—	API'nin kullanma kılavuzu, kendiliğinden üretilir	Kullanım kılavuzu
Expo	6	Aynı React bilgisiyle telefon uygulaması yazma aracı	—

Sık karıştırılan üç çift

1. Kütüphane ile framework Kütüphaneyi **sen çağırırsın**, framework **seni çağırır**. React bir kütüphanedir — sen "şunu çiz" dersin. Next.js ve NestJS framework'tür — kuralları onlar koyar, sen boşlukları doldurursun. Framework daha kısıtlayıcıdır ama karşılığında düzen verir; on kişilik ekipte bu düzen her şeydir.

2. Frontend ile backend Frontend = kullanıcının **gördüğü**. Backend = kullanıcının **göremediği** ama işi yapan. Bizde frontend Next.js, backend NestJS.

3. Veritabanı ile ORM Veritabanı (PostgreSQL) verinin durduğu yer. ORM (Prisma) oraya konuşan tercüman. ORM olmadan da olur — SQL'i elle yazarsın; ama o zaman tip güvenliğini ve migration yönetimini kendin kurarsın.

Tek cümlelik özet

"Kullanıcı **Next.js** ile yazılmış ekranı açar. Ekran, **NestJS** ile yazılmış API'ye istek atar. API gelen veriyi **Zod** ile doğrular, kullanıcının yetkisini **JWT** ile kontrol eder, iş kurallarını uygular ve **Prisma** üzerinden **PostgreSQL**'e gider. Sonuç JSON olarak döner. Kullanıcıyı bekletmemesi gereken işler **BullMQ** ile kuyruğa alınır ve ayrı bir worker süreci onları yapar. Hepsi **Docker** ile paketlenir, tek komutla ayağa kalkar."

BÖLÜM A — Hızlı eşleme tablosu

(Bu bölüm içindekiler niteliğindedir. Ayrıntılar Bölüm B ve C'de.)

A.1 Ödevin zorunlu teknolojileri → benim karşılıklarım

Çalışmada istenen	Bu projede kullandığım	Tek cümlelik gerekçe
Güncel .NET + ASP.NET Core Web API	NestJS	Node tarafında bağımlılık enjeksiyonu, modül sistemi ve yetki/doğrulama/hata katmanlarını hazır getiren yaygın çatı
C#	TypeScript (strict)	Derleme zamanında tip güvenliği
Entity Framework Core	Prisma	Şema tek dosyada okunur; migration motoru ve tip güvenli sorgu birlikte gelir
PostgreSQL	PostgreSQL	Değişmedi
FluentValidation	Zod	Tek şemadan hem doğrulama, hem TypeScript tipi, hem OpenAPI dokümanı üretilir
Hangfire	BullMQ + Redis	Gecikmeli iş, tekrarlayan iş, yeniden deneme ve yatay ölçekleme
AutoMapper veya Mapster	Zod şeması + Prisma <code>select</code>	Aday kütüphaneler ölçüldü; biri bakımsız, diğeri niş kaldı
OpenAPI + Scalar/Swagger	@nestjs/swagger → Swagger UI	Doküman Zod şemasından üretiliyor, elle güncelleme gerekmiyor
React	React	Değişmedi
JavaScript veya TypeScript	TypeScript	—
Vite	Next.js (kendi derleyicisi)	Next, Vite'in paketleyici görevini kendi içinde yapıyor
React Router	Next.js App Router	Next, yönlendirmeyi dosya yapısıyla çözüyor
xUnit / NUnit / MSTest	Vitest	JS tarafının yaygın test koşucusu
Assertion kütüphanesi	Vitest'in kendi <code>expect</code> 'i	Ayrı paket gerekmiyor
PostgreSQL ile integration test	Testcontainers	Test sırasında gerçek PostgreSQL konteyneri ayağa kaldırır
Docker · Docker Compose · Git	Aynı	Değişmedi

A.2 Ödevde adı geçmeyen ama eklediğim parçalar

Eklenen	Neden eklendi
Redis	BullMQ'nun iş kuyruğu burada durur; ayrıca hız sınırı sayacı ve dağıtık kilit için kullanılıyor
argon2	Şifrelerin güvenli saklanması (§5.1) için sektörde önerilen özetleme algoritması
nestjs-cls	Aktif kullanıcı ve iz kimliğini katmanlara parametre geçmeden taşır — §21'in "statik yapılardan alınmasın" şartının karşılığı
nestjs-pino	Yapılandırılmış (JSON) log üretir; §25'in istediği biçim
@nestjs/terminus	/health/live ve /health/ready uçları (§25)
dependency-cruiser	Katman bağımlılık kurallarını CI'da denetler — §23'ün mimari testleri
@dbml/cli	docs/database.dbml dosyasını migration'ın ürettiği gerçek şemadan üretir (§11)
Turborepo + pnpm workspaces	Web, API ve worker'ı tek depoda tutar; ortak tipler tek yerde yaşar (§9 DRY)
Renovate	Bağımlılık güncellemelerini otomatik takip eder

A.3 Çalışmanın bölümleri → hangi teknolojiyle karşılanıyor

Bölüm	Ne isteniyor	Nasıl karşılanıyor
§4 Kullanıcı rolleri	Dört rol, yetkileri ayrılmış	NestJS Guard 'ları; yetki kontrolü tek yerde
§5.1 Kimlik doğrulama	JWT, yenileme jetonu (token), rol bazlı yetki, pasif kullanıcı engeli	<code>@nestjs/jwt</code> + argon2 + jeton (token) döndürme ve yeniden kullanım tespiti
§5.2 Lokasyon yönetimi	Liste, detay, oluştur, güncelle, aktif/pasif	NestJS modülü + Prisma + Zod
§5.3 Varlık yönetimi	Liste, filtre, detay, durum değiştirme	Aynı yapı; filtreleme veritabanı seviyesinde
§5.4 Talep ve iş emri	Zorunlu alanlar, okunabilir iş emri numarası	Prisma şeması + veritabanı sırası (sequence)
§6 Durum yönetimi	Kontrollü geçişler, her değişiklikte geçmiş	Durum makinesi + tek transaction
§7 SLA ve Factory Pattern	Factory zorunlu, Open/Closed'a uygun	NestJS bağımlılık enjeksiyonu ile politika dizisi
§8 SOLID	Katman bağımlılıkları korunmalı	Katmanlı yapı + dependency-cruiser
§9 DRY	Tekrarlanan kural/mapping/hata olmasın	Ortak şema paketi + tek doğrulama + tek hata filtresi
§10 Proje mimarisi	Tutarlı ve gerekçeli mimari	Modüler monolit + Clean Architecture
§11 Veri tabanı tasarımı	Şema, index, audit, DBML	Prisma şeması + <code>@dbml/cli</code>
§12 Entity Framework Core	Migration, ilişki, transaction, N+1 önleme	Prisma
§13 Mapping	Entity dönülmesin, hassas alan sızmasın	Zod şeması + Prisma <code>select</code>
§14 DI ve yaşam döngüleri	Transient/Scoped/Singleton	NestJS DI
§15 Zamanlanmış görevler	Dört iş türü, idempotent, korumalı panel	BullMQ + Redis
§16 Bildirim yönetimi	Mükerrer bildirim engellenmeli	Benzersiz index + benzersiz iş anahtarı
§17 Listeleme ve sayfalama	Sunucu tarafı filtre, arama, sıralama	Prisma + <code>pg_trgm</code> + GIN index
§18 Validasyon	Hem biçim hem iş kuralı	Zod (biçim) + kurallar katmanı (iş kuralı)
§19 Hata yönetimi	Merkezî, tutarlı, tiplere ayrılmış	NestJS Exception Filter
§20 Transaction ve eşzamanlılık	Tek sınır, iyimser kilitleme	<code>prisma.\$transaction</code> + <code>version</code> kolonu

Bölüm	Ne isteniyor	Nasıl karşılanıyor
§21 Audit	Merkezî audit, saat ve kullanıcı soyutlanmış	Prisma eklentisi + nestjs-cls + Clock
§22 React uygulaması	12 ekran, korumalı rota	Next.js + TanStack Query + shadcn/ui
§23 Testler	Unit, integration, architecture	Vitest + Testcontainers + dependency-cruiser
§24 Docker	Tek komutla dört servis	Docker Compose
§25 Health check ve loglama	İki sağlık ucu, yapılandırılmış log	Terminus + nestjs-pino
§26–27 Git ve CI	Anlamli geçmiş, otomatik kapılar	git + GitHub Actions / GitLab CI
§28–29 Dokümantasyon	AI_USAGE.md + sekiz doküman	README + docs/

Bu tablodaki her satırın ayrıntısı nerede: sistemin *ne yapacağı* Bölüm B'de madde madde, *hangi araçla yapıldığı* Bölüm C'deki teknoloji kartlarında, *hangi kavram ve desene dayandığı* ise Bölüm E'de anlatılıyor. Aynı açıklama iki kez yazılmadı; her konu tek yerde durur.

BÖLÜM B — Sistemin yapması istenen on iki şey

Çalışma dosyasının 2. bölümünde sistemin yapabilmesi gerekenler on iki madde hâlinde sayılıyor. Her maddede sırayla şunu anlatıyorum: **ödev hangi teknolojiyi bekliyordu, ben yerine ne kullandım, o teknoloji nedir ve bu projede hangi somut sorunu çözüyor.**

1) Kuruma ait lokasyonlar yönetilebilmelidir

Ödevin beklediği: ASP.NET Core Web API · FluentValidation · Entity Framework Core **Benim kullandığım:** NestJS · Zod · Prisma · PostgreSQL

Bu teknolojiler nedir:

- **PostgreSQL**, verinin kalıcı olarak durduğu yer. Excel'in çok daha güçlü hâli gibi düşünün: satırlar ve sütunlar var, ama kayıtlar birbirine bağlanabiliyor ve kuralları sistemin kendisi koruyor.
- **Prisma**, kod ile veritabanı arasındaki tercüman. "Aktif lokasyonları getir" diye yazıyorum, o bunu veritabanının anladığı dile çeviriyor.

- **NestJS modülü**, lokasyonla ilgili her şeyin (uç noktalar, kurallar, veri erişimi) tek klasörde toplanması demek.
- **Zod**, dışarıdan gelen verinin doğru biçimde olup olmadığını denetleyen bekçi.

Bu projede hangi sorunu çözüyor: Kurumun onlarca lokasyonu var ve bunlar zamanla değişiyor: yeni bina açılıyor, eski depo kapanıyor. Kapanan bir lokasyonu **silmek** doğru değil — geçmiş iş emirleri ona bağlı, silersek tarih bozulur. Bu yüzden silmiyoruz, **pasife alıyoruz**. Pasif lokasyonda yeni varlık veya yeni iş emri açılmıyor, ama geçmiş kayıtlar yerli yerinde duruyor.

Neden böyle seçtim: Lokasyon adının boş gönderilmesi ya da 500 karakterlik saçma bir metin girilmesi gibi durumları tek tek kontrol yazarak değil, Zod şemasıyla engelliyorum. Aynı şema hem tarayıcıdaki formu hem sunucuyu koruyor — kuralı iki kez yazmıyorum.

2) Lokasyonlara bağlı varlıklar yönetilebilmelidir

Ödevin beklediği: Entity Framework Core ilişkileri **Benim kullandığım:** Prisma ilişkileri + PostgreSQL foreign key

Bu nedir: İki tablo arasında **bağ** kurmak demek. Her varlık (jeneratör, klima, hizmet aracı) bir lokasyona ait.

Foreign key dediğimiz şey, veritabanının kendisinin koruduğu bir kural: "*olmayan bir lokasyona varlık bağlanamaz.*"

Bu projede hangi sorunu çözüyor: Kullanıcı arayüzünde bir hata olsa bile veritabanı yanlış veriyi **kabul etmiyor** — koruma tek katmanda değil. Ayrıca varlıkların kritiklik seviyesi burada tutuluyor; bu bilgi ileride SLA süresini belirlerken kullanılacak (madde 9).

Neden böyle seçtim: Bu tür bağları uygulama kodunda "önce kontrol et, sonra kaydet" diye yazmak yaygın bir hatadır: iki kullanıcı aynı anda işlem yaparsa kontrol ile kayıt arasında lokasyon silinebilir. Veritabanının kendi kuralı bu açığı bırakmıyor.

3) Varlıklar için arıza, bakım veya kontrol talepleri oluşturulabilmelidir

Ödevin beklediği: ASP.NET Core Web API (kapıyı açan) · FluentValidation (gelen veriyi denetleyen) · Entity Framework Core (veritabanına yazan) **Benim kullandığım:** NestJS · Zod · Prisma

Neden değiştirdim: Kurum stack'i serbest bıraktı. Üçünün de yaptığı işi JavaScript tarafında birebir karşılayan, piyasada yaygın ve aktif bakımda olan karşılıklarını seçtim. Yapılan iş aynı, sadece araçlar değişti.

Bu teknolojiler nedir:

NestJS, sunucu tarafında çalışan bir uygulama çatısı. "Çatı" demek, size hazır bir düzen vermesi demek: hangi kodun nereye yazılacağını, isteğin hangi sırayla işleneceğini o belirler. Bir ofisin organizasyon şeması gibi — kim ne yapar bellidir, herkes kendi kafasına göre çalışmaz. Bu projede 50'den fazla uç nokta olacağı için düzen kendiliğinden korunmalı.

Uç nokta (endpoint), uygulamanın dışarıya açtığı bir kapı: "buraya şu bilgileri gönderirsen sana şunu yaparım." Talep oluşturma kapısına form bilgileri gelir.

Zod, dışarıdan gelen verinin doğru biçimde olup olmadığını denetleyen bekçi. "Başlık boş olamaz", "açıklama en fazla 2000 karakter", "öncelik şu dört değerden biri olmalı" gibi kuralları tek yerde tanımlarsınız.

Prisma, kod ile veritabanı arasındaki tercüman.

Bu projede hangi sorunu çözüyor: Herkes talep açabilmeli — ama her talep kabul edilmemeli. Kullanım dışı bırakılmış bir jeneratör için arıza talebi açılması anlamsız; kapalı bir lokasyon için de öyle. Burada iki farklı denetim var ve bunları karıştırmamak önemli: "başlık boş mu" bir **biçim** sorusudur, Zod cevaplar. "Bu varlık için talep açılabilir mi" bir **iş kuralı** sorusudur, ayrı bir katmanda duruyor.

Neden bu ayrımı yaptım: İş kurallarını kapının içine yazmak kolay olurdu, ama aynı talep mobil uygulamadan veya arka plan işinden geldiğinde o kural çalışmazdı. Ayrı katmanda olduğu için kural **tek yerde yaşıyor** ve testi veritabanı gerektirmeden, saniyenin altında koşuyor.

4) Oluşturulan talepler iş emrine dönüştürülebilmelidir

Ödevin beklediği: EF Core transaction yönetimi **Benim kullandığım:** Prisma transaction (`$transaction`)

Bu nedir: Transaction, birbirine bağlı birden fazla işlemi "ya hepsi ya hiçbiri" mantığıyla yapmak demek. Yarım kalmasına izin verilmez.

Bu projede hangi sorunu çözüyor: Talep iş emrine dönüşürken aynı anda birkaç şey oluyor: iş emri kaydı açılıyor, ilk geçmiş kaydı yazılıyor, iş emri numarası üretiliyor. Hata çıksa bunların **yarısının** kaydedilmesi sistemi tutarsız bırakır — numarası olan ama geçmişi olmayan bir iş emri gibi. Transaction bunu imkânsız kılıyor.

Neden böyle seçtim: Çalışma §20'de bunu açıkça istiyor. Ayrıca her işleme gereksiz transaction açmıyorum — sadece gerçekten birbirine bağlı işlemlerde.

5) İş emirleri uygun personele atanabilmelidir

Ödevin beklediği: ASP.NET Core `[Authorize]` + rol bazlı yetkilendirme **Benim kullandığım:** NestJS Guard'ları + iş kuralı doğrulaması

Bu nedir: Guard, kapıdaki güvenlik görevlisi gibi: istek işleme girmeden önce "sen kimsin, buna yetkin var mı" diye bakar.

Bu projede hangi sorunu çözüyor: İki ayrı soru var ve karıştırılmamalı: "Bu kişi atama yapabilir mi?" bir yetki sorusudur, Guard cevaplar. "Atanan kişi teknik personel mi ve aktif mi?" bir iş kuralı sorusudur, kurallar katmanı cevaplar. Muhasebe personeline jeneratör tamiri atanamaması yetki meselesi değil, iş kuralı meselesidir.

Neden böyle seçtim: Bu ayrım yapılmazsa yetki kontrolleri iş kurallarının içine karışır ve altı ay sonra kimse hangisinin nerede olduğunu bulamaz.

6) İş emirlerinin durumları takip edilebilmelidir

Ödevin beklediği: Belirli bir teknoloji şart koşulmamış; sadece "dağılık koşul blokları olmasın" denmiş

Benim kullandığım: Durum makinesi (state machine) — sade bir kurallar tablosu

Bu nedir: Durum makinesi, "hangi durumdan hangi duruma geçilebilir" sorusunun tek bir yerde yazılı hâli. Trafik ışığı gibi: kırmızıdan yeşile geçilir, kırmızıdan sarıya geçilmez.

Bu projede hangi sorunu çözüyor: İş emri "Açık → Atandı → İşlemde → Çözüldü → Kapatıldı" yolunu izliyor. Atanmamış bir işin "İşlemde" olması ya da kapatılmış bir işin tekrar açılması kabul edilemez. Bu kontroller uç noktaların içine dağıtılmış `if` blokları olarak yazılırsa, yeni bir durum eklendiğinde hepsini tek tek bulmak gerekir — biri unutulur ve açık kalır.

Neden böyle seçtim: Çalışma §6 bunu özellikle vurguluyor: "Durum geçiş kurallarının controller içerisinde dağılık koşul bloklarıyla yönetilmesi beklenmemektedir." Tek tablo hâlinde tutunca hem okunuyor hem test ediliyor.

7) İş emirlerine yorum ve işlem kaydı eklenebilmelidir

Ödevin beklediği: EF Core `SaveChanges` üzerinden merkezî audit **Benim kullandığım:** Ayrı yorum tablosu + Prisma eklentisi ile otomatik audit

Bu nedir: Audit alanları, her kaydın "kim oluşturdu, ne zaman, kim güncelledi, ne zaman" bilgisi. **Prisma eklentisi** ise bu alanları elle yazmak yerine **otomatik dolduran** bir ara katman.

Bu projede hangi sorunu çözüyor: Bu bilgileri her servis içinde tek tek doldurmak hem sıkıcı hem tehlikeli — biri unutulunca o kaydın izi kaybolur. Merkezî bir yerden doldurunca unutma ihtimali ortadan kalkıyor.

Neden böyle seçtim: Çalışma §21 "audit alanlarının merkezî bir yaklaşımla yönetilmesi" istiyor. Ayrıca aktif kullanıcı bilgisini global bir değişkenden almıyorum — bu, iki kullanıcının verisinin karışmasına yol açan klasik hatadır.

8) İş emri geçmişi görüntülenebilmelidir

Ödevin beklediği: İşlem geçmişi tabloları (§11) **Benim kullandığım:** Ayrı geçmiş tabloları + transaction garantisi

Bu nedir: Her durum değişikliği ve her atama, ayrı bir tabloya **satır olarak** yazılıyor. Kaydın üstüne yazmak yerine yeni satır eklemek — defter tutmak gibi.

Bu projede hangi sorunu çözüyor: "Bu iş emri neden 3 gün bekledi?" sorusunun cevabı ancak geçmiş varsa verilebilir. Kurumsal bir sistemde bu bilgi denetim için gerekiyor ve yıllarca saklanıyor.

Neden böyle seçtim: Geçmiş kaydını uygulama loglarına yazmak yaygın bir hatadır — loglar birkaç hafta sonra silinir, oysa "bu kaydı kim değiştirdi" sorusu iki yıl sonra sorulur. Bu yüzden geçmiş **veritabanında** **tablo** ve durum değişikliğiyle **aynı transaction içinde** yazılıyor.

9) İş emirleri için SLA süreleri hesaplanabilmelidir

Ödevin beklediği: Factory Pattern (zorunlu tutulmuş) **Benim kullandığım:** Factory Pattern + NestJS bağımlılık enjeksiyonu

Bu nedir: **SLA**, işin ne kadar sürede tamamlanması gerektiğine dair söz. **Factory Pattern**, "duruma göre doğru hesaplayıcıyı seçen" bir tasarım deseni. **Bağımlılık enjeksiyonu**, sınıfların ihtiyaç duyduğu parçaları kendilerinin üretmesi yerine dışarıdan verilmesi.

Bu projede hangi sorunu çözüyor: SLA süresi tek bir sayı değil; işin önceliğine, varlığın kritiklik seviyesine ve iş emri türüne göre değişiyor. Kritik bir jeneratörün arızası 4 saatte, düşük öncelikli bir boya işi 15 günde çözülmeli. Bunu tek yerde iç içe **if** bloklarıyla yazarsak, yeni bir kural geldiğinde o bloğa dokunmak zorunda kalırız — ve her dokunuş mevcut kuralları bozma riski taşır.

Neden böyle seçtim: Her SLA kuralını **ayrı bir sınıf** yaptım. Yeni kural gelince yeni sınıf yazılıyor, mevcut kod **hiç değişmiyor**. Yazılımda buna Open/Closed prensibi deniyor: geliştirmeye açık, değiştirmeye kapalı. Çalışma §7 Factory Pattern'i zorunlu tutuyor ve "göstermelik olmasın" diye özellikle uyarıyor.

10) Yaklaşan ve geçen SLA süreleri sistem tarafından takip edilebilmelidir

Ödevin beklediği: Hangfire (gecikmeli job + recurring job) **Benim kullandığım:** BullMQ + Redis + ayrı worker süreci

Bu nedir: Normalde uygulama **isteğe cevap verir** — biri bir şey sorar, o cevaplar. Ama burada kimse bir şey sormasa da çalışması gereken işler var. **İş kuyruğu**, "şunu 2 saat sonra yap" veya "şunu her 15 dakikada bir yap" diyebildiğiniz bir yapılacaklar defteri. **Worker**, o defteri okuyup işleri yapan ayrı bir program.

Bu projede hangi sorunu çözüyor: SLA süresi dolmadan önce uyarı gitmeli, süre aşılınca iş emri ihlal olarak işaretlenmeli. Bunu kimse ekranı açmasa bile sistemin kendisi yapmalı — gece 3'te bile.

Neden böyle seçtim ve dikkat ettiğim nokta: Bu işlerin **iki kez çalışması** en büyük risk. Sunucu yeniden başlarsa veya iş yeniden denenirse aynı bildirim tekrar üretilebilir. Bunu iki katmanda engelliyorum: kuyrukta benzersiz iş anahtarı, veritabanında benzersiz index. Ayrıca işler yalnızca kimlik numarası alıyor, durumu **kendisi veritabanından okuyor** — iş emri bu arada kapanmıyorsa hiçbir şey yapmıyor.

11) Kullanıcılara sistem içi bildirim oluşturulabilmelidir

Ödevin beklediği: Bildirim tekrarlarını engelleyen yapılar (§11, §16) **Benim kullandığım:** Bildirim tablosu + benzersiz index + BullMQ

Bu nedir: Benzersiz index (unique index), veritabanına "bu ikili tekrar edemez" demenin yolu. Örneğin "şu kullanıcıya, şu olay için" bildirimi bir kez yazılabilir.

Bu projede hangi sorunu çözüyor: Aynı olay için kullanıcıya beş bildirim gitmesi, sistemin güvenilirliğini bitiren şeydir — insanlar bildirimlere bakmayı bırakır. Uygulama kodunda "önce var mı diye bak, yoksa ekle" yazmak yeterli değil: iki iş aynı anda çalışırsa ikisi de "yok" görüp ikisi de ekler. Veritabanının kendi kuralı bu yarışı kaybettirmiyor.

Neden böyle seçtim: Çalışma §15 ve §16 bunu ayrı ayrı vurguluyor. Koruma tek katmanda olsaydı, o katman atlandığında sessizce bozulurdu.

12) Yönetim ekranında temel operasyon istatistikleri gösterilebilmelidir

Ödevin beklediği: EF Core sorguları + React ekranı **Benim kullandığım:** Prisma toplama sorguları + Next.js + TanStack Query

Bu nedir: Toplama (aggregation) sorgusu, "kaç tane açık iş emri var" gibi sayı üreten sorgu. **TanStack Query**, tarayıcı tarafında veriyi getiren, saklayan ve tazeleyen kütüphane.

Bu projede hangi sorunu çözüyor: Yönetici ekranında açık iş emri sayısı, SLA ihlali sayısı, kritik iş sayısı ve personel bazında iş yükü görünüyor. Bu sayıları **veritabanına saydırıyorum** — tüm kayıtları çekip uygulamada saymıyorum. On bin kayıttan bu iki yaklaşım arasında saniyelerce fark oluyor.

Neden böyle seçtim: Çalışma §17 açıkça "bütün kayıtlar belleğe alındıktan sonra filtreleme yapılmamalıdır" diyor; aynı ilke sayma için de geçerli. Ayrıca günlük özet, her gece çalışan bir arka plan işiyle önceden hesaplanıp saklanıyor — yönetici ekranı açtığında ağır sorgu beklemiyor.

BÖLÜM C — Teknoloji kartları

Bu bölümde her teknoloji **tek tek ve kendi içinde yeterli** biçimde anlatılıyor. Bir kartı okurken başka bölüme dönmeniz gerekmez.

C.1 NestJS

Nedir: Node.js için sunucu tarafı uygulama çatısı. Express'in üstünde çalışır. **Ne işe yarar:** API uçlarını, iş kurallarını ve yetkilendirmeyi düzenli bir yapıda barındırır. Modül sistemi, bağımlılık enjeksiyonu (DI), Guard (yetki), Interceptor (log/denetim), Pipe (doğrulama), Filter (merkezî hata) getirir. **Bu projede nerede:** Tüm API (`apps/api`) ve arka plan işçisi (`apps/worker`). **Ödevdeki karşılığı:** ASP.NET Core Web API.

1. "Express'in üstünde çalışır" ne demek

Gerçek hayat: Boş bir dükkân kiralamak ile hazır kurulmuş bir zincir mağaza şubesi açmak. Boş dükkânda rafları nereye koyacağınız, kasanın nerede duracağı, deponun nasıl düzenleneceği size kalmıştır — ilk gün özgürlük, üçüncü yıl kaos. Zincir mağazada düzen önceden bellidir; yeni gelen çalışan hangi ürünün nerede olduğunu bilir.

Düz Express'in sorunu: Express size hiçbir kural koymaz. Klasör yapısını, kodları nereye yazacağınızı siz seçersiniz. Proje büyüdükçe her geliştirici kendi düzenini kurar.

NestJS çözümü: Express'in üzerine kurulmuş çelik bir iskelet. Express'in hızını arkada kullanmaya devam eder, size disiplinli bir mimari sunar.

⚠ Bu yüzden "Express mi Nest mi" diye bir seçim yoktur — Nest'i seçtiğinizde Express'i zaten almış olursunuz.

2. NestJS'in getirdiği araçlar — istek tüneli

Gelen bir istek, cevaba dönene kadar sıralı bir tünelden geçer. Her katman tek bir soruya bakar:

İstek gelir

- Guard "sen kimsin, yetkin var mı?" → hayırsa 401/403
- Pipe "gönderdiğin veri geçerli mi?" → hayırsa 400
- Controller "hangi işi istiyorsun?"
- Service iş kuralları + veritabanı
- Interceptor süreyi ölç, logla, cevabı biçimlendir
- Filter yolda hata çıktıysa yakala, düzgün cevaba çevir

Cevap döner

Bu projedeki karşılıkları:

```
// Bu sınıftaki tüm uçlar /work-orders adresi altında toplanıyor
@Controller('work-orders')
// İki bekçi: önce "giriş yapmış mı", sonra "yetkisi var mı" kontrol ediliyor.
// Bu satır sayesinde her metoda ayrı ayrı kontrol yazmak gerekmiyor.
@UseGuards(JwtAuthGuard, RolesGuard)
export class WorkOrdersController {

    // POST /work-orders/1042/assign adresine gelen istekleri bu metod karşılıyor
    @Post(':id/assign')
    // Yalnızca bu iki rol atama yapabilir. Teknik personel gelirse 403 alır.
    @Roles('ADMIN', 'OPERATIONS')
    assign(
        // Adresteki "1042" metnini gerçek sayıya çeviriyor.
        // Çeviremezse (örn. "abc") istek buraya hiç ulaşmadan 400 dönüyor.
        @Param('id', ParseIntPipe) id: number,

        // İsteğin gövdesindeki JSON. Zod şemasıyla doğrulanıyor;
        // eksik veya hatalı alan varsa metod hiç çalışmıyor.
        @Body() dto: AssignWorkOrderDto,
    ) {
        // Controller'da iş kuralı YOK — işi servise devredip cevabı döndürüyor
        return this.service.assign(id, dto.assigneeId);
    }
}
```

Dört aracın ne yaptığı:

- **Dependency Injection:** Bir sınıfın ihtiyaç duyduğu şeyi kendisi yaratmaz (`new PrismaService()`), Nest verir. Faydası testte görülür: testte gerçek yerine sahtesini verirsiniz, sınıf farkı anlamaz.
- **Guard:** Kapıdaki güvenlik. Yetkisizse istek servise **hiç ulaşmaz**.

- **Pipe:** Gelen veriyi kontrol eder ve dönüştürür. Yukarıdaki `ParseIntPipe` , URL'den gelen `"1042"` metnini sayıya çevirir; çeviremezse 400 döner ve kodunuz hiç çalışmaz.
- **Filter:** Kodun neresinde hata çıkarsa çıksın yakalar. `try/catch` yazmayı unutsanız bile uygulama çökmez, kullanıcı düzgün bir hata mesajı alır (E.7).

3. Projedeki yeri — iki uygulama, aynı çatı

- `apps/api` : Kullanıcıların istek attığı, hızlı cevap dönen ana sunucu.
- `apps/worker` : Zamanlanmış ve ağır işleri arka planda çözen süreç (C.6).

İkisi de NestJS olduğu için **aynı servisleri, aynı iş kurallarını ve aynı veritabanı katmanını** paylaşıyor. Worker'da bir iş emri kapatıldığında, API'den kapatıldığındaki kuralların **birebir aynısı** çalışıyor.

4. Ödevin §14 maddesi — servis yaşam döngüsü

Ödev, Transient / Scoped / Singleton kavramlarının doğru kullanıldığını görmek istiyor. Bunlar bellekteki nesnelerin **ne kadar yaşayacağını** belirler:

Nest	Ne demek	Bu projede
<code>DEFAULT</code> (singleton)	Uygulama boyunca tek kopya	Yapılandırma, <code>Clock</code> , SLA politikaları, factory
<code>REQUEST</code> (scoped)	Her istekte yeni , istek bitince silinir	İsteğe özel bağlam
<code>TRANSIENT</code>	Her kullanımda yeni	Kullanılmıyor — gereksiz kullanılmaz

⚠ **Neden hayati:** İstek bazlı veriyi singleton bir serviste tutarsanız iki kullanıcının verisi karışır (C.16'daki Ahmet/Mehmet örneği). Bu hata tek kullanıcı testi **hiç görünmez**, yük altında ortaya çıkar ve kurumsal bir sistemde yanlış kişinin verisini göstermek demektir.

Bu projede aktif kullanıcı bilgisi scoped servis yerine `nestjs-cls` ile taşınıyor (C.16) — daha güvenli ve daha performanslı.

Savunma cümlesi: "Düz Express'in kurlsızlığı yerine kurumsal mimari araçlarını hazır sunan NestJS'i seçtim. En önemlisi, ödevin 14. maddesinde istenen servis yaşam döngülerini Node tarafında eksiksiz uygulayabilen yaygın çatı Nest olduğu için bu tercih teknik olarak zorunluydu."

C.2 Next.js

Nedir: React'in framework'ü. **Ne işe yarar:** React tek başına bir web sitesi değildir; "veriye göre ekranı çiz" işini yapan bir kütüphanedir. Ayakta durması için **paketleyici** ve **yönlendirici (router)** gerekir. Next ikisini de içinde barındırır. **Bu projede nerede:** Tüm arayüz (`apps/web`) — 12 ekran. **Ödevdeki karşılığı:** React + Vite + React Router.

Ödevin istediği üç ayrı ürün neydi

Gerçek hayat: Bir mutfak kuruyorsunuz. Ocak, davlumbaz ve tezgâhı üç ayrı markadan alırsanız her biri iyi olabilir — ama ölçüleri tutturmak, birinin yeni modeli çıkınca diğerine uyup uymadığını takip etmek sizin işiniz olur. Hazır mutfak setinde bu uyumu **üretici garanti eder**.

Parça	Ne işi yapar	Onsuz ne olur
React	Arayüzü küçük parçalara (component) bölerek yazmayı sağlar. Veri değiştiğinde tüm sayfayı değil yalnızca değişen kısmı günceller	Ekran çizilemez
Vite	Kod yazarken tarayıcıyı anında günceller; yayına çıkarken kodu tarayıcının anlayacağı, sıkıştırılmış hâle çevirir	Tarayıcı TypeScript ve JSX'i anlamaz
React Router	Adres → ekran eşlemesi. <code>/is-emirleri/42</code> adresine hangi bileşenin geleceğini yönetir, sayfa yenilenmeden geçiş sağlar	Uygulama tek ekrandan ibaret kalır

Next.js neden üçünün yerine geçiyor

Next.js zaten **React'i barındırır**; diğer ikisinin işini kendi içine gömülü mimarilerle çözer:

Vite'in yerine kendi derleyicisi. Geliştirme sunucusu ve yayın derlemesi aynı araçtan gelir.

React Router'ın yerine dosya sistemi tabanlı yönlendirme. Klasör yapısının kendisi rotadır — ayrıca rota tablosu yazılmaz:

```
apps/web/app/
├ (auth)/login/page.tsx      → /login          · giriş ekranı
├ (protected)/
│   ├── dashboard/page.tsx   → /dashboard
│   ├── is-emirleri/
│   │   ├── page.tsx         → /is-emirleri      (liste ekranı)
│   │   ├── yeni/page.tsx    → /is-emirleri/yeni (yeni kayıt formu)
│   │   └ [id]/
│   │       ├── page.tsx     → /is-emirleri/1042 (detay ekranı)
│   │       └ duzenle/page.tsx → /is-emirleri/1042/duzenle
│   ├── lokasyonlar/page.tsx → /lokasyonlar
│   └ bildirimler/page.tsx   → /bildirimler
└ not-found.tsx             → bulunamayan adreslerde açılan 404 ekranı
```

Parantezli klasörler – (auth) ve (protected) – adreste GÖRÜNMEZ.

Yalnızca gruplama yaparlar: (protected) altındaki tüm sayfalar

tek bir yerde yazılan oturum kontrolünden geçer.

★ Parantezli klasörler (`(auth)` , `(protected)`) **adrese girmez**; yalnızca grupta yaparlar. Bu projede işe yarayan yanı şu: `(protected)` altındaki tüm sayfalar **tek bir yerde** yazılan oturum kontrolünden geçiyor. Ödevin §22'de istediği "*protected route kullanılmalıdır*" maddesi böylece her sayfaya ayrı kontrol yazmadan karşılanıyor.

Üstüne ikisinin de vermediklerini verir: sunucuda render (ilk açılış hızı), görsel optimizasyonu, yerleşik önbellekleme.

Dürüst tarafı

Küçük ve tamamen istemci tarafı bir panel için Vite + React Router daha hafif ve öğrenmesi daha kolaydır. Next.js daha kuralcıdır. Bu projede Next seçildi çünkü sistem uzun ömürlü olacak ve **üç ayrı paketin sürüm uyumunu yıllarca elle takip etmek**, projeyi devralacak geliştiriciyi zorlar.

⚠ **Önemli:** React yine kullanılıyor. Next.js React'in framework'ü olduğu için ödevin "*React ile frontend geliştirme*" şartı doğrudan karşılanıyor.

C.3 Prisma

Nedir: ORM — veritabanıyla konuşmayı yapan katman. **Ne işe yarar:** Ham SQL yerine temiz kod yazmanızı sağlar, veritabanı şemasını yönetir ve tip güvenliği verir. **Bu projede nerede:** Yalnızca NestJS tarafında. Next.js Prisma'yı **hiç görmez** — veritabanına tek kapıdan girilir. **Ödevdeki karşılığı:** Entity Framework Core.

Prisma nerede duruyor

API (NestJS) dış dünyaya veri sunan kapımızsa, **Prisma** o kapının arkasında PostgreSQL'e gidip veriyi hızlı, hatasız ve TypeScript uyumlu şekilde çekip getiren işçidir.

Prisma kullanmasaydık, API fonksiyonunun içine uzun uzun SQL kodunu elle yazıp çalıştırmak gerekirdi. Prisma sayesinde SQL yerine **temiz ve okunabilir kod** yazıyoruz; o kodu arkada veritabanının anladığı SQL'e kendisi çeviriyor.

Prisma'nın dört kritik görevi

1. **Tür güvenliği (type safety)** `where` , `id` , `isActive` gibi alanları yazarken kod editörü otomatik tamamlamalar sunar. Yanlış bir harf yazarsanız kod anında uyarır — yani hatayı **projeyi çalıştırmadan önce**, derleme aşamasında görürsünüz.

2. **SQL yazmadan veri çekmek ve ilişkileri yönetmek** Karmaşık SQL sorguları yazmak zorunda kalmazsınız. Prisma, yazdığınız TypeScript kodunu arka planda en optimize SQL sorgusuna dönüştürür.

3. **Şemayı tek dosyada toplamak** `schema.prisma` , Prisma'nın kalbidir. Tüm veritabanı mimarisini tek yerde topladığınız, okunması kolay bir dosyadır. Üç şeyi tanımlar:

- **Datasource:** Hangi veritabanını kullandığınız ve adresi

- **Generator:** Hangi dil için kod üretileceği
- **Models:** Tablolar, kolonlar, veri tipleri ve tablolar arası ilişkiler

```
// 1) HANGİ VERİTABANI – bağlantı adresi koda yazılmıyor,  
// ortam değişkeninden okunuyor (local ile canlı farklı olsun diye)  
datasource db {  
  provider = "postgresql"  
  url      = env("DATABASE_URL")  
}  
  
// 2) NE ÜRETİLECEK – Prisma bu şemadan TypeScript kodu üretiyor  
generator client {  
  provider = "prisma-client-js"  
}  
  
// 3) TABLOLAR  
model Location {  
  id      Int      @id @default(autoincrement()) // birincil anahtar, otomatik artar  
  name    String   @unique                       // iki lokasyon aynı adı alamaz  
  isActive Boolean @default(true)                // pasife alınca false olur  
  assets  Asset[]                               // bu lokasyondaki varlıklar  
}  
  
model Asset {  
  id      Int      @id @default(autoincrement())  
  name    String  
  locationId Int                               // hangi lokasyona ait  
  // İki tabloyu bağlayan tanım: locationId, Location tablosundaki id'yi gösteriyor  
  location Location @relation(fields: [locationId], references: [id])  
}
```

★ **Devralınabilirlik açısından belirleyici olan nokta:** Entity Framework Core'da veri modelini görmek için 30 sınıf dosyası gezersiniz. Prisma'da tek dosya açarsınız, tüm model önünüzdedir.

4. Veritabanı değişikliklerini (migration) yönetmek Yeni tablo eklemek veya kolon değiştirmek gerektiğinde Docker konteynerine girip elle SQL çalıştırmazsınız. Akış şu:

1. `schema.prisma` dosyasında değişikliği yaparsınız
2. `npx prisma migrate dev` komutunu çalıştırırsınız
3. Prisma otomatik olarak `CREATE TABLE ...` gibi SQL üretir, PostgreSQL'e uygular ve bu değişikliği bir **geçmiş dosyası** olarak kaydeder

Bu geçmiş dosyaları git'e girer. Siz, DevOps ve CI **aynı SQL'i** çalıştırır; "bende tablo var sende yok" durumu oluşmaz.

⚠️ Zod ile Prisma ilişkisi — yön önemli

Yaygın bir yaklaşım, Prisma modellerinden **otomatik Zod şeması üretmektir** (`zod-prisma` gibi araçlar). **Bu projede bilerek bunu yapmıyoruz.**

Sebebi: o yön, API sözleşmenizi veritabanı şemanız hâline getirir. Kullanıcı tablosunda `passwordHash` varsa üretilen şemada da olur ve dışarı sızma riski doğar. Ödev §13 tam olarak bunu yasaklıyor: "*Entityler doğrudan API response olarak dönülmemelidir.*"

Bizim yönümüz tersi: Şemayı `packages/contracts` içinde Zod ile yazıyoruz, Prisma'nın `select` ifadesini o şemadan türetiyoruz. Böylece veritabanı yalnızca cevabın ihtiyaç duyduğu kolonları çekiyor ve çıkışta fazla alan kırılıyor. Sözleşme veritabanına değil, **ihtiyaca** göre şekilleniyor.

Özet mimari akış

```
[ Tarayıcı ]
  ↓
[ Next.js Frontend ]
  ↓ (API çağrısı)
[ NestJS Backend ]
  ↓ (Prisma Client)
[ Docker — PostgreSQL ]
```

Prisma burada NestJS'in veritabanıyla konuşan dili hâline gelir. Docker ise veritabanının bilgisayarınızda temiz bir kutu içinde stabil çalışmasını sağlar.

C.4 Zod

Nedir: Şema tanımlama ve doğrulama kütüphanesi. **Ne işe yarar:** Bir veri şeklini **bir kez** tanımlarsınız; o tanımdan hem çalışma anı doğrulaması, hem TypeScript tipi, hem OpenAPI dokümanı üretilir. **Bu projede nerede:** Her API girişinde doğrulama · `packages/contracts` içinde paylaşılan tipler · her API çıkışında fazla alanın kırılması · frontend form doğrulaması (aynı şema). **Ödevdeki karşılığı:** FluentValidation — ama o yalnızca doğrular; tip ve OpenAPI ayrı iştir.

Zod'un rolü

Zod, bu mimaride "**dışarıdan gelen verinin güvenlik muhafızı**" rolündedir.

Prisma veritabanından çıkan verinin tipini (**içeri**yi) korurken; Zod, kullanıcının formdan gönderdiği veya dışarıdan gelen verinin (**dışarı**yı) kurallara uygun olup olmadığını kontrol eder.

Akış — Zod tam olarak nerede devreye giriyor

⚠️ Zod ayrı bir servis veya sunucu **değildir**. NestJS'in **içindeki bir katmandır** (ValidationPipe). Yani kapıda ayrı bir bina yok, kapının içinde bir bekçi var:

```
[ Kullanıcı formu ]
    ↓
[ NestJS ]
├─ 1. Zod katmanı → "biçim doğru mu?" (hayırsa istek burada biter)
├─ 2. Guard      → "yetkin var mı?"
├─ 3. İş kuralları → "bu işlem yapılabilir mi?"
└─ 4. Prisma     → veritabanına yaz
    ↓
[ Docker — PostgreSQL ]
```

Zod katmanı: E-posta gerçekten @ içeriyor mu? Başlık boş mu? Öncelik tanımlı dört değerden biri mi? Veri onaylanmazsa istek **kod içine bile girmeden** "hatalı format" cevabıyla geri döner.

Prisma katmanı: Zod'dan temiz çıkan güvenli veri Prisma'ya teslim edilir, o da SQL'e çevirip kaydeder.

İki farklı denetimi karıştırmamak

Soru	Kim cevaplar
"Başlık boş mu, 2000 karakteri aştı mı?" — biçim	Zod
"Bu varlık için talep açılabilir mi?" — iş kuralı	Kurallar katmanı

Bu ayırım yapılmazsa iş kuralları doğrulama koduna karışır ve aynı kural mobil uygulamadan gelen istekte çalışmaz.

C.5 PostgreSQL

Nedir: İlişkisel veritabanı. **Ne işe yarar:** Veriyi tablolarda tutar; ilişkileri ve tutarlılığı **kendisi** garanti eder (foreign key, unique constraint, transaction). **Bu projede nerede:** Tüm veri. Ayrıca metin araması **pg_trgm** + GIN index ile veritabanının içinde çözülüyor — ayrı arama motoru gerekmiyor. **Ödevdeki karşılığı:** Aynı; ödev zaten PostgreSQL istiyordu.

PostgreSQL verileri Excel tabloları gibi satır ve sütunlarda tutar. En büyük gücü ise bu tablolar arasındaki ilişkileri ve kuralları **asla esnetmeden** korumasıdır.

Foreign key (yabancı anahtar)

Gerçek hayat: Nüfus müdürlüğü, üzerine kayıtlı aracı olan bir kişiyi sistemden silmenize izin vermez. Önce aracı devredersiniz, sonra kaydı kapatırsınız. Kural memurda değil, **sistemin kendisinde** olduğu için kimse "acelem var" diye atlayamaz.

Bu projede:

```
model Asset {
    id          Int      @id @default(autoincrement()) // varlığın kendi numarası

    // Bu varlığın hangi lokasyona ait olduğunu tutan alan
    locationId Int

    // İki tabloyu birbirine bağlayan tanım.
    // 🚫 onDelete: Restrict → üzerinde varlık olan bir lokasyon SİLİNEZ.
    //   Koruma uygulama kodunda değil, veritabanının kendisinde duruyor.
    location Location @relation(fields: [locationId], references: [id],
                                onDelete: Restrict)
}
```

onDelete: Restrict sayesinde, üzerinde varlık olan bir lokasyon silinemez. Bu koruma **uygulama kodunda değil, veritabanında** durur — arayüzde veya API'de hata olsa bile veri bozulmaz.

⚠ Bu yüzden zaten lokasyonları **silmiyoruz**, pasife alıyoruz (E.9 → soft delete). Foreign key, kazara silmeye karşı ikinci savunma hattı.

Unique constraint (benzersizlik kuralı)

Gerçek hayat: Aynı koltuğa iki bilet satılamaz. Gişе memuru dalgın olsa bile sistem ikinci bileti basmaz.

Bu projede en kritik kullanımı — mükerrer bildirim engelleme:

```
model Notification {
    id          Int      @id @default(autoincrement()) // bildirimin kendi numarası
    userId      Int      // bildirim kime gidecek
    eventKey     String   // hangi olay: "SLA_BREACH:1042"

    // 🚫 Korumanın kalbi: aynı kullanıcıya aynı olay için İKİNCİ bir bildirim
    //   yazılamaz. Kural uygulama kodunda değil, veritabanının kendisinde.
    @@unique([userId, eventKey])
}
```

Neden uygulama kodunda kontrol yetmiyor:

```
// 🚫 YANLIŞ YOL: önce bak, yoksa ekle

// 1) "Bu bildirim daha önce yazılmış mı?" diye soruyoruz
const varMi = await prisma.notification.findFirst({ where: { userId, eventKey } });

// 2) Yazılmamışsa ekliyoruz
if (!varMi) await prisma.notification.create({ ... });

// ⚠️ İki arka plan işi aynı anda 1. satırı çalıştırırrsa İKİSİ DE "yok"
//      cevabı alır ve ikisi de ekler. Kullanıcı aynı olay için iki bildirim görür.
```

İki arka plan işi aynı anda kontrol ederse ikisi de "kayıt yok" görür ve ikisi de ekler. Buna **yarış koşulu (race condition)** denir. Veritabanının kuralı bu yarış yapısal olarak kaybettirir: ikincisi hata alır, kod onu sessizce yutar.

Transaction (işlem bütünlüğü)

"Ya hep ya hiç" kuralıdır. Birbirine bağlı işlemler ya tamamen olur ya hiç olmaz.

Bu projedeki somut senaryo:

```
// "Ya ikisi birden ya hiçbiri" garantisi
await prisma.$transaction(async (tx) => {

  // 1) İş emrini kapalı olarak işaretle
  await tx.workOrder.update({ where: { id }, data: { status: 'CLOSED' } });

  // 2) Aynı işlemde geçmişe de yaz: kim, nereden nereye aldı
  await tx.workOrderHistory.create({
    data: { workOrderId: id, from: 'RESOLVED', to: 'CLOSED', byUserId },
  });

  // Arada hata çıkarsa ikisi de geri alınır. Böylece "kapatılmış ama
  // kim kapattığı belli olmayan" bir kayıt asla oluşmaz.
});
```

Transaction olmasaydı ve tam ikisi arasında hata çıksaydı: iş emri "Kapatıldı" görünürdü ama **kimin, ne zaman kapattığı kaydı olmazdı**. Denetim açısından bu, kaydın hiç olmamasından kötüdür — sistem doğru görünür ama izi yoktur.

⚠ **Transaction'ın sınırı.** Bir PostgreSQL transaction'ı yalnızca **kendi veritabanındaki** işlemleri geri alabilir. Farklı bir kurumun sistemine (örneğin bir bankaya) gönderilmiş işlemi geri alamaz; orada **telafi işlemi** (iade) devreye girer ve buna *saga* denir. Bu projedeki tüm transaction senaryoları tek veritabanı içinde kaldığı için tam garanti sağlanıyor.

pg_trgm + GIN index ile arama

Problem: İş emri listesinde metin araması var. `WHERE title LIKE '%pompa%'` yazarsanız PostgreSQL tüm tabloyu satır satır tarar. 100 kayıta fark etmez, 500 bin kayıta ekran donar.

Index nedir: Kitabın arkasındaki dizin. Ama varsayılan index (B-tree) "şununla **başlayanları**" bulmakta hızlıdır; **ortasında geçenleri** bulamaz — yani `%pompa%` aramasında hiç işe yaramaz.

Çözüm iki parçalı:

- **pg_trgm** : Kelimeleri üçer harflik parçalara böler ("pompa" → "pom", "omp", "mpa"). Böylece "ortada geçiyor mu" sorusu "şu parçaları içeriyor mu" sorusuna dönüşür. Yazım hatasını da tolere eder ("jeneratör" / "jenerator").
- **GIN index:** Bu parçaları aranabilir hâle getiren index türü.

```
-- 1) PostgreSQL'in kelime parçalama eklentisini aç (yoksa kur).
-- Bu eklenti "pompa" kelimesini "pom","omp","mpa" parçalarına bölüyor.
CREATE EXTENSION IF NOT EXISTS pg_trgm;

-- 2) İş emri başlıkları için arama index'i oluştur.
-- GIN = bu parçaları hızlı aramaya uygun index türü.
-- Bu index olmadan "%pompa%" araması tüm tabloyu satır satır tarardı.
CREATE INDEX work_order_title_trgm_idx
ON "WorkOrder" USING GIN (title gin_trgm_ops);
```

★ **Kazanç:** Normalde bu iş için Elasticsearch gibi ayrı bir arama motoru kurulur — ayrı sunucu, ayrı bakım, ayrı maliyet. Burada PostgreSQL'in kendi yetenekleriyle çözülüyor.

Ne zaman yetmez: Alaka sıralaması ve çok dillilik gereken gerçek tam metin aramada PostgreSQL FTS veya ayrı motor gerekir — bu proje için gereksiz.

C.6 BullMQ + Redis

Nedir: Node için iş kuyruğu; verilerini Redis'te tutar. **Ne işe yarar:** **Gecikmeli** ("2 saat sonra hatırlat") ve **tekrarlayan** ("her 15 dakikada bir tara") işleri yönetir. Hata alırsa otomatik yeniden dener. **Bu projede nerede:** Ödevin dört zamanlanmış işi. **Ödevdeki karşılığı:** Hangfire.

Neden gerekli

Gerçek hayat: Bir restoranda garson siparişi alıp mutfağa asıyor ve **masaya dönüyor**. Yemeğin pişmesini mutfağın önünde beklemiyor — beklerse diğer masalar hizmet alamaz.

Yazılımda: API isteğe cevap verir ve biter. Ama bazı işler kimse bir şey sormasa da çalışmalı: gece 03:00'te SLA'sı geçmiş iş emirlerini taramak gibi. Bu işleri API içinde yaparsanız, o sırada gelen kullanıcı istekleri bekler.

Bileşenler

- **BullMQ:** İşleri sıraya koyar, zamanı geleni dağıtır, hata alanı yeniden dener.
- **Redis:** Diske değil doğrudan belleğe yazan çok hızlı veri deposu. Kuyruk burada durur.

Bu kartın iki eki: yönetim ekranı ve hız sınırı

Bull Board — kuyruğun **yönetim ekranı**. Hangi işler bekliyor, hangisi başarısız oldu, kaç kez denendi; bir işi elle yeniden çalıştırma. ★ Ödevin istediği **Hangfire Dashboard'un birebir karşılığıdır**; sorulduğunda gösterilecek somut karşılık budur.

🚫 **Korumasız açılmaz.** Bull Board bütün iş verisini gösterir ve işleri tetikleyebilir. Yalnızca **yönetici** rolüyle erişilir; canlıda kapatılabilir olması `.env` üzerinden ayarlanır.

`@nestjs/throttler` — **hız sınırı** (rate limiting). Aynı adresten gelen istekleri sayar ve tavanı aşarsa **429 Too Many Requests** döner.

Nerede	Neden
Giriş ucu (<code>/auth/login</code>)	🚫 Brute-force koruması — şifre deneyen botu durdurur
Genel API	Tek istemcinin sistemi doldurmasını engeller

Sayaç **Redis'te** tutulur — çünkü birden fazla API kopyası çalıştığında (§12 ölçekleme) her kopyanın kendi sayacı olsaydı sınır kopya sayısı kadar katlanırdı. .NET karşılığı `AspNetCoreRateLimit` .

⚠️ Redis bu projede yalnızca kuyruk için değil, **hız sınırı sayacı** ve **dağıtık kilit** için de kullanılıyor — tek amaçlı bir bağımlılık değil.

İki iş türü — kodla

```
// GECİKMELİ İŞ – "şimdi değil, ileri bir saatte çalış"
await queue.add(
  'sla-reminder', // işin adı
  { workOrderId: 1042 }, // işe verilen tek bilgi: KİMLİK NUMARASI
  // (iş emrinin tamamı gönderilmiyor –
  // kuyrukta beklerken eskiyebilirdi)
  {
    // Kaç milisaniye sonra çalışsın. Hatırlatma zamanına kalan süre.
    delay: remindAt.diffNow().milliseconds,

    // İşin benzersiz kimliği. Aynı kimlikle ikinci kez eklenirse
    // kuyruk onu kabul etmiyor – mükerrer hatırlatma olmuyor.
    jobId: `sla-reminder-1042`,

    attempts: 3, // hata alırsa 3 kez dene
    backoff: { type: 'exponential', delay: 5000 }, // her denemede daha uzun bekle
  },
);

// TEKRARLAYAN İŞ – kimse tetiklemese de düzenli çalışır
await queue.add('sla-breach-scan', {}, {
  // Cron biçimi: "her 15 dakikada bir"
  repeat: { pattern: '*/*15 * * * *' },
});
```

Projedeki dört iş

İş	Türü	Ne yapar
SLA hatırlatma	Gecikmeli	Süre dolmadan önce ilgili personele uyarı üretir
SLA ihlali taraması	Tekrarlayan	Süresi geçmiş açık iş emirlerini bulur, ihlal işaretler
Günlük operasyon özeti	Tekrarlayan	Gece bir kez: açık iş sayısı, ihlaller, personel iş yükü
Arşiv adayı belirleme	Tekrarlayan	Eski kapalı kayıtları arşivlenebilir işaretler

En kritik kural: idempotency (aynı iş iki kez çalışsa da sonuç değişmez)

Gerçek hayat: Asansör düğmesine beş kez basmak asansörü beş kez çağırılmaz. Sistem "zaten çağırıldı" durumunu biliyor.

Neden risk: Sunucu yeniden başlarsa veya iş hata alıp yeniden denenirse aynı iş ikinci kez çalışır. Önlem alınmazsa kullanıcı **iki bildirim** alır.

Bu projede üç katmanlı korunuyor:

```
// Bu iş saatler önce kuyruğa bırakılmıştı. Şimdi çalışıyor.
async handleSlaBreach(workOrderId: number) {

  // 1) İş emrinin GÜNCEL hâlini veritabanından oku.
  //   Kuyruğa girerken gönderilen bilgiye güvenmiyoruz – o bilgi eskimiş olabilir.
  const wo = await this.prisma.workOrder.findUnique({ where: { id: workOrderId } });

  // 2) Bu arada iş kapanmış veya iptal edilmişse hiçbir şey yapma, sessizce çık
  if (!wo || wo.status === 'CLOSED' || wo.status === 'CANCELLED') return;

  // Zaten ihlal olarak işaretlenmişse tekrar işaretleme (iş ikinci kez çalışmış olabilir)
  if (wo.slaBreached) return;

  // 3) İşaretleme ve bildirim BİRLİKTE yazılıyor.
  //   Bildirim tablosundaki benzersiz kural, aynı olay için ikinci
  //   bildirimin yazılmasını veritabanı seviyesinde engelliyor.
  await this.prisma.$transaction(async (tx) => {
    await tx.workOrder.update({ where: { id: wo.id }, data: { slaBreached: true } });
    await tx.notification.create({
      data: { userId: wo.assigneeId, eventKey: `SLA_BREACH:${wo.id}` },
    });
  });
}
```

★ **1. maddedeki kural en önemlisi:** İş, parametre olarak **yalnızca kimlik numarası** alıyor; iş emrinin durumunu kendisi veritabanından okuyor. Nesnenin tamamı parametre olarak geçirilseydi, iş kuyrukta beklerken durum değişmiş olabilirdi ve iş **eskimiş veriyle** karar verirdi.

Ödev §15 bunu doğrudan söylüyor: "*Job parametresi olarak büyük nesneler yerine tanımlayıcı değerler kullanılmalıdır.*"

Dashboard güvenliği

Ödev "*Hangfire Dashboard yetkisiz erişime açık bırakılmamalıdır*" diyor. Bull Board arayüzü de aynı şekilde yalnızca yönetici rolüne açık — Guard arkasında.

C.7 Testcontainers

Nedir: Test başlarken Docker üzerinde **gerçek** bir veritabanı konteyneri ayağa kaldırır, test bitince silen kütüphane. **Bu projede nerede:** Tüm entegrasyon testleri. **Ödevdeki karşılığı:** Ödevin "*EF Core InMemory provider kullanılmamalıdır*" maddesinin olumlu karşılığı.

Neden sahte veritabanı kullanılmıyor

Gerçek hayat: Uçuş simülatöründe pilot mükemmel iniş yapıyor. Ama simülatörde yan rüzgâr, buzlanma ve motor arızası modellenmemişse, o pilotun "iniş yapabildiği" **kanıtlanmış olmaz**. Test yeşil yanar, gerçek uçuşta sorun çıkar.

Yazılımda: Sahte (in-memory) veritabanı gerçek bir SQL motoru değildir — bellekteki bir sözlüktür. Şunları uygulamaz:

Test edilmek istenen	Sahte veritabanında
Unique constraint ihlali	Yakalanmaz — ikinci kayıt yazılır
Foreign key ihlali	Yakalanmaz
Transaction geri alma	Gerçekten geri almaz
Eşzamanlılık çakışması	Kavram yok
<code>pg_trgm</code> metin araması	Çalışmaz

Ödev §23 tam da bu beşini test etmeyi istiyor. Yani sahte veritabanıyla **testler yeşil yanar ama hiçbir şey ölçmez** — üstelik yanlış bir güven verir.

⚠ Bizim stack'teki aynı tuzak: **Prisma Client'ı mock'lamak**. Yapmıyoruz.

Nasıl çalışıyor

```
let container: StartedPostgreSqlContainer;

// beforeAll = "testler başlamadan önce bir kez çalış"
beforeAll(async () => {
  // 1) Docker'da gerçek bir PostgreSQL konteyneri başlat
  container = await new PostgreSQLContainer('postgres:18-alpine').start();

  // 2) Uygulamaya "veritabanının burada" de.
  //   Adres rastgele bir porta çıkıyor, bilgisayardaki diğer
  //   veritabanlarıyla çakışmıyor.
  process.env.DATABASE_URL = container.getConnectionUri();

  // 3) Tabloları oluştur – gerçek migration dosyaları çalışıyor
  execSync('npx prisma migrate deploy');
}, 60_000); // konteyner inmesi uzun sürebilir, 60 saniye süre tanı

// afterAll = "tüm testler bittikten sonra çalış"
afterAll(async () => {
  // Konteyneri kapat ve sil. Bilgisayarda hiçbir kalıntı bırakma.
  await container.stop();
});
```

Dört adım:

1. **Konteyner ayağa kalkar.** Belirtilen imaj indirilir, boş bir veritabanı başlar.
2. **Rastgele port atanır.** Bilgisayarda çalışan başka bir PostgreSQL varsa çakışma olmaz — bu projede zaten paralel bir proje çalışıyor, dokunulmuyor.
3. **Testler gerçek motora karşı koşar.** Migration'lar uygulanır; benzersizlik, foreign key ve transaction canlıdaki gibi davranır.
4. **Her şey silinir.** Test başarılı da olsa başarısız da olsa konteyner kapanır; artık veri kalmaz.

Bu projede test edilen senaryolar

```
it('aynı olay için ikinci bildirim yazılamaz', async () => {
  // 1) Birinci bildirimi yaz – bu başarılı olmalı
  await createNotification(userId, 'SLA_BREACH:1042');

  // 2) Aynı bildirimi tekrar yazmayı dene – bu HATA VERMELİ
  await expect(
    createNotification(userId, 'SLA_BREACH:1042'),
  ).rejects.toThrow(/Unique constraint/);
  // Gelen hata gerçek PostgreSQL'in benzersizlik hatası.
  // Sahte bir veritabanında bu kural hiç uygulanmaz, test yeşil yanar
  // ve koruma hiç doğrulanmamış olurdu.
});
```

Bu test sahte veritabanında **her zaman geçerdi** ve mükerrer bildirim koruması hiç doğrulanmamış olurdu.

★ **Koruma testi disiplini:** Bir korumayı test eden test yazıldığında, koruma geçici olarak kaldırılıp testin **kırmızıya döndüğü gözle görülür**. Dönüyorsa test korumayı değil başka bir şeyi ölçüyordur.

C.8 dependency-cruiser

Nedir: Kod içindeki `import` ilişkilerini kurallara göre denetleyen otomatik mimari denetim aracı. **Bu projede nerede:** Ödev §23'ün mimari testleri. **Ödevdeki karşılığı:** .NET tarafındaki NetArchTest.

Çözdüğü sorun

Gerçek hayat: Bir binada yangın kapısı var ve "sürekli kapalı tutulacak" yazıyor. Ama kapı ağır olduğu için biri altına takoz koyuyor — herkes o kapıyı kullanmaya başlıyor. Uyarı levhası duruyor, **kural fiilen ölmüş**.

Kuralı yaşatan şey levha değil, kapının **kendiliğinden kapanan mekanizmasıdır**.

Yazılımda: "*Domain katmanı veritabanını tanımaz*" kuralı dokümana yazılır, herkes başta uyar. Sonra biri acele eder, domain'e bir Prisma import'u koyar — çalışır, kimse fark etmez. Altı ay sonra domain katmanı Prisma'sız çalışmaz hâle gelmiştir ve mimari kâğıt üstünde kalmıştır.

Nasıl çalışıyor

Projenin başında kural dosyası yazılır:

```
// Dosya: .dependency-cruiser.js – mimari kuralların yazılı hâli
forbidden: [
  {
    name: 'domain-altyapiya-bagimli-olamaz',
    severity: 'error', // ihlal = hata, uyarı değil
    // KİM: domain klasöründeki dosyalar
    from: { path: '^packages/domain' },
    // NEYİ ÇAĞIRAMAZ: veritabanı aracı, Nest çatısı veya API klasörü
    to: { path: '(node_modules/@prisma|@nestjs|^apps/api)' },
  },
  {
    name: 'web-veritabanina-erisemez',
    severity: 'error',
    // KİM: arayüz projesi
    from: { path: '^apps/web' },
    // NEYİ ÇAĞIRAMAZ: veritabanı aracını doğrudan
    to: { path: 'node_modules/@prisma' },
    // Sebep: veritabanına yalnızca API üzerinden gidilmeli.
  },
]
]
```

Sonra CI'da çalışır:

```
✗ domain-altyapiya-bagimli-olamaz
  packages/domain/work-order.ts → node_modules/@prisma/client
error Process exited with code 1
```

Build kırmızı yanar, o kod ana dala giremez.

Bu projede denetlenen kurallar

Kural	Neden
Domain, Prisma ve NestJS'i import edemez	İş kuralları altyapıdan bağımsız kalsın (E.1)
Application, infrastructure'a doğrudan bağlanamaz	Bağımlılık yönü korunsun
<code>apps/web</code> , Prisma'yı import edemez	Veritabanına tek kapıdan girilsin
Controller, Prisma'yı doğrudan çağıramaz	İş kuralı atlanmasın
Modüller birbirinin iç dosyasına erişemez	Modül sınırı korunsun (E.1)

Neden bu araç

Katman kuralları sözlü kararlar alınır veya yalnızca dokümana yazılırsa altı ay içinde çiğnenir; projeye yeni katılan biri dokümanı okumamış olabilir.

Kural ancak kapıya dönüşürse yaşar.

Ödevin §23'te istediği "*katman bağımlılıklarını doğrulayan testler*" maddesi tam olarak budur — ve bu, dokümantasyonda anlatılan değil **kod seviyesinde zorlanan** bir mimari demektir.

C.9 TypeScript

Nedir: JavaScript'in üzerine **tip kontrolü** eklenmiş hâli. **Bu projede nerede:** Her yerde — backend, frontend, testler, ortak paketler. **Ödevdeki karşılığı:** C#.

Çözdüğü gerçek sorun

Gerçek hayat: Bir formda "doğum tarihi" alanına biri `15/03/1990`, biri `1990-03-15`, biri `15 Mart 1990` yazıyor. Form bunu kabul ediyor çünkü hepsi "metin". Hata, o veriyle yaş hesaplamaya çalıştığınızda — yani **çok sonra** ortaya çıkıyor.

Yazılımda: Düz JavaScript'te hatayı ancak kod **çalışırken** görürsünüz:

```
// JavaScript – hata vermiyor ama yanlış çalışıyor
const gun = isEmri.slaBitis; // Alanın gerçek adı slaDueAt. Yanlış yazdık.
console.log(gun);           // Sonuç: undefined. Uyarı yok, çökme yok.
                             // Bu hatayı büyük ihtimalle KULLANICI bulacak.
```

```
// TypeScript – aynı hatayı yazarken yakalıyor
const gun = isEmri.slaBitis;
//           ~~~~~ Editör anında altını çiziyor:
//           "WorkOrder tipinde slaBitis diye bir alan yok.
//           slaDueAt demek mi istediniz?"
// Yani hata kullanıcıya değil, geliştirme sırasında sana gidiyor.
```

Fark şu: birinde hatayı **kullanıcı** bulur, diğerinde **derleyici**.

Bu projede neden özellikle kritik

Sistem aynı veriyi üç yerde kullanıyor: backend, web arayüzü, ileride mobil. Bir alanın adı değiştiğinde diğerlerinin de güncellenmesi gerekiyor.

```
// packages/contracts – tek tanım
export type WorkOrderListItem = z.infer<typeof WorkOrderListItem>;
```

API'de `slaDueAt` alanı `dueAt` olarak değişirse, arayüz **derlenmiyor**. Hata ekranda `undefined` olarak değil, geliştirme sırasında kırmızı çizgi olarak çıkıyor (E.3).

"Strict" modu ne katıyor

En katı ayar açık. En çok işe yarayan kısmı **boş değer kontrolü**:

```
// strict modu açık – bu kod DERLENMİYOR
function ata(wo: WorkOrder) {
  return wo.assigneeId.toString();
//      ~~~~~ "assigneeId boş (null) olabilir" uyarısı.
// ⚠ İş emri henüz kimseye atanmamışsa bu alan boş olur; boş bir değer
//   üzerinde işlem yapmak uygulamayı çalışma anında çökertir.
}

// Derleyici, boş olma ihtimalini ele almanı ZORUNLU kılıyor
function ata(wo: WorkOrder) {
  // Atanmamış iş emri durumunu açıkça düşünmek zorundasın
  if (wo.assigneeId === null) throw new NotAssignedError(wo.id);

  // Buradan sonrası güvenli: derleyici artık boş olmadığını biliyor
  return wo.assigneeId.toString();
}
```

★ Bu projede iş emrinin `assigneeId` alanı **atanmamışken boş**. Strict mod, "atanmamış iş emri" durumunu düşünmeyi **zorunlu kılıyor** — ödevin "*atanmamış bir iş emri işleme alınamamalıdır*" kuralı böylece derleyici tarafından hatırlatılıyor.

Sürüm kısıtı

Projede TypeScript 6 kullanılıyor, 7 değil. Sebep tercih değil kısıt: `typescript-eslint` paketi henüz TS 7'yi desteklemiyor. TS 7'ye çıkmak lint kapısını tamamen devre dışı bırakırdı. Kısıt kalkınca yükseltilecek.

C.10 Docker ve Docker Compose

Nedir: Uygulamayı, çalışması için ihtiyaç duyduğu her şeyle birlikte tek bir pakete koyan sistem. **Ne işe yarar:** "Benim bilgisayarımda çalışıyordu" sorununu ortadan kaldırır. **Bu projede nerede:** Tüm sistemin tek komutla ayağa kalkması. **Ödevdeki karşılığı:** Aynı; ödev §24 zaten Docker ve Docker Compose istiyor.

Çözdüğü gerçek sorun

Bir uygulama boşlukta çalışmaz: belirli bir Node.js sürümüne, belirli bir PostgreSQL sürümüne, belirli ayarlara ihtiyaç duyar. Geliştiricinin bilgisayarında Node 24, sunucuda Node 18 varsa uygulama sunucuda patlar ve sebebini bulmak günler alabilir.

Docker bu sorunu kökten çözer: uygulamayı **ihtiyaçlarıyla birlikte** paketler. O paket nerede çalıştırılırsa çalıştırılsın içi aynıdır.

Temel kavramlar

Terim	Ne demek	Benzetme
Dockerfile	Paketin nasıl yapılacağıının tarifi: "Node 24 al, şu dosyaları kopyala, derle, şu portu aç"	Yemek tarifi
İmaj (image)	Tarifin çalıştırılmış hâli — donmuş, değişmez paket	Dondurulmuş yemek
Konteyner	İmajın çalışan hâli	Isıtılıp tabağa konmuş yemek
docker-compose.yml	Birden fazla konteyneri birlikte tarif eden dosya	Menü

Bu projede Compose ne yapıyor

Sistem dört ayrı parçadan oluşuyor ve hepsinin birlikte çalışması gerekiyor:

1. **PostgreSQL** — veritabanı
2. **API** — NestJS sunucusu
3. **Worker** — arka plan işlerini yapan süreç
4. **Web** — Next.js arayüzü

Compose bu dördünü tek dosyada tarif eder: hangi portlarda çalışacakları, hangi sırayla başlayacakları, birbirlerini nasıl bulacakları. Tek komutla hepsi ayağa kalkar:

```
docker compose up --build
```

Ödev §24 tam olarak bunu şart koşuyor: değerlendirmeyi yapan kişi projeyi indirip tek komutla çalıştırabilmeli.

★ Port ve konteyner planı — çakışma nasıl önleniyor

Problem: Aynı bilgisayarda ikinci bir proje varsa ikisi de 3000 ve 5432'yi ister. İkincisi ya açılmaz ya da sessizce başka bir porta oturur ve hangi projeye baktığın anlaşılmaz.

Çözüm iki parçalı:

	Konteyner içi port	Host portu (senin makinendeki kapı)
Değer	3000 · 4000 · 5432 · 6379 — standart	Makineye göre değişir
Değişir mi	🚫 Asla — her konteynerin kendi ağı var, orada çalışma imkânsız	✅ Çakışma varsa değiştirilir
Nereden gelir	<code>docker-compose.yml</code> içinde sabit	<code>.env</code> dosyasından

```
# docker-compose.yml
name: bakım          # ★ Ağ ve volume adları "bakım-" ön ekli olur;
                    # başka projenin "default" ağıyla karışmaz.

services:
  web:
    container_name: bakım-web
    ports: ["${WEB_PORT:-3000}:3000"]
    #      └─ host: .env'den      └─ konteyner içi: HEP 3000
```

Servis	Konteyner adı	Konteyner içi	Host portu değişkeni
Next.js arayüz	<code>bakım-web</code>	3000	<code>WEB_PORT</code>
NestJS API	<code>bakım-api</code>	4000	<code>API_PORT</code>
Worker	<code>bakım-worker</code>	— (port açmaz)	—
PostgreSQL	<code>bakım-postgres</code>	5432	<code>DB_PORT</code>
Redis	<code>bakım-redis</code>	6379	<code>REDIS_PORT</code>

Volume: `bakım-pgdata` .

★ **Host portlarının değeri makineye özeldir, koda yazılmaz.** Boş bir bilgisayarda `.env` içine hiçbir şey yazmasan varsayılanlar (3000/4000/5432/6379) geçerli olur. Başka bir proje o portları tutuyorsa `.env` içinde `WEB_PORT=3100` yazarsın — **başka hiçbir dosyaya dokunmadan.**

⚠️ `apps/web/package.json` içindeki `dev` betiği de aynı değişkeni okur (`next dev -p ${WEB_PORT:-3000}`). **Varsayılan güvenilmez:** Next, 3000 doluysa uyarı verip sessizce 3001'e oturur ve iki sekmeden hangisinin bu proje olduğunu anlayamazsın.

🚫 **Docker Desktop ekranından port değiştirmek kalıcı değildir.** O ekran eşlemeyi *gösterir*; `docker compose up` yeniden çalıştığında `compose` dosyası ne diyorsa o geçerli olur. Tek doğru kaynak `.env` + `docker-compose.yml` (12-factor: *config ortamdan gelir*).

★ **Testcontainers etkilenmez (C.7):** entegrasyon testleri **rastgele** port kullanır, çalışan hiçbir veritabanına dokunmaz.

Çok aşamalı (multi-stage) yapı

Ödev bunu özellikle istiyor. Sebebi şu: uygulamayı **derlemek** için gereken araçlar (derleyiciler, geliştirme paketleri) **çalıştırmak** için gerekmez. Çok aşamalı yapıda önce büyük bir ortamda derleme yapılır, sonra yalnızca sonuç dosyaları küçük bir imaja taşınır. Sonuç: birkaç kat küçük imaj, daha hızlı dağıtım ve daha az güvenlik yüzeyi.

C.11 Vitest

Nedir: Test çalıştırıcı — yazdığınız testleri koştan ve sonucu raporlayan araç. **Bu projede nerede:** İş kuralları, SLA politikaları, durum geçişleri, Factory. **Ödevdeki karşılığı:** xUnit / NUnit / MSTest.

Testin gerçekte çözdüğü sorun

Gerçek hayat: Bir binada yangın alarmı var. Amacı yangını **söndürmek** değil; yangın çıktığında **haber vermek**. Ayda bir test edilmezse, gerçek yangında çalışıp çalışmayacağı bilinmez.

Yazılımda: Test yazmanın amacı "kod çalışıyor mu" değildir — o zaten elle bakılır. Asıl amaç: **altı ay sonra biri bu koda dokunduğunda, farkında olmadan bir şeyi bozarsa haberi olsun.**

Bu projede tipik bir test

```
describe('İş emri kapatma', () => {

  it('çözüm açıklaması boşsa kapatılamaz', () => {
    // HAZIRLIK: çözülmüş durumda bir iş emri oluştur
    const wo = workOrder({ status: 'RESOLVED' });

    // ÇALIŞTIR ve BEKLE: boş açıklamayla kapatmayı dene, hata fırlatmalı
    expect(() => wo.close('', clock)).toThrow(ResolutionNoteRequired);
  });

  it('RESOLVED olmayan iş emri kapatılamaz', () => {
    // HAZIRLIK: henüz üzerinde çalışılan bir iş emri
    const wo = workOrder({ status: 'IN_PROGRESS' });

    // Açıklama dolu olsa bile durum uygun değil → geçiş hatası bekleniyor
    expect(() => wo.close('Tamam', clock)).toThrow(InvalidTransition);
  });
});

// Dikkat: bu testlerde veritabanı, HTTP veya NestJS yok.
// Kural saf bir sınıfta durduğu için test milisaniyede bitiyor.
```

★ Dikkat: bu testlerde **veritabanı yok, HTTP yok, Nest yok**. İş kuralları domain katmanında saf durduğu için (E.1) test milisaniyede koşuyor. Yüzlerce kural testi birkaç saniyede bitiyor.

Zamana bağlı kuralları test etmek

SLA kuralları "şu an saat kaç" bilgisine bağlı. Gerçek saati beklemek imkânsız:

```
it('SLA süresi geçmiş iş emri ihlal olarak işaretlenir', () => {
  // Saati sabitliyoruz: "şu an 10:00" diyoruz.
  // Gerçek saati beklemeden zamana bağlı kuralı test edebiliyoruz.
  const clock = fixedClock('2026-08-21T10:00:00Z');

  // SLA bitişi 09:00 olan bir iş emri – yani süre bir saat önce dolmuş
  const wo = workOrder({ slaDueAt: '2026-08-21T09:00:00Z' });

  // Sonuç: ihlal edilmiş sayılmalı
  expect(isBreach(wo, clock)).toBe(true);
});
```

`Clock` soyutlaması (E.2 → D harfi) sayesinde teste "sen şu an şu andasın" denebiliyor. Kod içinde doğrudan `new Date()` çağrılseydi bu test yazılamazdı.

Ödevin istediği test alanları

Ödev §23 hangi alanların test edileceğini tek tek sayıyor: Factory Pattern, SLA politikaları, geçerli ve **geçersiz** durum geçişleri, atama kuralları, pasif lokasyon kuralı, mükerrer bildirim engelleme, arka plan işinin kapalı iş emrinde işlem yapmaması.

Ortak özellikleri: hepsi **iş kuralı** ve hepsi veritabanı gerektirmeden test edilebiliyor.

Koruma testi disiplini

⚠ Bir korumayı test eden test yazıldığında, koruma **geçici olarak kaldırılıp** testin kırmızıya döndüğü gözle görülür. Dönüyorsa test korumayı değil başka bir şeyi ölçüyordur ve size **yanlış bir güven** verir.

C.12 Playwright

Nedir: Gerçek bir tarayıcıyı otomatik olarak sürüp uygulamayı bir kullanıcı gibi kullanan test aracı. **Ne işe yarar:** "Giriş yap → iş emri oluştur → ata → kapat" gibi uçtan uca akışların gerçekten çalıştığını doğrular. **Bu projede nerede:** Kritik kullanıcı yolculukları. **Ödevdeki karşılığı:** Doğrudan istenmemiş; ekran testleri için eklendi.

Diğer testlerden farkı

Birim testleri tek bir kuralı ölçer. Playwright ise **parçaların birlikte çalıştığını** ölçer: arayüz doğru isteği atıyor mu, API doğru cevap veriyor mu, gelen cevap ekranda doğru görünüyor mu, yetkisiz kullanıcı doğru sayfaya yönlenecek mi.

Gerçek hayatta çoğu hata tek tek parçalarda değil, **parçaların birleştiği yerde** çıkar. Playwright tam oraya bakar.

C.13 JWT ve argon2 — kimlik doğrulama

Nedir:

- **JWT (JSON Web Token):** Giriş yapan kullanıcıya verilen, **imzalı** dijital kimlik kartı.
- **argon2:** Şifreyi geri döndürülemez şekilde karıştıran algoritma.

Bu projede nerede: Giriş, yetkilendirme, oturum yenileme. **Ödevdeki karşılığı:** §5.1 — JWT tabanlı kimlik doğrulama, yenileme jetonu (token), rol bazlı yetkilendirme, pasif kullanıcı engeli.

JWT gerçekte hangi sorunu çözüyor

Gerçek hayat: Otele giriş yaptınız. Resepsiyon her seferinde kimliğinizi kontrol etmiyor; size bir **oda kartı** veriyor. Kart üzerinde hangi odaya, hangi tarihe kadar girebileceğiniz kodlu. Kartı kopyalayıp "süit oda" yazamazsınız çünkü kart **sistem tarafından imzalanmış**.

Yazılımda: Kullanıcı bir kez giriş yapar; sonraki her istekte "sen kimsin" sorusunun yeniden cevaplanması gerekir. Her seferinde şifre sormak mümkün değil, şifreyi saklamak ise tehlikeli.

Jetonun içi şuna benzer:

```
{
  "sub": 42,                // kullanıcının kimlik numarası
  "role": "TECHNICIAN",    // rolü – yetki kontrolü buna bakıyor
  "exp": 1755503600        // son kullanma zamanı; geçince jeton geçersiz
}

// 🚫 Bu içerik ŞİFRELİ DEĞİL, herkes okuyabilir – sadece imzalı.
// Bu yüzden jetona TCKN, telefon gibi kişisel veri KONMAZ.
// ★ İmza sayesinde içerik değiştirilemez: biri "role": "ADMIN" yapmaya
// kalkarsa imza tutmaz ve sunucu jetonu reddeder.
```

Üzerinde oynanamaz: biri **"role": "ADMIN"** yapmaya kalkarsa **imza tutmaz** ve sunucu jetonu (token) reddeder.

⚠ **Sık yapılan hata:** Jetonun içi **şifreli değildir**, yalnızca imzalıdır. Herkes içeriğini okuyabilir. Bu yüzden jetona TCKN, telefon, e-posta gibi kişisel veri **konmaz** — sadece kimlik numarası ve rol.

Neden iki ayrı jeton (token)

Erişim jetonu (token) (access token) kısa ömürlüdür — dakikalar. Çalınırsa zararı sınırlı kalsın diye. Ama kullanıcıya her 15 dakikada bir giriş yaptırmak da olmaz.

Bu yüzden ikinci bir **yenileme jetonu (token) (refresh token)** verilir: süresi uzundur, tek işi yeni bir erişim jetonu (token) almaktır.

Rotasyon (döndürme): Yenileme jetonu (token) her kullanıldığında **değişir**. Eskisi geçersiz olur.

```
// Kullanıcı "oturumumu yenile" dedi. Tüm adımlar tek işlemde.
await prisma.$transaction(async (tx) => {

  // 1) Gelen yenileme jetonunu veritabanında ara.
  //   Jetonun kendisi değil, karıştırılmış hâli (hash) saklanıyor.
  const stored = await tx.refreshToken.findUnique({ where: { hash } });

  // 2) Böyle bir jeton hiç yoksa istek reddedilir
  if (!stored) throw new UnauthorizedException();

  // 3) Jeton var AMA daha önce kullanılıp iptal edilmiş.
  //   Bir kez kullanılmış jetonun tekrar gelmesi = kopyalanmış demektir.
  if (stored.revokedAt) {
    // ★ Güvenlik önlemi: o kullanıcının AÇIK TÜM oturumlarını kapat.
    //   Saldırgan da meşru kullanıcı da yeniden giriş yapmak zorunda kalır.
    //   Jetonun çalındığını anlamanın tek güvenilir yolu bu.
    await tx.refreshToken.updateMany({
      where: { userId: stored.userId, revokedAt: null },
      data: { revokedAt: clock.now() },
    });
    throw new UnauthorizedException('Token reuse detected');
  }

  // 4) Jeton geçerli: kullanıldığı için hemen iptal et (bir daha kullanılamaz)
  await tx.refreshToken.update({ where: { hash }, data: { revokedAt: clock.now() } });

  // 5) Yeni bir jeton çifti üret ve kullanıcıya ver
  return issueNewPair(stored.userId);
});
```

★ **Yeniden kullanım tespiti** bu kodun kalbi: iptal edilmiş bir jeton (token) tekrar gelirse, jetonun kopyalandığı anlaşılır ve o kullanıcının **tüm oturumları** kapatılır. Ödev bunu doğrudan istemiyor ama gerçek sistemlerde standarttır.

İşlemin tamamı **tek transaction** içinde — ödev §20 bunu ayrıca sayıyor: *"Refresh token yenileme ve eski token'ın geçersiz hâle getirilmesi."*

Jeton (token) nerede saklanıyor

İstemci	Nerede	Neden
Web	httpOnly çerez	JavaScript okuyamaz → XSS saldırısında çalınamaz
Mobil / Swagger	Authorization: Bearer ... başlığı	Çerez kavramı yok

Aynı doğrulama katmanı ikisini de kabul ediyor; API istemci tanımıyor (E.10'daki "tek API, çok istemci" ilkesi).

argon2 gerçekte hangi sorunu çözüyor

Gerçek hayat: Kâğıt öğütücü. Belgeyi öğütürsünüz — geri birleştirilemez. Ama aynı belgeyi tekrar öğütürseniz **aynı şeritler** çıkar. Karşılaştırma bu şekilde yapılır; belgenin kendisi hiç saklanmaz.

Yazılımda: Şifreler veritabanına **asla düz metin yazılmaz**. Veritabanı sızarsa tüm kullanıcıların şifresi ele geçer — üstelik insanlar aynı şifreyi başka yerlerde de kullandığı için zarar o sistemle sınırlı kalmaz.

```
// KAYIT SIRASINDA – kullanıcının şifresini geri döndürülemez hâle getir.
// Veritabanına yazılan şey şifre değil, bu karıştırılmış çıktı.
const hash = await argon2.hash(password, { type: argon2.argon2id });

// GİRİŞTE – şifre asla geri çözülüyor.
// Kullanıcının yazdığı şifre aynı işlemde geçiriliyor ve
// veritabanındaki çıktıyla karşılaştırılıyor. Eşleşirse giriş başarılı.
const ok = await argon2.verify(user.passwordHash, password);
```

★ **argon2 kasıtlı olarak yavaştır ve çok bellek kullanır.** Bu bir kusur değil, tasarım tercihidir: saldırgan çalınmış bir veritabanında saniyede milyonlarca şifre deneyememeli. Hızlı algoritmalar (MD5, SHA-1) tam da bu yüzden şifre için **yanlıştır** — hızlı olmaları saldırganın işine yarar.

Ödevin diğer şartları

Şart	Karşılığı
Pasif kullanıcı giriş yapamamalı	Guard, jetonu (token) doğruladıktan sonra kullanıcının isActive alanını kontrol eder
Token süreleri yapılandırmadan yönetilmeli	.env içinde; kodda sabit değer yok
Yetkisiz ve yasaklı erişim doğru kod dönmeli	Kimliksiz 401 , yetkisiz 403 (E.7)
Secret'lar source control'a girmemeli	.env commit edilmez, .env.example edilir

C.14 Turborepo + pnpm workspaces

Nedir: Birden fazla uygulamayı tek depoda düzenli yönetmeyi sağlayan araçlar. **Ne işe yarar:** Web, API ve worker aynı depoda yaşar; ortak kodlar tek yerde tanımlanır; tek komutla hepsi başlar. **Bu projede nerede:** Projenin genel iskeleti. **Ödevdeki karşılığı:** Doğrudan istenmemiş; §9 (DRY) maddesinin mekanizması.

Çözdüğü gerçek sorun

Bu projede üç ayrı program var (web, API, worker) ve hepsi aynı veri şekillerini kullanıyor. Bunları ayrı depolarda tutsaydık, ortak tipleri paylaşmanın tek yolu her değişiklikte paket yayınlamak olurdu: sürüm yükselt → yayınl → diğer depolarda güncelle. Bu döngü unutulduğunda depolar sessizce ayrışır.

Tek depoda ise ortak şemalar `packages/contracts` içinde bir kez tanımlanır ve üç taraf da oradan okur. API'de bir alanın adı değiştiğinde **arayüz derlenmez** — hata anında ortaya çıkar.

Turborepo ne yapıyor

Komutları doğru sırayla ve paralel çalıştırır, sonuçları önbelleğe alır. Tek komutla tüm sistem geliştirme modunda başlar:

```
docker compose up -d      # PostgreSQL ve Redis
pnpm dev                  # web + api + worker aynı anda
```

Çıktılar tek ekranda etiketli görünür, biri çökerse hangisi olduğu anlaşılır.

⚠ **Monorepo ile monolit farklı şeylerdir.** Monorepo *kodun nerede durduğu*, monolit *programın nasıl çalıştığı* hakkındadır. Bu projede monorepo (tek depo) kullanılıyor ama çalışırken birden fazla süreç var.

C.15 nestjs-pino — Yapılandırılmış Loglama (Sistemin Gözü)

Modern backend altyapısı: takip edilebilirlik, bağlam ve güvenlik

Bu altyapı; kurumsal yazılımlarda işlerin aksamadan yürümesi, hataların saniyeler içinde bulunması ve veritabanı kayıtlarının güvenliği için tasarlanmıştır. İki temel kütüphane üzerine kuruludur: **nestjs-pino** (sistemin gözü/kulağı) ve **nestjs-cls** (sistemin hafızası — C.16).

Nedir: Node'un en hızlı log kütüphanesi olan **pino**'nun NestJS entegrasyonu. **Ne işe yarar:** Her log satırını **JSON** olarak üretir (düz metin olarak değil) ve isteğe ait bağlamı (correlation ID, kullanıcı, süre, durum kodu) otomatik ekler. **Bu projede nerede:** API ve worker'ın tüm log çıktısı. **Ödevdeki karşılığı:** §25 — structured logging.

Neden düz metin değil de JSON

Düz metin: "Ali saat 14:30'da 1042 nolu siparişi sildi ve hata oluştu." Bu satırı yalnızca insan gözü okuyabilir. Milyonlarca log arasında arama motorları bu metni anlamlandıramaz.

JSON (yapılandırılmış): Log alanlara ayrılır:

```
{
  "level": "info",          // önem derecesi: info | warn | error
  "time": 1755500000,       // olayın zamanı
  "correlationId": "9f3c...", // bu isteğin takip numarası – tüm izler bununla bulunur
  "userId": 42,             // işlemi yapan kullanıcı
  "method": "POST",         // HTTP yöntemi
  "path": "/api/v1/work-orders/1042/close", // hangi uç çağrıldı
  "statusCode": 200,        // sonuç: 200 başarılı, 4xx/5xx hata
  "durationMs": 63          // kaç milisaniye sürdü – yavaş uçları bulmak için
}
```

Bu sayede DevOps ekipleri log toplayıcı araçlarda (Loki, ELK, CloudWatch) nokta atışı sorgu yapabilir:

"Son 1 saatte, 42 nolu kullanıcının attığı ve hata (500) alan tüm istekleri getir."

Correlation ID – hata ayıklamanın en değerli alanı

Sisteme gelen **her HTTP isteğine** veya uyanan **her worker işine** benzersiz bir takip numarası (**correlationId**) verilir.

Faydası: Bir kullanıcı "hata aldım" dediğinde ekranda bu kod yazar. Yazılımcı bu kodu arattığı an, o isteğin veritabanında, servislerde ve arka planda bıraktığı **tüm izleri** kronolojik olarak listeler — tek bir isteğe ait yüzlerce satır log. Paralel akan yüzlerce isteğin logları birbirine karışmaz.

🚫 Log güvenliği (maskeleme / redaction)

Loglara **asla** şifre, kredi kartı, TCKN veya token gibi kişisel veriler yazılmaz. **nestjs-pino** içinde merkezî bir süzgeç (maskeleme listesi) vardır. İstek gövdesinde bu veriler geçse bile, loga yazılmadan önce otomatik olarak sansürlenir.

Kritik ayırım: uygulama logu vs. denetim kaydı (audit trail)

Yazılımcıların en çok karıştırdığı konulardan biri budur. İkisi tamamen farklı amaçlara hizmet eder:

Özellik	Uygulama logu (nestjs-pino)	Denetim kaydı (audit / veritabanı)
Örnek	POST /orders/1042 200 63ms	"Ahmet, 1042 nolu sipariş durumunu 'Beklemede'den 'İptal Edildi'ye çekti."
Depolama yeri	Konsol çıktısı (stdout) → log toplayıcı	Doğrudan veritabanı tablosu
Saklama süresi	Günler veya haftalar (sonra silinir)	Yıllarca (silinmesi yasaktır)
Kullanıcı	Yazılımcı / DevOps (hata çözmek için)	Müfettiş, hukuk, iş birimi (denetim için)
Şema yapısı	Serbest / esnek	Sabit kolonlar (createdBy , updatedBy)

C.16 nestjs-cls — İstek Bağlamı (Sistemin Hafızası)

Nedir: Node.js'in yerleşik `AsyncLocalStorage` API'sini NestJS'e bağlayan ince bir katman. **Ne işe yarar:** Bir isteğe ait bilgileri (aktif kullanıcı, correlation ID) çağrı zincirinin her katmanına, **parametre olarak taşımadan** ulaştırır. **Bu projede nerede:** Audit alanlarının doldurulması, loglama, worker bağlamı. **Ödevdeki karşılığı:** §21 — "sistem saati ve mevcut kullanıcı bilgisi doğrudan statik yapılardan alınmamalıdır."

Problem

Veritabanına bir veri yazarken `createdBy` (yazan kim) alanını doldurmak için, o anki kullanıcının kimlik bilgisini en üstteki controller'dan en alttaki Prisma veritabanı katmanına kadar indirmemiz gerekir.

Bunu yapmak için iki yaygın ama **yanlış** yöntem vardır:

1. Parametre olarak taşımak (hamallık). Geçtiğiniz her fonksiyona `(..., userId)` eklemek. Kod kirlenir. Yarın bir gün takibe `correlationId` veya `ipAddress` eklemek isterseniz **yüzlerce fonksiyon imzasını** değiştirmek zorunda kalırsınız.

2. Global değişkende tutmak (felaket). Node.js tek bir iş parçacığında (single thread) asenkron çalışır. Global bir değişken kullanırsanız asenkron beklemler (`await`) sırasında işlemler çakışır:

```
Ahmet sisteme girer          → Global isim "Ahmet" olur
Ahmet'in veritabanı işlemi (await) beklemeye alınır
0 sırada Mehmet sisteme girer → Global isim "Mehmet" olarak EZİLİR
Ahmet'in işlemi kaldığı yerden devam eder ve veritabanına kaydedilir
                               → Sistem veriyi MEHMET kaydetti sanır
```

⚠ Bu hata testlerde çıkmaz; sadece sistem canlıya geçip **yük altında** kaldığında ortaya çıkar.

Çözüm: AsyncLocalStorage (nestjs-cls)

Node.js'in yerleşik `AsyncLocalStorage` yapısı, her asenkron çağrı zincirine özel, **birbirini asla görmeyen izole hafıza kutuları** açar. .NET dünyasındaki `AsyncLocal` veya Java'daki `ThreadLocal` yapısının aynısıdır.

Nasıl çalışır: İstek geldiği an `nestjs-cls` bir kutu açar, içine `userId` ve `correlationId` koyar. Katmanlar boyunca hiçbir fonksiyona parametre geçilmez. En alttaki Prisma veya log sistemi, veriyi bu kutudan doğrudan okur. Ahmet ve Mehmet'in kutuları asenkron dünyada asla birbirine karışmaz.

Bu projede her şey nasıl birleşiyor

Kurulan bu altyapı sayesinde projedeki dört ana unsur otomatik olarak tıkr tıkr çalışır:

1. Audit (otomatik takip)

Prisma için bir eklenti (extension) yazılmıştır. Yazılımcı servis kodunda tek bir satır bile `createdBy = user.id` yazmaz. Prisma veritabanına kayıt giderken eklenti araya girer, `nestjs-cls` hafıza kutusundan o anki aktif kullanıcıyı okur ve alanları otomatik doldurur.

2. Log

`nestjs-pino` her log satırına hafıza kutusundaki `correlationId` ve `userId` bilgilerini otomatik ekler.

3. Sistem saati (Clock servisi)

Kodun içinde doğrudan `new Date()` çağırılması **yasaktır**. Zaman bilgisi merkezî bir `Clock` servisi üzerinden okunur.

Test yazarken bu servise *"sen şu an 30 gün sonrasındasın"* denerek, zamana bağlı tüm kurallar (SLA) gerçek zamanı beklemeden saniyeler içinde test edilir.

4. Arka plan işlerinde (worker/job) bağlam

🚫 **Kritik uyarı (§15):** Bir worker (örneğin gece çalışan robot) tetiklendiğinde ortada bir HTTP isteği, tarayıcı veya token **yoktur**. Dolayısıyla HTTP bağlamı boş (`undefined`) gelecektir. Eğer yazılımcı servis kodunu yazarken *"nasılsa HTTP isteğinden kullanıcı gelir"* diye güvenerek kod yazarsa, worker çalıştığı an sistem çöker.

Çözüm: Robot işe başladığı an **kendi bağlam kutusunu** (`nestjs-cls`) kendisi açar. İçine `userId: "SYSTEM_ROBOT"` ve kendi ürettiği `correlationId` bilgisini koyar. Böylece:

- Ortak yazılan servis kodları çökmeden çalışır
- Prisma audit alanları otomatik dolar
- Loglarda bu işi bir insanın değil **robotun** yaptığı net şekilde görünür

Ödevin §15'teki *"job içerisinde HTTP request context'e güvenilmemelidir"* maddesinin karşılığı budur.

C.17 @nestjs/terminus — sağlık kontrolü

Nedir: Uygulamanın sağlıklı olup olmadığını dışarıya bildiren uçları oluşturan kütüphane. **Bu projede nerede:** `/health/live` ve `/health/ready`. **Ödevdeki karşılığı:** \$25 — bu iki uç açıkça isteniyor.

Çözdüğü sorun

Gerçek hayat: Bir mağazanın kapısında "AÇIK" tabelası var ama içeride kasa sistemi çökmüş. Kapı açık, ışıklar yanıyor — ama **alışveriş yapılamıyor**. Dışarıdan bakan biri farkı anlamaz; içeri girip denemesi gerekir.

Yazılımda: Uygulama süreci ayakta olabilir ama veritabanına bağlanamıyor olabilir. Dışarıdan bakınca "çalışıyor" görünür; gerçekte her istek hata alır.

İki ucun farkı

```
@Controller('health')
export class HealthController {

  // GET /health/live – "Uygulama süreci yaşıyor mu?"
  @Get('live')
  live() {
    // Hiçbir şeyi kontrol etmiyor, sadece cevap veriyor.
    // Cevap gelmiyorsa süreç donmuş demektir → izleme sistemi yeniden başlatır.
    return { status: 'ok' };
  }

  // GET /health/ready – "İstek almaya HAZIR mı?"
  @Get('ready')
  ready() {
    return this.health.check([
      // Veritabanına kısa bir sorgu atıp cevap verip vermediğine bakıyor.
      // 1.5 saniyede cevap gelmezse hazır değil sayılıyor.
      () => this.db.pingCheck('database', { timeout: 1500 }),
    ]);
    // Hazır değilse bu kopyaya trafik yönlendirilmiyor –
    // ama süreç yeniden başlatılmıyor da. Farkı bu.
  }
}
```

Uç	Soru	Cevap gelmezse ne olur
/health/live	Süreç yaşıyor mu	İzleme sistemi konteyneri yeniden başlatır
/health/ready	İstek alabilir mi	O kopyaya trafik yönlendirilmez

★ **Ayrımın sebebi:** İkisi aynı olsaydı, veritabanı birkaç saniyeliğine yavaşladığında izleme sistemi uygulamayı **gereksiz yere yeniden başlatırdı** — ve yeniden başlatma sırasında zaten hiç cevap veremezdi. Yani çözüm sorunu büyüttü.

live yalnızca sürecin donmadığını söyler, **ready** bağımlılıkları yoklar.

Bu projede kim kullanıyor

Bu uçlar **sizin için değil**, kurumun izleme sistemi için. DevOps ekibi konteynerleri bu uçlara bakarak yönetir:

- Uygulama açılırken veritabanı henüz gelmemişse **ready** "hayır" der, o aralıkta gelen istekler hata almaz
- docker-compose.yml** içinde de aynı uç kullanılır: **api** servisi ayağa kalkmadan **worker** başlatılmaz

Yani bu iki uç, teslim paketinin **DevOps'la sözleşmesinin** bir parçası.

C.18 Swagger / OpenAPI

Nedir: API'nin hangi uçları olduğunu, her ucun hangi veriyi beklediğini ve ne döndürdüğünü anlatan otomatik dokümantasyon. **Bu projede nerede:** **/docs** adresinde tarayıcıdan açılabilen arayüz. **Ödevdeki karşılığı:** §3 — OpenAPI + Scalar veya Swagger.

Çözdüğü gerçek sorun

Gerçek hayat: Bir cihazın kullanma kılavuzu, cihazın üçüncü sürümüne göre yazılmış ama elinizde beşinci sürüm var. Kılavuzdaki düğme cihazda yok. Kılavuz **zararlı** hâle gelmiş — hiç olmamasından kötü, çünkü ona güveniyorsunuz.

Yazılımda: API dokümantasyonu elle yazıldığında **kaçınılmaz olarak eskir**. Kod değişir, doküman güncellenmez; bir süre sonra kimse dokümana güvenmez ve herkes koda bakmaya başlar.

Bu projede doküman elle yazılmıyor

Doküman **Zod şemalarından otomatik üretiliyor** — yani doğrulama kuralı ne ise dokümanda yazan da odur:

```
// TEK KAYNAK: iş emri oluşturma isteğinin kuralları
export const CreateWorkOrder = z.object({
  title: z.string().min(5).max(200), // 5-200 karakter
  priority: z.enum(['LOW', 'NORMAL', 'HIGH', 'CRITICAL']), // bu dört değerden biri
  assetId: z.number().int().positive(), // pozitif tam sayı
});

// Aynı şema iki işi birden yapıyor:
// 1) gelen isteği doğruluyor
// 2) Swagger dokümanını üretiyor
// Yani kural değişince doküman kendiliğinden güncelleniyor.
@Post()
@ApiOperation({ summary: 'Yeni iş emri oluşturur' }) // dokümandaki açıklama
create(@Body() dto: CreateWorkOrderDto) { ... }
```

★ Kural değiştiğinde doküman **kendiliğinden** değişir. Eskime ihtimali yapısal olarak ortadan kalkıyor — E.3'teki DRY ilkesinin bir uygulaması.

Sadece okunmaz, denenir

Swagger arayüzü uçları **tarayıcıdan çalıştırmayı** sağlar. Bu projede iki somut faydası var:

1. **Frontend yazılmadan API gösterilebilir.** Değerlendirmede "iş emri oluşturma çalışıyor mu" sorusuna, ekran hazır olmasa bile cevap verilebilir.
2. **Mobil geliştirici** (ileride) API'yi koda bakmadan öğrenir.

Neden Swagger UI, Scalar değil

Ödev ikisine de izin veriyor. Swagger UI seçildi çünkü `@nestjs/swagger` ile **ekstra bağımlılık olmadan** geliyor ve piyasada herkesin tanıdığı arayüz. Scalar daha modern görünüyor ama çok daha az kullanılıyor; uzun ömürlü bir projede omurgayı az bilinen bir arayüze bağlamak devraları zorlar.

C.19 TanStack Query

Nedir: Tarayıcı tarafında sunucudan gelen veriyi yöneten kütüphane. **Ne işe yarar:** Veriyi getirir, saklar, tazeler; yükleniyor ve hata durumlarını yönetir. **Bu projede nerede:** Tüm liste ve detay ekranları. **Ödevdeki karşılığı:** §22 — "API çağrıları ortak bir katmanda yönetilmelidir, tekrarlanan request kodlarından kaçınılmalıdır."

Hangi projede yaşıyor

Bu ayrımı netleştirmek önemli: **API ve worker** backend (arka plan) projesindedir. **TanStack Query** ise frontend (ön yüz) projesinde yazılan ortak bir katmandır.

TanStack Query'nin dođası: tarayıcının hafızasını (cache) yönetir.

Çözdüğü gerçek sorun

Web sitesini açan insanın ekranı donmasın, butonlar hızlı çalışsın ve aynı veri için sunucuya gereksiz yere üst üste iki kez istek gitmesin diye **önbelleğe alma** yapmaktır.

Ödevdeki "ortak katman" maddesi ne demek istiyor (§22)

Ödevde bahsedilen "API çağrıları ortak bir katmanda yönetilmelidir" uyarısı tamamen **frontend** projesi içindir.

Yanlış yapılan (tekrarlayan kod): Frontend yazılımcısı sipariş listesi sayfasında, ürün listesi sayfasında ve profil sayfasında tek tek elle `fetch('/api/v1/...')` yazıp, `isLoading` (yükleniyor) durumlarını her ekranda sıfırdan elle kodlarsa hata yapar ve kod kirlenir.

Doğru yapılan (ortak katman): Frontend projesinin içinde tüm API istekleri TanStack Query ile tek bir klasörde (örneğin `/hooks` klasöründe) toplanır. Ekranlar sadece bu ortak katmanı çağırır.

Ortak katman pratikte neye benziyor

```
// Dosya: apps/web/hooks/use-work-orders.ts
// Tüm ekranlar iş emri listesini BURADAN alıyor. Tek yer.
export function useWorkOrders(filters: WorkOrderFilters) {
  return useQuery({
    // Bu verinin "adresi". Filtre değişince adres de değişir ve
    // kütüphane otomatik olarak yeni veriyi çeker.
    queryKey: ['work-orders', filters],

    // Veriyi fiilen getiren fonksiyon: API'ye istek atıyor
    queryFn: () => api.get('/work-orders', { params: filters }),

    // 30 saniye içinde aynı veri tekrar istenirse ağa çıkmıyor,
    // hafızadaki kopyayı veriyor. Ekranlar arası geçiş anında oluyor.
    staleTime: 30_000,
  });
}
```

```
// Ekran tarafı – üç satır, isLoading ve error hazır geliyor
// Ekran tarafı üç satır. Yükleniyor ve hata durumları hazır geliyor –
// her ekranda elle yazmak gerekmiyor.
const { data, isLoading, error } = useWorkOrders({ status: 'OPEN' });

if (isLoading) return <TabloIskeleti />; // veri gelene kadar iskelet göster
if (error) return <HataKutusu error={error} />; // hata varsa mesaj göster
return <WorkOrderTable rows={data.items} />; // veri geldi, tabloyu çiz
```

★ **queryKey** bu yapının kalbi: aynı anahtarla iki ekran veri isterse **ağa tek istek** çıkar, ikisi de aynı sonucu alır.

Bu projede özellikle işe yarayan yanı

Bir iş emrinin durumu değiştiğinde, liste ekranındaki verinin artık **eskiğini bilir** ve otomatik tazeler. Kullanıcı sayfayı elle yenilemek zorunda kalmaz.

```
// Veri DEĞİŞTİREN işlemler useMutation ile yapılıyor (okuma değil, yazma)
const kapat = useMutation({
  // Kapatma isteğini API'ye gönderen fonksiyon
  mutationFn: (id: number) => api.post(`/work-orders/${id}/close`),

  // İşlem başarılı olunca çalışıyor
  onSuccess: () => {
    // "İş emri listesi artık eski" diyoruz.
    // 0 listeyi gösteren tüm ekranlar kendiliğinden yeniden çekiyor –
    // kullanıcının sayfayı elle yenilemesi gerekmiyor.
    queryClient.invalidateQueries({ queryKey: ['work-orders'] });
  },
});
```

C.20 Tailwind CSS + shadcn/ui + React Hook Form

Bu projede nerede: 12 ekranın tamamı. **Ödevdeki karşılığı:** §22 — "hazır UI kütüphanesi kullanılabilir." Ödev arayüzün ileri seviye görsel tasarıma sahip olmasını zorunlu tutmuyor, ama **kullanılabilir** olmasını istiyor. Sıfırdan bileşen yazmak zaman kaybı olurdu.

1. Tailwind CSS — hızlı stil verme

Ne işe yarar: Klasik yöntemde bir butona renk vermek veya kenarlarını yuvarlatmak için ayrı bir CSS dosyası açıp sayfalarca kod yazmanız gerekir. Tailwind CSS ise hazır takma isimler (utility class) sunar.

Nasıl çalışır: Doğrudan butonun yanına `bg-blue-500 rounded p-2` yazarsınız; buton anında mavi, köşeleri yuvarlak ve içi genişlemiş hâle gelir. Ekstra CSS dosyalarıyla uğraşmadan, tasarımı kodun içinden çok hızlı bitirmenizi sağlar.

```
// KLASİK YÖNTEM: burada sadece bir isim var.  
// Bu ismin ne yaptığını görmek için ayrı bir CSS dosyası açman gerekiyor.  
<button className="birincil-buton">Kaydet</button>  
  
// TAILWIND: stilin tamamı burada, okunabiliyor.  
<button className="bg-blue-600 // arka plan: mavi  
                hover:bg-blue-700 // fare üstüne gelince koyu mavi  
                text-white // yazı rengi beyaz  
                rounded // köşeler yuvarlak  
                px-4 py-2"> // içeriden boşluk: yanlarda 4, üst-altta 2  
  Kaydet  
</button>
```

2. shadcn/ui — hazır ve esnek tasarım parçaları

Ne işe yarar: Bir web sitesinde ihtiyaç duyulan buton, tablo, modal (açılır pencere) veya menü gibi temel parçaları sıfırdan kodlamak büyük zaman kaybıdır. shadcn/ui bu parçaları hazır olarak verir.

Özel yanı nedir: Klasik kütüphanelerde (örneğin Material UI veya Ant Design) hazır bir butonu projenize eklediğinizde o buton kütüphaneye gömülüdür; rengini veya çalışma mantığını değiştirmek çok zordur.

shadcn/ui ise seçtiğiniz bileşenin **tüm kaynak kodunu doğrudan sizin kendi klasörünüzün içine kopyalar**. Buton artık tamamen sizin kodunuz olur. İstedığınız gibi kurcalayabilir, projenize göre özelleştirebilirsiniz.

★ Uzun ömürlü projelerde bu belirleyici bir avantajdır: bir davranışı değiştirmek için kütüphane güncellemesi beklemezsiniz.

3. React Hook Form + Zod uyumu — akıllı form yönetimi

Bu üçlünün en kritik ve en teknik kısmı burasıdır. Büyük projelerde form yönetimi iki büyük probleme yol açar:

Problem 1 — ekranın donması (performans). Kullanıcı bir kutucuğa her harf yazdığında tüm sayfa hafızada baştan çizilirse (re-render), form donmaya başlar. React Hook Form, kullanıcı yazarken sayfayı gereksiz yere baştan çizmez; performansı çok yüksektir.

Problem 2 — "ön yüzde geçti, arka yüzde reddedildi" tutarsızlığı. En çok yaşanan hatadır. Örneğin frontend yazılımcısı fiyata **en az 5 TL** sınırı koymuştur, ama backend yazılımcısı koda **en az 10 TL** yazmıştır. Kullanıcı formu doldurur, ön yüz hata vermez ama "Gönder" dediğinde backend'den hata döner.

Çözüm: React Hook Form, projedeki Zod şemalarını doğrudan kullanabilir (aradaki bağlantıyı [@hookform/resolvers](#) sağlar). Backend'de doğrulamayı yapan o Zod kuralını — örneğin `price: z.number().min(10)` — alır, frontend formunun içine bağlarsınız.

```
// Sunucunun doğrulama için kullandığı şemanın AYNISINI alıyoruz.
// Ayrı bir kural yazmıyoruz — tek kaynak.
import { CreateWorkOrder } from '@contracts/work-order';

const form = useForm({
  // Köprü burası: Zod şemasını React Hook Form'a tanıtan çevirici
  resolver: zodResolver(CreateWorkOrder),
});

// Form alanını kütüphaneye bağlıyoruz — yazılan her harfi o takip ediyor

```

★ Hata mesajı bile şemadan geliyor — backend hangi metni döndürüyorsa kullanıcı "Gönder"e basmadan **aynı metni** görüyor.

Böylece doğrulama kuralı **tek bir merkezden** yönetilir. Kural değiştikçe hem backend hem frontend aynı kuralı çalıştırır. Kullanıcı daha "Gönder" butonuna basmadan, backend ile birebir aynı kurallara göre hata mesajlarını ekranda görür.

Özetle

Araç	Ne katıyor
Tailwind CSS	Tasarımı hızlandırır
shadcn/ui	Form, buton ve tabloları hazır verir ama kontrolü size bırakır
React Hook Form + Zod	Formların donmadan çalışmasını sağlar ve backend kurallarıyla frontend kurallarını eşitleyerek hata yapılmasını engeller

C.21 @dbml/cli — veri modeli dokümanı (Database Markup Language)

Nedir: Normalde bir veritabanında hangi tabloların olduğunu, bu tabloların içinde hangi kolonların yer aldığını ve tabloların birbirine nasıl bağlandığını (ilişkilerini) görmek için karmaşık SQL kodlarına bakmak zordur.

DBML, veritabanı şemasını insanların çok rahat okuyabileceği **sade bir metin formatına** dönüştürür. En güzel yanı ise şu: bu metni dbdiagram.io gibi sitelere yapıştırdığınızda, size otomatik olarak kutucuklar ve çizgilerden oluşan **görsel bir veritabanı diyagramı (ERD)** çizer.

Ne işe yarar: Veri modelinin görsel ve paylaşılabılır dokümanını üretir. **Bu projede nerede:** [docs/database.dbml](#). **Ödevdeki karşılığı:** §11 — bu dosya **zorunlu**.

Üretilen dosya şuna benzer — SQL bilmeyen biri de okuyabilir:

```
Table WorkOrder {
  id          int          [pk, increment]  // pk = birincil anahtar, otomatik artar
  number      varchar      [unique, note: 'IE-2026-000148'] // benzersiz, insan okur
  status      varchar      [not null]       // boş bırakılamaz
  assetId     int          [not null]       // hangi varlığa ait
  slaDueAt    timestamp                    // SLA bitiş zamanı (boş olabilir)
}

// İlişki tanımı: bir varlığın birden çok iş emri olabilir.
// dbdiagram.io gibi araçlar bu satırı okuyup iki kutu arasına ok çiziyor.
Ref: WorkOrder.assetId > Asset.id
```

Problem: elle yazılan doküman ilk şema değişikliğinde eskir

Diyelim ki projeye başlarken elle bir DBML dosyası yazdınız ve ödevi teslim ettiniz. İki gün sonra projedeki bir kural değişti ve veritabanındaki **User** tablosuna **phoneNumber** adında yeni bir kolon eklemek zorunda kaldınız.

Gittiniz, veritabanı kodlarını güncellediniz (migration attınız). Ama o yoğunlukta unutup DBML doküman dosyasını elle güncellemediniz.

Sonuç: Veritabanı ile doküman birbirinden farklı oldu — uyumsuzluk doğdu. Ödevi inceleyen değerlendirmeci veya iş yerindeki bir başka geliştirici dokümana bakarak kod yazmaya çalışırsa sistem patlar.

Ödevdeki §11 maddesi tam olarak bu uyumsuzluğu yasaklıyor.

Çözüm: @dbml/cli ile otomatik üretim

Bu projede doküman **asla elle yazılmıyor**. Süreç şöyle işliyor:

1. Yazılımcı Prisma üzerinden veritabanında bir değişiklik yapıyor (örneğin yeni bir tablo ekliyor)
2. Değişiklik bittiği an `@dbml/cli` aracı devreye giriyor
3. Bu araç, **fiilen değişen canlı veritabanı şemasına** bakıyor ve saniyeler içinde `docs/database.dbml` dosyasını baştan, hatasız biçimde yeniden üretiyor

Kodda ne değiştiyse, dokümanda da saniyesinde o değişiyor.

CI sürecinde kontrol — "build'in kırmızı yanması" ne demek

Peki ya yazılımcı bu otomatik aracı kendi bilgisayarında çalıştırmayı unutup kodu sunucuya yüklerse? Projenin **emniyet kilidi** burada devreye giriyor.

CI (Continuous Integration / sürekli entegrasyon), kodu depoya yüklediğiniz an sunucuda otomatik çalışan bir test robotudur.

1. Robot kodu alır almaz `@dbml/cli` aracını çalıştırır ve yeni bir DBML üretir
2. Sizin yüklediğiniz DBML dosyası ile kendi ürettiği güncel dosyayı karşılaştırır
3. İki dosya arasında **tek bir karakter bile fark varsa** robot şunu der: *"Yazılımcı veritabanını değiştirmiş ama dokümanı güncellemeyi unutmuş!"*
4. Build sürecini iptal eder (kırmızı yakar) ve o kodun canlıya geçmesini engeller

```
# CI adımı – doküman güncel değilse build burada durur

# 1) Veritabanının GERÇEK yapısını dışa aktar (veriyi değil, sadece şemayı)
- run: pg_dump --schema-only "$DATABASE_URL" > /tmp/schema.sql

# 2) O yapıyı okunabilir DBML biçimine çevir
- run: npx sql2dbml /tmp/schema.sql --postgres -o /tmp/database.dbml

# 3) Yeni üretilen dosyayla depodakini karşılaştır.
# Tek karakter fark varsa diff hata koduyla çıkar ve CI kırmızı yanar.
- run: diff /tmp/database.dbml docs/database.dbml
```

diff komutu iki dosya arasında fark bulursa sıfırdan farklı bir kodla çıkar ve CI durur.

Yazılımcı hatasını düzelterip dokümanı güncelleyene kadar sistem geçişi izin vermez.

Özetle

"Uyumludur" ifadesinin bir iddia olmaktan çıkıp **mekanizmaya** dönüşmesi şu demektir: yazılımcı sözlü olarak *"dokümanım kodla uyumlu, kontrol ettim"* demez. Sistem, doküman güncel değilse zaten kodun geçmesine izin vermeyerek bu uyumluluğu **mekanik olarak zorunlu** kılar.

C.22 Renovate

Nedir: Kullanılan paketlerin yeni sürümlerini takip edip otomatik güncelleme önerisi açan bot. **Bu projede neredede:** Depo genelinde, haftalık. **Ödevdeki karşılığı:** İstenmemiş; sürdürülebilirlik için eklendi.

Çözdüğü gerçek sorun

Gerçek hayat: Bir binada 40 yangın tüpü var ve her birinin dolum tarihi farklı. "Ara sıra kontrol ederiz" denince hiç kontrol edilmez; yangın çıktığında yarısının boş olduğu anlaşılır.

Yazılımda: Bir projede 40–60 paket kullanılır ve hepsi zaman zaman güncellenir. Bazı güncellemeler **güvenlik açığı kapatır**. Bunları elle takip etmek gerçekçi değildir — kimse yapmaz, iki yıl sonra proje eski ve açık bir sürümde kalır.

Nasıl çalışıyor

```
// Dosya: renovate.json – güncelleme botunun ayarları
{
  // Hazır önerilen ayar setini temel al
  "extends": ["config:recommended"],

  // Ne zaman tarasın: pazartesi sabahı, mesai başlamadan
  "schedule": ["before 6am on monday"],

  "packageRules": [
    // Küçük güncellemeleri TEK bir öneride topla (haftada bir istek)
    { "matchUpdateTypes": ["minor", "patch"], "groupName": "küçük güncellemeler" },

    // Büyük sürüm atlamaları genelde kırıcı değişiklik taşır –
    // otomatik açılmasın, önce onay beklesin
    { "matchUpdateTypes": ["major"], "dependencyDashboardApproval": true }
  ]
}
```

Her hafta tarar ve güncelleme için otomatik değişiklik önerisi açar.

★ **Asıl değerli kısım:** O öneri açıldığında **tüm test zinciri onun üzerinde koşar**. Güncelleme bir şeyi bozuyorsa, birleştirilmeden önce görülür. Yani Renovate sadece haber vermiyor, **güvenli olduğunu da kanıtlıyor**.

Küçük güncellemeler tek öneride gruplanıyor (haftada bir tane), büyük sürüm atlamaları ise ayrı ayrı ve **onay bekleyerek** geliyor — çünkü büyük sürüm genellikle kırıcı değişiklik taşır.

Neden Dependabot değil

Dependabot yalnızca GitHub'da çalışır. Bu proje kurumun **GitLab**'ına taşınacak; Renovate ikisinde de çalışıyor. Araç, platform değişince taşınabilir olsun diye seçildi.

Neden ödevde olmadığı hâlde eklendi

"Bağımlılıkları güncel tutalım" bir niyet olarak kalırsa yapılmaz. Bu proje yıllarca sürdürülecek ve devredilecek; niyeti **otomatik bir süreç** çevirmek devralınabilirliğin parçası.

C.23 Expo — ileride mobil için

Nedir: React Native'in üstüne kurulmuş araç takımı; React bilgisiyle telefon uygulaması yazmayı sağlar. **Bu projede nerede:** Şu an kapsam dışı; **mimari buna hazır tasarlandı. Ödevdeki karşılığı:** İstenmemiş.

Gerçek hayat karşılığı

Bir aracın motoru ile o motorun etrafına kurulmuş araç arasındaki fark gibi. Motor tek başına da çalışır ama sürülebilmek için şanzıman, direksiyon ve gösterge paneli gerekir. React Native motordur; Expo o motoru sürülebilir hâle getiren takımdır.

İkisi arasındaki ilişki

React Native, React ile **gerçek native mobil uygulama** yazma framework'üdür — çıktı bir web sayfası değil, App Store veya Play Store'a yüklenebilen gerçek bir uygulamadır.

Expo bunun üstünde çalışan araç takımıdır: Xcode veya Android Studio kurmadan geliştirme, tek komutla derleme, kamera ve bildirim gibi özelliklerin hazır gelmesi, mağazaya göndermeden güncelleme.

⚠ "Expo mu React Native mi" bir seçim değil — Expo kullanınca React Native'i zaten kullanıyorsunuz. Nest/Express ilişkisinin aynısı (C.1).

Mobil yapılmadığı hâlde neden şimdiden kararı verildi

Çünkü mobilin varlığı **backend tasarımını değiştirir**, ve bu kararlar sonradan alınırsa API'yi baştan yazmak gerekir:

Karar	Mobil olmasa	Mobil olacaksa
Sürümleme	Gerekmeyebilir — web güncellenince herkes yeni sürümü alır	Zorunlu — uygulama kullanıcının telefonunda eski sürümde kalır
Jeton (token) taşıma	Sadece çerez yeterdi	Çerez ve Authorization başlığı birlikte desteklenmeli
Sayfalama	Gevşek olabilirdi	Zorunlu — mobilde 5000 kayıt uygulamayı dondurur
Cevap boyutu	Önemsiz	Kritik — mobil veri kotası

Bu yüzden API baştan `/api/v1/...` biçiminde sürümlendi ve **istemci tanımaz** şekilde tasarlandı (E.10). Mobil eklendiğinde backend'e dokunmak gerekmeyecek.

★ Ödevin kendisi mobil istemiyor. Ama kurumun mevcut sistemlerinde mobil uygulamalar var; bu sistemin de eninde sonunda mobilden tüketileceğini varsaymak gerçekçi bir kabul.

Nasıl geliştirilir

Kod yazma yeri değişmez — aynı editör, aynı dil. Fark yalnızca çalışanı görmede:

	Web	Mobil
Çalıştırma	<code>pnpm dev</code>	<code>npx expo start</code>
Görme	Tarayıcı	Telefonda Expo Go uygulaması, QR kod okutularak
Xcode / Android Studio	Gerekmez	Çoğu iş için gerekmez

Mobil eklendiğinde API tarafında yazılacak tek şey, jetonu (token) başlıktan da kabul etmek — ve o zaten baştan yazılı:

```
// Jetonun nereden okunacağı. Sıra önemli: önce çerez, olmazsa başlık.
jwtFromRequest: ExtractJwt.fromExtractors([

  // WEB: tarayıcı jetonu çerezde taşıyor.
  // Çerez httpOnly olduğu için JavaScript okuyamıyor → XSS'te çalınamıyor.
  (req) => req?.cookies?.access_token,

  // MOBİL ve Swagger: çerez kavramı yok, jeton başlıkta geliyor.
  // "Authorization: Bearer eyJhbGc..." biçiminde.
  ExtractJwt.fromAuthHeaderAsBearerToken(),
]),
// Aynı doğrulama kodu iki taşıma yolunu birden tanıdığı için
// mobil eklendiğinde sunucu tarafında değişiklik gerekmiyor.
```

BÖLÜM E — Kavramlar, prensipler ve desenler

Önceki bölümde anlatılanlar **kurulan paketlerdi**. Bu bölümdekiler ise kurulmaz; **karar verilir**. Ödevin en çok puan verdiği kısımlar da burasıdır, çünkü paket kurmak kolaydır — doğru kararı verip tutarlı uygulamak zordur.

E.0 Önce temel kelimeler

Bu bölümdeki her şey birkaç temel kavrama dayanıyor. Her birini üç adımda açıyoruz: **gerçek hayattan karşılığı** → **yazılımdaki tanımı** → **bu projede tam olarak nerede**.

★ Önce sözlük: aynı şeyin Türkçesi ve İngilizcesi

Bu belgede bazı şeyler hem Türkçe hem İngilizce adıyla geçiyor. Karşılıklarını bir kere burada veriyorum; metinde her seferinde tekrar etmiyorum.

Bu belgede yazan	İngilizcesi	Ne demek
uç, uç noktası	endpoint	API'nin dışarıya açtığı tek bir adres: <code>POST /api/v1/work-orders</code>
arayüz (UI)	user interface	Kullanıcının gördüğü ekran : sayfa, buton, form, tablo
arayüz (kod bağlamında)	interface	Bir sınıfın söz verdiği metot listesi — ekranla ilgisi yok (E.2 → I)
arka uç	backend	Sunucuda çalışan taraf: API + veritabanı + worker
ön uç	frontend	Tarayıcıda çalışan taraf — bu projede Next.js arayüzü
istemci	client	API'yi <i>tüketen</i> taraf: web arayüzü, mobil uygulama, başka bir kurum
jeton	token	Kullanıcının kim olduğunu kanıtlayan, süresi olan dijital bilet
kap	container	Uygulamayı bağımlılıklarıyla paketleyen izole çalışma birimi (Docker)
kuyruk	queue	Sonra yapılacak işlerin sıraya alındığı yer (BullMQ)
iş	job	Kuyruğa bırakılan tek bir görev
kütüphane	library	Sen çağırırsın, işini yapar, geri döner (Zod)
çatı	framework	O seni çağırır — iskeleti o kurar (NestJS, Next.js)

⚠ **En çok karıştırılan iki satır 2. ve 3. satırdır.** Türkçede tek kelime ("arayüz") İngilizcede iki farklı kavramı karşılıyor. Bu belgede ekran kastediliyorsa **arayüz (UI)** yazıyorum; kod sözleşmesi kastediliyorsa yalnızca *arayüz* yazıp bağlamı belirtiyorum.

★ Kısaltmalar — hepsinin açılımı ve ne demek olduđu

Metinde geen her kısaltma burada bir kez açılıyor. Bir kısaltmayı ilk kez gördüğünde buraya bak.

Kısaltma	Açılımı	Ne demek
API	<i>Application Programming Interface</i>	Bir programın başka programlara açtığı kapı. İnsan arayüzü ekrandır; program arayüzü API'dir
REST	<i>Representational State Transfer</i>	API yazmanın en yaygın biçimi : her kaynağın bir adresi var, işlemler HTTP fiilleriyle yapılır
HTTP	<i>HyperText Transfer Protocol</i>	Tarayıcı ile sunucunun konuştuğu dil/kural seti . GET , POST gibi fiiller buradan gelir
SQL	<i>Structured Query Language</i>	Veritabanına soru sorma dili: SELECT * FROM work_order WHERE ...
ORM	<i>Object-Relational Mapping</i>	SQL yazmak yerine nesnelerle çalışmayı sağlayan katman. Bu projede Prisma
CRUD	<i>Create, Read, Update, Delete</i>	Bir kayıt üzerindeki dört temel işlem: oluştur, oku, güncelle, sil
DTO	<i>Data Transfer Object</i>	★ Dışarı gönderilen cevabın şekli . Veritabanı kaydının aynısı değildir — şifre özeti gibi alanlar burada bulunmaz (E.6)
POJO	<i>Plain Old Java Object</i>	"Süslemesiz düz nesne". Prisma'nın döndürdüğü şey budur: sadece veri, davranış yok
DI	<i>Dependency Injection</i>	Bir sınıfın ihtiyacı olan şeyi kendisi yaratmaz, dışarıdan verilir (C.1)
JWT	<i>JSON Web Token</i>	İçinde kullanıcı bilgisi taşıyan, imzalı ve süreli jeton (token) biçimi
XSS	<i>Cross-Site Scripting</i>	Saldırganın sayfaya kendi JavaScript'ini çalıştırtması. httpOnly çerez buna karşı korur
IDOR	<i>Insecure Direct Object Reference</i>	Adresteki numarayı değiştirip başkasının kaydını görmek. Her uçta sahiplik kontrolü şart
CORS	<i>Cross-Origin Resource Sharing</i>	Tarayıcının "başka adresteki API'ye istek atılabilir mi" kuralı
CI / CD	<i>Continuous Integration / Delivery</i>	Her gönderimde testleri otomatik koşturma / yayına alma
ADR	<i>Architecture Decision Record</i>	"Şu kararı şu yüzden aldık" belgesi (E.12)
SLA	<i>Service Level Agreement</i>	Hizmet süre taahhüdü: "kritik arıza şu kadar sürede çözülür"
DBML	<i>Database Markup Language</i>	Veritabanı şemasını okunabilir metin olarak yazma biçimi (C.9)
GIN	<i>Generalized Inverted Index</i>	PostgreSQL'de "içinde geçiyor mu" aramalarını hızlandıran index türü (C.5)
SSR	<i>Server-Side Rendering</i>	Sayfanın HTML'inin sunucuda üretilmesi; ilk açılış hızlanır (C.2)

Kısaltma	Açılımı	Ne demek
TTL	<i>Time To Live</i>	Bir şeyin geçerlilik süresi: jetonun (token) ömrü, önbelleğin tazeliği
UI / UX	<i>User Interface / User Experience</i>	Kullanıcı arayüzü / kullanıcı deneyimi
PR / MR	<i>Pull Request / Merge Request</i>	Değişiklik önerisi. GitHub'da PR, GitLab'da MR denir — aynı şey
E2E	<i>End-to-End</i>	Uçtan uca test: gerçek tarayıcıda gerçek tıklama (C.12)
LTS	<i>Long Term Support</i>	Uzun süre desteklenecek sürüm. Node'da bu yüzden çift numaralı sürümler seçilir
KVKK	<i>Kişisel Verilerin Korunması Kanunu</i>	Türkiye'nin kişisel veri mevzuatı (C.22)
MVCC	<i>Multi-Version Concurrency Control</i>	PostgreSQL'in aynı satırı okuyan/yazanları birbirine kilitletmeden yönetme yöntemi

Sınıf ve nesne

Gerçek hayat: Kurabiye kalıbı ile kurabiye. Kalıp tektir; ondan yüzlerce kurabiye çıkar. Her kurabiyenin şekli aynıdır ama biri çilekli, biri kakaolu olabilir.

Yazılımda: **Sınıf** kalıptır — hangi bilgileri tuttuğu ve neler yapabildiği yazar. **Nesne** o kalıptan üretilmiş tek bir örnektir.

Bu projede:

```
// SINIF = kalıp. "Bir iş emri neye benzer, ne yapabilir?" sorusunun cevabı.
// Burada henüz gerçek bir iş emri yok — sadece tanımı var.
class WorkOrder {
  id: number; // Veritabanının verdiği benzersiz numara
  number: string; // İnsanın okuyacağı numara: "IE-2026-000148"
  status: Status; // Hangi aşamada: OPEN | ASSIGNED | IN_PROGRESS ...
  assigneeId: number | null; // Atanan teknik personel. null = henüz atanmamış
  resolutionNote: string | null; // Çözüm açıklaması. Kapatılana kadar boş
}

// NESNE = o kalıptan üretilmiş TEK BİR gerçek kayıt.
// Veritabanındaki her satır, kod tarafında böyle bir nesneye dönüşüyor.
const isEmri = { id: 1042, number: 'IE-2026-000148', status: 'OPEN', ... };
```

Veritabanındaki **WorkOrder** tablosunun her satırı, bu kalıptan üretilmiş bir nesnedir. 5.000 iş emri = 1 sınıf, 5.000 nesne.

Metot

Gerçek hayat: Çamaşır makinesinin düğmeleri. Makine **ne olduğunu** (marka, kapasite) taşır; düğmeler **ne yapabildiğidir** (yıkla, sık, durula).

Yazılımda: Sınıfın yapabildiği iş. Veri "ne olduğu", metot "ne yapabildiği".

Bu projede — ekrandaki butondan koda kadar zincir:

Kullanıcı iş emri detay ekranında **"Kapat"** butonuna basıyor. Arkada şu olur:

```
// 1. ADIM – Ekran (Next.js): kullanıcı "Kapat" butonuna bastı.
//   Tarayıcı, 1042 numaralı iş emri için API'ye istek gönderiyor.
//   Kullanıcının yazdığı çözüm açıklaması da isteğin içinde gidiyor.
POST /api/v1/work-orders/1042/close { resolutionNote: "Sigorta değiştirildi" }

// 2. ADIM – API katmanı (NestJS controller): isteği karşılayan yer.
//   Burada iş kuralı YOK; sadece gelen bilgiyi alıp servise devrediyor.
close(id, dto) {
  return this.service.close(id, dto.resolutionNote); // işi asıl yapan yere gönder
}

// 3. ADIM – Domain katmanı: kuralın fiilen çalıştığı yer.
//   Bu sınıf veritabanını da interneti de tanımıyor; sadece kuralı biliyor.
class WorkOrder {
  close(note: string, clock: Clock) {

    // Kural 1: iş emri "Çözüldü" aşamasında değilse kapatılamaz
    if (this.status !== 'RESOLVED') throw new InvalidTransition(this.status);

    // 🚫 Kural 2: çözüm açıklaması boş bırakılamaz (ödevin §6 şartı)
    if (!note?.trim()) throw new ResolutionNoteRequired();

    // Kurallar geçildi – kaydın durumunu değiştir
    this.status      = 'CLOSED';
    this.closedAt    = clock.now(); // saati doğrudan değil Clock servisinden al
  }
}
```

★ Dikkat: kural (**çözüm açıklaması boşsa kapatma**) butonda değil, ekranda değil, controller'da değil — **nesnenin kendi metodunda**. Böylece iş emri mobil uygulamadan da, arka plan işinden de kapatılsa aynı kural çalışır.

Arayüz (interface) — ⚠ ekran DEĞİL, kod sözleşmesi

Gerçek hayat: Elektrik prizi. Priz bir **söz** verir: "*iki delik, 220 volt.*" Fişi takan cihazın ütü mü, şarj aleti mi, buzdolabı mı olduğunu umursamaz. Cihaz da prizin arkasındaki kablonun bakır mı alüminyum mu olduğunu bilmez. **Söz ortaktır, arkası serbesttir.**

Yazılımda: Arayüz, "bu işi yapan her şey şu metotlara sahip olmalı" diyen bir sözleşmedir. **Nasıl** yapıldığını söylemez.

Neden kullanılır: Çağırın kodun, **hangi somut sınıfın** geldiğini bilmek zorunda kalmaması için. Böylece yenisi eklendiğinde çağırın kod değişmez.

Bu projede — SLA politikaları:

```
// SÖZLEŞME (arayüz): "SLA hesaplayan her sınıf şu iki işi yapabilmeli."
// Burada hesap YOK — sadece "ne yapabilmesi gerektiği" yazıyor.
interface SlaPolicy {
    supports(ctx: SlaContext): boolean;    // Soru: bu iş emri bana uyuyor mu?
    calculate(ctx: SlaContext): SlaPlan;    // Soru: süreleri hesapla ve geri ver
}

// SÖZLEŞMEYİ UYGULAYAN 1 — kritik varlıkta arıza çıktıysa 4 saat verir
class CriticalAssetBreakdownPolicy implements SlaPolicy { ... }

// SÖZLEŞMEYİ UYGULAYAN 2 — düşük öncelikli bakım işine 15 gün verir
class LowPriorityMaintenancePolicy implements SlaPolicy { ... }

// ★ Bu ikisini kullanan kod, hangisinin geldiğini BİLMEK ZORUNDA DEĞİL.
// Tek bildiği: "elimde SlaPolicy sözü veren bir şey var, calculate
// diyebilirim." Yeni politika eklenince çağırın kod hiç değişmiyor.
```

Factory (E.4) bu politikaları **SlaPolicy** olarak tanır — hangisinin geldiğini bilmez, sadece **sözü** bilir. Yarın yeni bir politika eklendiğinde factory'ye dokunulmaz.

★ Bu tek kavram, SOLID'in **O** ve **D** harflerinin (E.2) ve Factory deseninin (E.4) tamamının dayandığı fikirdir.

Katman

Gerçek hayat: Restoran. **Salon** (garson) sipariş alır, **mutfak** (aşçı) pişirir, **depo** malzeme tutar. Garson yemek pişirmez, aşçı sipariş almaz.

Kritik olan şu: **aşçı değişse garsonun işi değişmez**. Yeni aşçı aynı yemeği pişirdiği sürece salon hiçbir şey fark etmez.

Yazılımda: Sorumluluğa göre ayrılmış bölümler. Her katman kendi işini yapar ve komşusuyla belirli bir noktadan konuşur.

Bu projede dört katman var:

Katman	Ne yapar	Bu projedeki örnek
API (controller)	HTTP isteğini alır, cevabı döner	<code>POST /work-orders/1042/close</code> ucu
Application (servis)	Adımları sıraya koyar, transaction açar	"Kapat" işleminin akışı: kural çalıştır → kaydet → geçmiş yaz
Domain	Saf iş kuralları	" <i>Kapalı iş emri güncellenemez</i> "
Infrastructure	Dış dünya: veritabanı, kuyruk, e-posta	Prisma sorguları, BullMQ işleri

"Biri değişse diğeri etkilenmez" bu projede ne demek — somut üç örnek:

Değişiklik	Etkilenen	Etkilenmeyen	Neden
Prisma'dan başka bir ORM'e geçilse	Yalnızca infrastructure	Domain ve application	Domain, Prisma'yı hiç tanımıyor
REST yerine GraphQL eklense	Yalnızca API katmanı	Domain, application, infrastructure	Kurallar HTTP'yi bilmiyor
"Çözüm açıklaması artık 20 karakter olmalı"	Yalnızca domain	API, veritabanı	Kural tek yerde yaşıyor

🚫 **Katman ihlali neye benzer:** Controller'ın doğrudan `prisma.workOrder.update()` çağırması. Çalışır — ama o an iş kuralı atlanmış olur ve aynı işlem mobilden geldiğinde kural uygulanmaz. Bu yüzden ihlal `dependency-cruiser` (C.8) ile CI'da engelleniyor.

Bağımlılık

Gerçek hayat: Kahve makineniz belirli bir kapsül markasına bağımlıysa, o marka üretimi durdurduğunda makineniz çöp olur. Bağımlılık tek yönlü bir borçtur.

Yazılımda: Bir kodun çalışmak için başka bir koda ihtiyaç duyması. A, B'ye bağımlıysa **B değiştiğinde A bozulabilir**.

Bu projede neden kritik: Mimarideki asıl soru "*kim kime bağımlı olacak*" sorusudur. Domain katmanı Prisma'ya bağımlı olsaydı, veritabanı değiştiğinde iş kuralları da bozulurdu. Bu yüzden ok **tersine çevriliyor** (E.2 → D harfi).

Projeksiyon

Gerçek hayat: Restoranda 60 kalemlik menü var ama garsona "*sadece tatlıları söyle*" diyorsunuz. Garson 60 kalemi baştan sona okuyup siz seçmiyorsunuz — **isteneni getiriyor**.

Yazılımda: Bir kaydın **yalnızca ihtiyaç duyulan alanlarını** veritabanından istemek. `SELECT *` değil, `SELECT id, title, status`.

Bu projede hangi somut sorunu çözüyor:

İş emri tablosunda ~25 kolon var: açıklama metni, çözüm notu, tüm zaman damgaları, audit alanları, iç notlar. Ama **liste ekranı 7 tanesini gösteriyor**: numara, başlık, durum, öncelik, atanan kişi, SLA bitişi, lokasyon.

Sayfada 20 kayıt varsa:

	Alan sayısı	Taşınan veri
Projeksiyonsuz (<code>SELECT *</code>)	20 × 25 = 500 alan	Açıklama metinleri dahil — kayıt başına birkaç KB
Projeksiyonlu (<code>select</code>)	20 × 7 = 140 alan	Yalnızca listede görünenler

Aradaki fark yalnızca hız değil: **açıklama metni ve çözüm notu gibi uzun alanlar hiç okunmuyor**. Bu alanlar detay ekranında zaten ayrıca isteniyor.

En kritik faydası güvenlik tarafında: Şemada olmayan alan cevaba çıkamıyor. Kullanıcı tablosunda `passwordHash` var; liste şemasında yok, o yüzden sorguya da girmiyor, cevaba da (E.6).

```
// Veritabanına "bu kaydın hangi alanlarını istiyorum" diye söylüyoruz.  
// 🚫 true yazılmayan hiçbir kolon çekilmiyor – açıklama metni, çözüm notu,  
//   audit alanları ve hassas alanlar veritabanından hiç çıkmıyor.  
select: { id: true, number: true, title: true, status: true,  
          priority: true, assigneeId: true, slaDueAt: true }
```

Ödevin §17 maddesi bunu doğrudan istiyor: "*Listeleme işlemlerinde bütün tablo belleğe alınarak mapping yapılmamalıdır*."

SLA (Service Level Agreement — Hizmet Seviyesi Taahhüdü)

Gerçek hayat: İnternet servis sağlayıcınızla sözleşmeniz der ki: "*arıza bildirimini 24 saat içinde çözeriz.*" Bu bir **taahhüttür**; tutulmazsa raporlanır, gerekirse yaptırımı olur.

Yazılımda / bu projede: Bir iş emrinin **ne kadar sürede çözülmesi gerektiğine** dair kurumsal taahhüt. Üç zaman üretir:

Alan	Ne demek	Örnek (kritik jeneratör arızası)
dueAt	Bitmesi gereken an	Açılıştan 4 saat sonra
remindAt	İlk hatırlatma	Bitişe 1 saat kala
escalateAt	Üst amire yükseltme	Bitişe 30 dakika kala

Süre neye göre belirleniyor: üç girdi — iş emrinin **önceliği**, varlığın **kritiklik seviyesi**, iş emrinin **türü**.

Bu projede tam olarak nerede kullanılıyor:

- İş emri oluşturulurken** hesaplanır ve kaydedilir
- Hatırlatma işi planlanır** (BullMQ gecikmeli iş — C.6)
- Tarama işi** her 15 dakikada bir süresi geçenleri bulur ve ihlal işaretler
- Liste ekranında** "SLA ihlali" filtresi bu alana bakar
- Yönetim ekranında** ihlal sayısı buradan sayılır

Süre nereye yazılıyor — kod akışı:

```
// 1. ADIM – Kullanıcının formdan gönderdiği üç bilgiyi factory'ye veriyoruz.
//   Factory bunlara bakıp HANGİ SLA kuralının geçerli olduğunu seçiyor.
const policy = this.slaFactory.resolve({
  priority: dto.priority,           // Kullanıcının seçtiği öncelik: HIGH
  assetCriticality: asset.criticality, // Varlık kaydından okundu: HIGH
  type: dto.type,                 // İş emri türü: BREAKDOWN (arıza)
});

// 2. ADIM – Seçilen kural üç zamanı hesaplıyor.
const plan = policy.calculate(ctx);
// Sonuç → { dueAt: 14:00,      bitmesi gereken an
//           remindAt: 13:00,   hatırlatma zamanı
//           escalateAt: 13:30 } üst amire bildirme zamanı

// 3. ADIM – Hesaplanan zamanlar iş emriyle AYNI SATIRA yazılıyor.
//   Ayrı tabloya konsaydı, liste ekranındaki "SLA'sı yaklaşanlar" filtresi
//   her seferinde iki tabloyu birleştirmek zorunda kalırdı.
await tx.workOrder.create({
  data: { ...dto, slaDueAt: plan.dueAt, slaRemindAt: plan.remindAt },
});

// 4. ADIM – Hatırlatmayı şimdi değil, ileri bir saatte çalışacak şekilde
//   kuyruğa bırakıyoruz. Kimse ekranı açmasa da o saatte kendisi çalışacak.
await this.queue.add('sla-reminder', { workOrderId: created.id },
  { delay: plan.remindAt.diffNow().milliseconds, // kaç milisaniye sonra çalışsın
    jobId: `sla-reminder-${created.id}` });      // aynı iş iki kez sıraya girmesin
```

★ **slaDueAt** 'in iş emriyle **aynı satırda** tutulması bilinçli: liste ekranında "SLA'sı yaklaşanlar" filtresi ek bir tablo birleştirmesi (JOIN) gerektirmeden çalışıyor ve bu kolona index konabiliyor.

E.1 Modüler monolit + Clean Architecture (§10)

Ne isteniyor: Ödev mimarinin tarafımızdan tasarlanmasını ve **gerekçelendirilmesini** istiyor.

"Monolit" ne demek

Gerçek hayat: Tek binalı bir hastane. Poliklinik, laboratuvar, eczane — hepsi aynı binada, aynı kapıdan girilir. Bir bölümden diğerine geçmek koridorda yürümektir: hızlı ve sorunsuz.

Karşıtı — mikroservis: Aynı hastanenin bölümleri şehrin farklı noktalarında ayrı binalarda. Hasta laboratuvara gitmek için **dışarı çıkıp taksiye biniyor**. Trafik varsa gecikir, taksi bulunamazsa hiç gidemez.

Yazılımda:

	Monolit	Mikroservis
Kaç program	Tek	Modül başına ayrı program
Modüller nasıl konuşur	Doğrudan fonksiyon çağırısı — mikrosaniye	Ağ üzerinden — milisaniye + hata riski
Nerede çalışır	Tek süreç	Ayrı süreçler, genelde ayrı sunucular
Yayın	Tek seferde hepsi	Her servis ayrı ayrı

i Sık karıştırılan nokta — mikroservisler nasıl konuşur

Mikroservisler genellikle **HTTP/REST API** ile veya bir **mesaj kuyruğu** üzerinden haberleşir. Bu trafik internette değil, kurumun **iç ağından** geçer — yani "internet üzerinden" değil, "**ağ üzerinden**".

Fark önemsiz görünüyor ama sonucu büyük: servisler aynı veri merkezinde bile olsa aradaki her çağrı bir **ağ çağırısıdır ve başarısız olabilir**. Monolitte bir fonksiyon çağırısı başarısız olmaz; mikroserviste olur ve bu durumun yönetilmesi (yeniden deneme, zaman aşımı, kısmi başarısızlık) mimarinin parçası hâline gelir.

Bu projede hangi modüller monolit içinde, hangileri değil

Bu projede modüllerin tamamı tek programda (monolit içinde):

Modül	Neden monolit içinde
Lokasyon	İş emriyle aynı transaction içinde okunuyor (pasif lokasyon kontrolü)
Varlık	Aynı — iş emri açılırken varlık durumu kontrol ediliyor
İş emri	Sistemin çekirdeği; diğer her şey buna bağlı
Bildirim	İş emri değişiklikleriyle aynı transaction içinde yazılıyor
Kullanıcı/yetki	Her istekte kontrol ediliyor, ağ gecikmesi kaldırılamaz

★ **Belirleyici ölçüt: transaction sınırı.** İki modülün verisi "*ya ikisi birden ya hiçbiri*" kuralıyla yazılıyorsa, onları ayrı programlara bölmek transaction'ı **kaybettirir**. İş emri durumu değişip geçmiş kaydı yazılmazsa sistem tutarsız kalır (E.5). Mikroserviste bunu çözmek için *saga* denen çok daha karmaşık bir mekanizma kurulur — bu projede karşılığı olmayan bir maliyettir.

Ayrı süreçte çalışan tek parça: worker.

Ama dikkat — worker bir **mikroservis değildir**. Aynı kod tabanını, aynı veritabanını ve aynı iş kurallarını kullanır; sadece **farklı bir süreç olarak** başlatılır. Ayrılma sebebi iş bölümü değil, **çalışma biçimi**: API isteğe

cevap verir, worker istek olmadan çalışır (C.6).

ileride hangisi mikroservise ayrılabilir: Bildirim modülü. Sebebi: e-posta ve SMS gönderimi eklendiğinde dış servislere bağımlı hâle gelir, yavaşlar ve ayrı ölçeklenmesi gerekebilir. Sınırı **şimdiden çizili** olduğu için o gün geldiğinde ayırmak kolay olacak — modüler monolitin asıl vaadi budur.

"Modüler" ne katıyor

Tek program, ama içi net sınırlı modüllere bölünmüş. Modüller birbirinin iç koduna elini sokmaz; yalnızca **ilan edilmiş arayüzünü** çağırır.

```
src/modules/  
├─ locations/      → LocationsModule  (dışarı sadece LocationsService açılır)  
├─ assets/         → AssetsModule  
├─ work-orders/    → WorkOrdersModule  
└─ notifications/ → NotificationsModule
```

🚫 **Yasak olan:** `work-orders` modülünün `notifications` modülünün Prisma sorgusunu doğrudan çağırması. **Serbest olan:** ilan edilmiş servisini çağırması. Fark şudur: ilki değişirse ikisi birden bozulur, ikincisi sözleşme olduğu için korunur.

Clean Architecture'ın tek kuralı

Bağımlılık oku hep içe bakar.

```
infrastructure  (Prisma, HTTP, BullMQ, Redis)  
    ↓ bilir  
application    (use case: "iş emrini kapat")  
    ↓ bilir  
domain         (saf iş kuralları — hiçbir kütüphane bilmez)
```

Dış katman içeriği bilir; **iç katman dışarıyı bilmez.**

Somut karşılığı: "*Kapatılmış iş emri güncellenemez*" kuralı en içte yaşar ve Prisma'yı da NestJS'i de HTTP'yi de tanımaz.

İki faydası:

1. O kuralı test etmek için **veritabanı gerekmez** — test milisaniyede koşar
2. Yarın Prisma'dan başka bir araca geçilse **iş kuralları hiç bozulmaz**

★ **Ödevle birebir örtüşen nokta:** §8 şunu şart koşuyor: "Domain katmanı Entity Framework Core, Hangfire veya ASP.NET Core bağımlılıklarını bilmemelidir." Bu cümle kelimesi kelimesine Clean Architecture'in bağımlılık kuralıdır.

Ve bu kural yorum olarak değil **test olarak** korunuyor: **dependency-cruiser** (C.8), domain katmanına Prisma import edildiği an derlemeyi durduruyor.

Monorepo, monolit ve mikroservis — üç terim, üç ayrı soru

Bunlar sık karıştırılır ama **farklı sorulara** cevap verirler:

Terim	Hangi soruya cevap verir	Bu projedeki cevap
Monorepo / polyrepo	Kod nerede duruyor?	Monorepo — tek git deposu
Monolit / mikroservis	Program nasıl çalışıyor?	Modüler monolit — tek program (+ worker süreci)
Tek sunucu / çok sunucu	Nerede yayınlanıyor?	Konteynerler, kurumun sunucusunda

Bunlar bağımsız eksenlerdir. Mikroservisler **aynı monorepo içinde** de durabilir (Google böyle çalışır), ayrı depolarda da.

Karar nasıl verilir — mikroservis ne zaman haklı olur:

Koşul	Bu projede var mı
Modülün kendi ekibi ve kendi yayın takvimi var	✗ Tek geliştirici
Modül çok farklı ölçekleniyor (biri 100 kat yük alıyor)	✗ Yük dengeli
Modül farklı teknoloji gerektiriyor (ör. Python ile görüntü işleme)	✗ Hepsi TypeScript
Modüller arası tutarlılık transaction gerektirmiyor	✗ Gerektiriyor

Dördü de "hayır" olduğu için mikroservis bu projede **maliyeti olan, karşılığı olmayan** bir karar olurdu. Ödev §32 bunu doğrudan uyarıyor: "Gereksiz teknoloji, pattern, katman veya abstraction eklemek tek başına olumlu değerlendirilmez."

Sunumda söylenecek cümle: "Mikroservis düşünmedim değil — dört ölçüte baktım: ayrı ekip, ayrı ölçekleme ihtiyacı, farklı teknoloji ve transaction bağımsızlığı. Dördü de bu projede yok. Ama modül sınırlarını mikroservise ayrılacak şekilde çizdim; bildirim modülü gün geldiğinde ayrılabilir."

E.2 SOLID prensipleri (§8)

Ne isteniyor: Ödev SOLID'e uyulmasını **zorunlu** tutuyor ve "dokümantasyonda açıklanması değil, kod tabanında uygulanması" bekleniyor.

SOLID beş prensibin baş harfidir. Her birini **gerçek hayat** → **kod** → **bu projede nerede** sırasıyla açıyoruz.

S — Single Responsibility (tek sorumluluk)

Gerçek hayat: İsviçre çakısı ile mutfak bıçağı. Çakı her işi yapar ama hiçbirini iyi yapmaz; ekmek kesmeye kalkarsanız ekmek de çakı da zarar görür.

Kural: Bir sınıfın **değişmek için tek bir sebebi** olmalı.

Bu projede — yanlış ve doğru:

```
// 🚫 YANLIŞ: tek sınıf birbiriyle ilgisiz dört işi birden yapıyor
class WorkOrderService {
    create()      { /* kayıt + SLA hesabı + bildirim + e-posta hepsi burada */ }
    calculateSla(){ ... }    // SLA kuralı değişince bu dosya açılır
    sendEmail()  { ... }    // e-posta sağlayıcısı değişince de bu dosya açılır
    generatePdf() { ... }    // rapor biçimi değişince de bu dosya açılır
}

// Sonuç: dört farklı sebeple aynı dosyaya dokunuluyor, çakışma kaçınılmaz.
```

Bu sınıf **dört ayrı sebeple** değişir: iş kuralı değişirse, SLA kuralı değişirse, e-posta sağlayıcısı değişirse, rapor formatı değişirse. Dört ekip aynı dosyaya dokunur.

```
// ✅ DOĞRU: her sınıfın değişmek için TEK bir sebebi var
class WorkOrderService { /* sadece iş emri akışı – kural değişirse burası */ }
class SlaPolicyFactory { /* sadece SLA seçimi – süre kuralı değişirse burası */ }
class NotificationService { /* sadece bildirim – sağlayıcı değişirse burası */ }
```

Ödev §8 bunu doğrudan söylüyor: "*Tek bir servis, sistemdeki bütün işlemlerden sorumlu olmamalıdır.*"

O — Open/Closed (geliştirmeye açık, değiştirmeye kapalı)

Gerçek hayat: Priz çoklayıcı. Yeni bir cihaz eklemek için **duvardaki tesisata dokunmazsınız**, çoklayıcıya bir fiş daha takarsınız.

Kural: Yeni davranış **eklenerek** gelmeli, mevcut kod değiştirilerek değil.

Bu projede — SLA politikaları:

```
// 🚫 YANLIŞ: yeni bir kural gelince ÇALIŞAN fonksiyonun içini açmak gerekiyor
function hesapla(wo) {
  if (wo.priority === 'CRITICAL') return 4;    // kritik iş → 4 saat
  if (wo.priority === 'HIGH')      return 24;  // yüksek öncelik → 24 saat
  // Yeni kural buraya girecek. Her giriş, üstteki iki satırı bozma riski taşır
  // ve bu fonksiyonun tüm testleri yeniden koşturmak zorunda.
}

// ✅ DOĞRU: yeni kural = tamamen yeni bir sınıf. Mevcut hiçbir satır değişmiyor.
class WeekendMaintenancePolicy implements SlaPolicy { ... }
// Modülde kayıt listesine tek satır eklenir; çalışan kod olduğu gibi kalır.
```

Neden bu kadar önemli: Çalışan bir kodu her açtığınızda bozma riski alırsınız. Eklemek risksizdir, değiştirmek risklidir.

L — Liskov Substitution (yerine geçebilirlik)

Gerçek hayat: Priz sözü "220 volt" der. Bir cihaz o prize takılıp 380 volt beklerse **söz bozulmuştur** — cihaz yanar. Prize takılabiliyor olmak yetmez, **sözü tutmak** gerekir.

Kural: Bir arayüzü uygulayan her sınıf, sözü **bozmadan** yerine geçebilmelidir.

Bu projede:

```
// Arayüzün verdiği SÖZ: "calculate çağrılırsa MUTLAKA geçerli bir plan döner."
interface SlaPolicy {
  calculate(ctx: SlaContext): SlaPlan;
}

// 🚫 SÖZÜ BOZAN uygulama – derlenir ama çalışma anında patlar
class BrokenPolicy implements SlaPolicy {
  calculate(ctx) {
    // Kontrol işi türü ise hiçbir şey döndürmüyor. Oysa söz "plan döner"di.
    if (ctx.type === 'INSPECTION') return null;
    // ⚠️ Hata BURADA değil, bu kodu çağıran yerde görünecek: orası
    // plan.dueAt okumaya çalışıp null bulacak. Bulunması en zor hata türü.
  }
}
```

Bu sınıf derlenir ama çağıran kodu **çalışma anında** patlatır: factory'yi kullanan servis `plan.dueAt` okumaya çalışır ve `null` bulur. Üstelik hata, politikada değil **onu kullanan yerde** görünür — bulunması zor

bir hatadır.

Doğrusu: uymuyorsa `supports()` zaten `false` dönmeli; `calculate()` çağrılmışsa mutlaka plan üretmeli.

I — Interface Segregation (arayüz ayrımı)

⚠ Buradaki "arayüz" EKRAN DEĞİLDİR — kelime iki ayrı şeyi karşılıyor

Türkçede tek kelime, İngilizcede de tek kelime (*interface*) — ama iki apayrı kavram. Karıştırılınca bu bölüm hiç anlaşılmıyor:

Hangi "arayüz"	İngilizcesi	Nedir	Kim görür
Kullanıcı arayüzü	UI (User Interface)	Ekrandaki sayfa, buton, form	İnsan
Kod arayüzü	interface	Bir sınıfın söz verdiği metot listesi — "bende şu işlemler var"	Sadece kod

Bu bölümün tamamı ikinci anlamda. SOLID'in I harfi ekranla hiç ilgili değil; bir kod parçasının diğerine verdiği **sözü** düzenliyor.

Kod arayüzü nedir — gerçek hayat: Bir işe alım ilanı. "*Bu pozisyondaki kişi şunları yapabilmeli: A, B, C.*" İlan işi kimin yapacağını söylemez, **hangi işlerin yapılabileceğini** söyler. Kod arayüzü de aynen böyle: `WorkOrderReader` demek "*bunu uygulayan her şey `findMany` metodunu sunacaktır*" demek. Prisma mı sunuyor, testteki sahte bir nesne mi sunuyor — çağırın taraf bilmez ve umursamaz.

Gerçek hayat: Otel odasındaki 30 düğmeli kumanda. Siz sadece ışığı açmak istiyorsunuz ama önünüzde perde, klima, müzik ve "temizlik istemiyorum" düğmeleri var. **Kullanmadığınız şeyler yolunuza çıkıyor.**

Kural: Bir arayüz, uygulayanına **kullanmayacağı şeyleri dayatmamalı.**

Bu projede:

```
// 🚫 YANLIŞ: sekiz işi birden dayatan dev arayüz.
// Sadece okuma yapan bir rapor servisi bunu uygulamak zorunda kalırsa,
// kullanmayacağı yedi metodu boş bırakmak veya hata fırlatmak durumunda kalır.
interface IWorkOrderRepository {
  find(); create(); update(); delete();
  archive(); export(); recalculateSla(); sendReminder();
}
```

Sadece okuma yapan bir rapor servisi bu arayüzü uygulamak zorunda kalırsa, kullanmayacağı yedi metodu boş bırakmak (veya hata fırlatmak) durumunda kalır.

```
// ✅ DOĞRU: okuma ve yazma ayrı sözleşmeler.  
// Rapor servisi yalnızca Reader ister; testte sahtesini yazmak tek metot demek.  
interface WorkOrderReader { findMany(f: Filter): Promise<WorkOrder[]>; }  
interface WorkOrderWriter { save(wo: WorkOrder): Promise<void>; }
```

Rapor servisi yalnızca **WorkOrderReader** ister — testte sahtesini yazmak da tek metot yazmak demektir.

D — Dependency Inversion (bağımlılığın tersine çevrilmesi)

Bu, en çok karıştırılan harftir. **Somut örnekle açalım.**

Gerçek hayat: Bir ev inşa ediyorsunuz. İki seçenek var:

- **Bağımlı yol:** Duvara "*yalnızca X marka klima takılabilir*" şeklinde özel bir yuva açıyorsunuz. X marka üretimi durdurduğunda **duvarı kırmanız** gerekiyor.
- **Tersine çevrilmiş yol:** Duvara **standart priz** koyuyorsunuz. Hangi marka klima olursa olsun takılır; marka değişince duvara dokunmuyorsunuz.

Burada **duvar (üst katman) markayı değil, prizi (sözleşmeyi) tanıyor**. İşte "tersine çevirme" budur: üst katman alt katmanın **somut hâline** değil, **sözleşmesine** bağlanır.

Yazılımda — bu projedeki gerçek durum:

Domain katmanında bir kural var: "*Aynı iş emri için aynı gün ikinci bir SLA hatırlatma bildirimi üretilemez.*" Bu kuralı uygulamak için mevcut bildirimlere **bakması** gerekiyor — yani veriye ihtiyacı var.

```
// 🚫 YANLIŞ — iş kuralı sınıfı doğrudan veritabanı aracını tanıyor  
import { PrismaClient } from '@prisma/client'; // ← altyapı, domain'e sızdı  
  
class NotificationRule {  
  // ⚠ Bu sınıf artık Prisma olmadan çalışamaz — testi için bile veritabanı kurmak gerekir  
  constructor(private prisma: PrismaClient) {}  
  
  async canNotify(workOrderId: number) {  
    // Veritabanına gidip "bu bildirim daha önce yazılmış mı" diye bakıyor  
    const existing = await this.prisma.notification.findFirst({ ... });  
    return !existing; // yazılmamışsa bildirim üretilabilir  
  }  
}
```

Bunun üç somut zararı var:

1. Bu kuralı test etmek için **veritabanı kurmak** gerekir — test saniyeler sürer
2. Prisma'dan başka bir araca geçilse **iş kuralı da değişir**
3. **dependency-cruiser** (C.8) bu import'u görür ve **build'i kırmızı yakar**

```
// ✅ DOĞRU – iş kuralı, veritabanını değil yalnızca bir SÖZÜ tanıyor

// 1) SÖZLEŞME domain katmanında yazılıyor: "bana şunu sorabilen bir şey lazım."
//    Nasıl cevaplanacağı burada yazmıyor – sadece sorunun kendisi tanımlı.
interface NotificationChecker {
  wasNotifiedToday(workOrderId: number, kind: NotificationKind): Promise<boolean>;
}

// 2) İş kuralı yalnızca bu söze bağlanıyor.
//    Prisma'yı, SQL'i, veritabanının varlığını bile bilmiyor.
class NotificationRule {
  constructor(private checker: NotificationChecker) {} // dışarıdan verilecek

  async canNotify(workOrderId: number) {
    // "Bugün bu bildirim gitti mi?" diye soruyor. Cevabı kimin verdiği önemsiz.
    return !(await this.checker.wasNotifiedToday(workOrderId, 'SLA_REMINDER'));
  }
}

// 3) SÖZÜ TUTAN sınıf altyapı katmanında. Prisma ancak burada görünüyor.
//    Testte bunun yerine sahte bir sınıf verilir, iş kuralı farkı anlamaz.
class PrismaNotificationChecker implements NotificationChecker {
  constructor(private prisma: PrismaClient) {}

  wasNotifiedToday(workOrderId, kind) {
    // Sorunun veritabanı karşılığı: böyle bir kayıt var mı?
    return this.prisma.notification.findFirst({ ... }).then(Boolean);
  }
}
```

★ **"Tersine çevrilen" tam olarak ne:** Normalde bağımlılık oku **üstten alta** akar (kural → veritabanı).

Burada **arayüzün tanımlandığı yer** domain katmanıdır; infrastructure onu *uygulamak zorundadır*. Yani ok tersine döner: **altyapı, domain'e bağımlı hâle gelir**.

Bu projede nerede karşılaşılabacak:

Domain'in ihtiyacı	Sözleşme	Uygulayan
Şu an saat kaç	<code>Clock</code>	Gerçekte sistem saati, testte sabit saat
İşlemi yapan kim	<code>CurrentUser</code>	<code>nestjs-cls</code> (C.16)
Bu bildirim bugün gitti mi	<code>NotificationChecker</code>	Prisma
İş emri numarası üret	<code>WorkOrderNumberGenerator</code>	Veritabanı sayacı

Testte kazanç somut: `Clock` sözleşmesine testte "sen şu an 30 gün sonrasındasın" dersiniz ve SLA ihlali kurallarını **gerçek zamanı beklemeden** doğrularsınız. `new Date()` doğrudan çağrılseydi bu test yazılamazdı.

Ödevin özellikle uyardığı dört hata

Ödev §8'de dört somut hata sayıyor; hepsi bu projede engelleniyor:

Ödevin uyarısı	Bu projedeki karşılığı
"Controller'da iş kuralı bulunmamalı"	Kurallar domain'de; controller yalnızca isteği alıp cevabı döner (E.0 → metot)
"Tek servis her işten sorumlu olmamalı"	S harfi — modül başına ayrı servis
"Geniş arayüzlerde toplanmamalı"	I harfi (E.2 → I) — okuma ve yazma ayrı sözleşmeler : <code>WorkOrderReader</code> / <code>WorkOrderWriter</code> . Buradaki "arayüz" ekran değil, kod sözleşmesi
"Yeni kural mevcut kodu büyük ölçüde değiştirmemeli"	O harfi — Factory deseni (E.4)

E.3 DRY prensibi (§9)

Ne demek: *Don't Repeat Yourself* — "kendini tekrar etme". Aynı bilgi veya kural sistemde **tek bir yerde** yaşamalı.

Gerçek hayat: Bir kurumda telefon rehberi hem muhasebede hem insan kaynaklarında ayrı ayrı tutuluyor. Biri güncelleniyor, diğeri güncellenmiyor. Bir süre sonra **hangisinin doğru olduğu bilinmiyor** — ve genellikle insanlar önce baktıklarına inanıyor.

Asıl zarar "fazla iş" değil; **iki farklı doğrunun oluşması**.

Ödevin saydığı yedi tekrar ve bu projedeki karşılığı

Ödev 9'da yedi somut tekrar türü sayıyor. Her birini bu projedeki gerçek örneğiyle açıyoruz.

1. Aynı doğrulamanın farklı uçlarda tekrar yazılması

Somut örnek: İş emri başlığı **en az 5, en fazla 200 karakter** olmalı.

Bu kural en az dört yerde gerekir: yeni iş emri formu (tarayıcı), iş emri düzenleme formu (tarayıcı), oluşturma ucu (sunucu), güncelleme ucu (sunucu).

```
// 🚫 YANLIŞ: aynı kural dört ayrı dosyada elle yazılmış

// Tarayıcı – yeni iş emri formu
if (title.length < 5) setError('Başlık çok kısa');

// Tarayıcı – düzenleme formu. Kural aynı ama HATA METNİ farklı yazılmış
if (title.length < 5) setError('Başlık en az 5 karakter olmalı');

// Sunucu – oluşturma ucu
if (dto.title.length < 5) throw new BadRequest();

// Sunucu – güncelleme ucu. Burada sınır 3 yazılmış!
// Yani kullanıcı düzenleme ekranından 4 karakterlik başlık kaydedebiliyor,
// oluşturma ekranından kaydedemiyor. Aynı sistem iki farklı davranıyor.
if (dto.title.length < 3) throw new BadRequest();
```

Son satır tam da olan şeydir: kural bir yerde güncellenir, diğerleri geride kalır. Sonuç: kullanıcı düzenleme ekranından 4 karakterlik başlık kaydedebilir, oluşturma ekranından kaydedemez. **Aynı sistem iki farklı davranır.**

```
// ✔ DOĞRU: kural tek dosyada tanımlı, üç yer birden bunu kullanıyor
// Dosya: packages/contracts/work-order.ts

export const CreateWorkOrder = z.object({
  // Başlık: metin olmalı, en az 5 en fazla 200 karakter.
  // Hata mesajı da burada – tarayıcı ve sunucu AYNI metni gösteriyor.
  title: z.string().min(5, 'Başlık en az 5 karakter olmalı').max(200),

  // Öncelik: yalnızca bu dört değerden biri olabilir, başkası reddedilir
  priority: z.enum(['LOW', 'NORMAL', 'HIGH', 'CRITICAL']),

  // Varlık numarası: tam sayı ve pozitif olmalı (0 veya eksi kabul edilmez)
  assetId: z.number().int().positive(),
});
```

Bu tek tanım **üç yeri birden** besliyor:

Kullanan	Nasıl
Tarayıcıdaki form	React Hook Form + @hookform/resolvers (C.20)
Sunucudaki uç	NestJS doğrulama katmanı
API dokümanı	Swagger, aynı şemadan üretiliyor (C.18)

★ Kural değiştiğinde **tek satır** değişiyor ve üçü birden güncelleniyor. Hata mesajı bile aynı.

2. Durum geçiş kontrollerinin farklı servislerde tekrarı

"Kapalı iş emri güncellenemez" kuralı; güncelleme ucunda, atama ucunda, yorum ekleme ucunda ve arka plan işinde ayrı ayrı yazılırsa biri mutlaka unutulur.

Çözüm: Tek geçiş tablosu (E.5). Her değişiklik oradan geçer.

3. Aynı mapping'in farklı sınıflarda bulunması

İş emri listesinin hangi alanları döndüreceği iki ayrı yerde yazılırsa, birine yeni alan eklenip diğerine eklenmediğinde ekranlar farklı veri gösterir.

Çözüm: Response şekli tek Zod şemasından türetiliyor (E.6).

4. Aynı hata yapısının uçlarda tekrar oluşturulması

Her uç kendi hata biçimini üretirse, arayüz her ucun hatasını ayrı ele almak zorunda kalır.

Çözüm: Tek merkezî hata filtresi (E.7).

5. Audit alanlarının her işlemde ayrı doldurulması

`createdBy` , `updatedBy` alanlarını her serviste elle yazmak — biri unutulunca o kaydın izi kaybolur.

Çözüm: Prisma eklentisi merkezî doldurur (C.16).

6. Filtreleme ve sayfalama kodunun kopyalanması

İş emri, varlık ve lokasyon listelerinin üçünde de sayfalama var. Kod kopyalanırsa, azami sayfa boyutu sınırı birinde unutulur ve orası açık kalır.

Çözüm: Ortak sayfalama yardımcısı (E.10).

7. Bildirim kurallarının farklı işlerde tekrarı

Dört arka plan işinin dördü de bildirim üretiyor. Mükerrer engelleme mantığı dördüne ayrı yazılırsa, birinde eksik kalır ve kullanıcı çift bildirim alır.

Çözüm: Tek bildirim üretici; işler onu çağırır (C.6).

⚠ DRY'ın tehlikeli tarafı — her tekrar ortaklaştırılmaz

Ödev şunu ekliyor: "*DRY prensibi uygulanırken gereksiz ve anlamsız abstraction oluşturulmamalıdır.*"

Ayırt edici soru: İki kod **aynı sebeple mi** değişecek?

Durum	Ortaklaştırılır mı
Başlık kuralı hem formda hem sunucuda — aynı kural	✅ Evet
İş emri başlığı <code>min(5)</code> , lokasyon adı da <code>min(5)</code> — tesadüfen aynı sayı	❌ Hayır

İkincisini ortaklaştırırsanız, yarın lokasyon adı sınırı 3'e indiğinde iş emri başlığı da inmek zorunda kalır — ya da ortak fonksiyona parametre eklersiniz ve ortaklaştırmanın anlamı kalmaz.

Kural: Benzer **görünen** değil, aynı **sebeple değişen** kod ortaklaştırılır. Aksi hâlde olmayan bir bağımlılık icat edilmiş olur.

E.4 Factory Pattern ve SLA hesabı (§7)

Ne isteniyor: Ödev Factory Pattern kullanımını **zorunlu** tutuyor, SLA politikasının belirlenmesinde kullanılmasını istiyor ve "*göstermelik bir sınıf olmamalı*", "*Service Locator olarak kullanılmamalı*", "*Open/Closed ile uyumlu olmalı*" diye özellikle uyarıyor.

★ KARARLAŞTIRILAN SLA POLİTİKASI (2026-08-26, onaylandı)

Ödev 7: "SLA sürelerini ve hesaplama kurallarını tarafınızdan belirlemeniz beklenmektedir. Belirlediğiniz kurallar dokümente edilmelidir." Bu bölüm o dokümantasyondur.

Önce tanım: SLA saati neyi ölçüyor

🚫 **Geliştiricinin çalıştığı süre değil.** Biletin açılışından sahibine iade edilmesine kadar geçen **toplam** süre:

```
Personel arıza bildirir      ← SLA saati BAŞLAR
  ↳ Destek sınıflandırır, geliştiriciye yönlendirir
    ↳ Geliştirici çözer
      ↳ Kontrol edilir
        ↳ Bilet sahibine iade edilir  ← SLA saati DURUR
```

Yani sınıflandırma, bekleme ve kontrol de bu sürenin içindedir.

1) Öncelikten temel süre

Öncelik	Çözüm süresi	Ne demek
Kritik	3 saat	Hizmet tamamen durmuş, alternatifi yok
Yüksek	8 saat	İş aksıyor ama geçici çözüm var
Normal	24 saat	Rahatsız edici, engelleyici değil
Düşük	72 saat	Planlanabilir, aciliyeti yok

Oran **1 : 2.7 : 8 : 24** — ITIL'deki yaygın P1–P4 merdiveniyle aynı biçimde artıyor.

2) Varlığın kritikliği çarpan uygular

Varlık kritikliği	Çarpan	Örnek
Kritik	×0.5	Su pompası, jeneratör, sunucu odası kliması
Yüksek	×0.75	Hizmet binası asansörü
Normal	×1	Ofis bilgisayarları
Düşük	×1.5	Depodaki yedek ekipman

★ Böylece "kritik öncelikli ama önemsiz varlık" ile "kritik öncelikli ve hayati varlık" aynı süreyi almıyor. Ödev 7 varlığın kritiklik seviyesini hesaba katmayı **şart koşuyor**.

3) Hatırlatma ve escalation — sabit saat değil, YÜZDE

An	Ne zaman	Ne oluyor
İlk hatırlatma	Sürenin %50'si	Atanan teknik personele bildirim
Escalation	Sürenin %80'i	Amirine + operasyon sorumlusuna bildirim
İhlal	%100	İş emri "SLA aşıldı" işaretlenir, yönetim panosuna düşer

Hesaplanan değerler (varlık çarpanı ×1 iken):

Öncelik	Çözüm	İlk hatırlatma	Escalation
Kritik	3 sa	1 sa 30 dk	2 sa 24 dk
Yüksek	8 sa	4 sa	6 sa 24 dk
Normal	24 sa	12 sa	19 sa 12 dk
Düşük	72 sa	36 sa	57 sa 36 dk

★ **Yüzde kullanmanın sebebi mimari, estetik değil.** Yeni bir öncelik eklendiğinde yalnızca **süresi** yazılır; hatırlatma ve escalation kendiliğinden doğru hesaplanır. Ödev §7'nin "*yeni SLA politikası eklenince mevcut kod mümkün olduğunca az değişmeli*" şartı — yani **Open/Closed** — tam olarak budur.

4) ★ İş emri türü — Factory'nin GERÇEKTEN farklı sınıf üretmesi

Buradaki tasarım kararı ödevin en çok denetlenecek yeri:

❌ **Planlı bakım ve periyodik kontrol için "X saat içinde bitir" mantığı YANLIŞTIR.** Onların bir **planlanan tarihi** vardır; son tarih o tarihtir, "şu andan itibaren 24 saat" değil.

Tür	SLA nasıl hesaplanıyor	Politika sınıfı
Arıza	$\text{şimdi} + (\text{öncelik süresi} \times \text{varlık çarpanı})$	ArizaSlaPolitikasi
Planlı bakım	Planlanan tarih son tarihtir ; hatırlatma 1 gün önce	PlanliBakimSlaPolitikasi
Periyodik kontrol	Planlanan tarih ± tolerans penceresi (varsayılan 3 gün)	PeriyodikKontrolSlaPolitikasi

★ **Ödev §7 diyor ki:** "*Factory yalnızca göstermelik bir sınıf olmamalıdır.*" Üç politika **gerçekten farklı hesap** yapıyor — biri süre ekliyor, biri sabit tarihe bakıyor, biri pencere kontrol ediyor. Sahte çeşitlilik değil.

5) ❌ Takvim — KARMA model (onaylanan karar)

Problem: Saat 17:00'de açılan **Yüksek** öncelikli (8 saat) bir bilet, 7/24 sayımda gece 01:00'de ihlal olur — kimse çalışmıyorken. Her akşam açılan bilet otomatik ihlal ederdi; ölçüm anlamsızlaşır ve personele haksızlık olur.

Karar: Takvim, önceliğin bir **özellik**idir — genel bir ayar değil.

Öncelik	Takvim	Gerekçe
Kritik	7/24 kesintisiz	Nöbet vardır; su pompası gece de patlar
Yüksek · Normal · Düşük	Mesai saati (hafta içi 08:00–17:00, resmî tatiller hariç)	Bu işler için gece müdahale yok

Örnek — aynı bilet, iki öncelik:

Bilet Cuma 16:00'da açıldı

KRİTİK (3 sa, 7/24) → son tarih Cuma 19:00
(hatırlatma 17:30 · escalation 18:24)

YÜKSEK (8 sa, mesai) → Cuma'da 1 saat işledi, kalan 7 saat
Pazartesi 08:00'den sayılır
→ son tarih Pazartesi 15:00

★ **Bu karar mimariyi güçlendiriyor:** takvim politikanın parçası olduğu için Factory gerçekten farklı davranan sınıflar üretiyor. Ödevin istediği şey bu.

⚠ **Bedeli dürüstçe:** Mesai takvimi hesaplayıcısı + resmî tatil listesi gerekiyor (~150 satır + testler). Tatil listesi yıllık güncellenir ve `docs/project/altyapi-durumu.md` 'ye not düşülür.

🚫 **Saat dilimi tek yerde:** Tüm hesaplar `Europe/Istanbul` üzerinden yapılır, veritabanında `timestamptz` olarak saklanır. Sistem saati doğrudan okunmaz — `Clock` servisi üzerinden alınır (ödev §8: "*sistem saati abstraction üzerinden kullanılmalıdır*"), böylece testte sahte saat verilebilir.

Bu politikanın PRD'deki yeri

Bu tablolar `docs/project/PRD.md` → "*SLA kuralları*" bölümüne kopyalanır ve `docs/sla-rules.md` olarak teslim paketine girer — ödev §7 dokümanite edilmesini **zorunlu** tutuyor.

Problem

SLA (Service Level Agreement), işin tamamlanması gereken süredir. Tek bir sayı değil; üç girdiye göre değişir:

- İş emrinin **önceliği**: düşük · normal · yüksek · kritik
- Varlığın **kritiklik seviyesi**
- İş emrinin **türü**: arıza · bakım · kontrol · kurulum

Her kombinasyon için üç ayrı zaman hesaplanmalı: **bitiş**, **ilk hatırlatma** ve **escalation** (yükseltme — süre aşılmadan önce üst amire haber verme).

Naif çözüm ve neden sürdürülemez

```
// 🚫 Bu yaklaşım kullanılmadı – neden kullanılmadığı aşağıda
function hesapla(wo: WorkOrder) {
  // Kritik öncelikli iş + kritik varlık → 4 saat
  if (wo.priority === 'CRITICAL' && wo.asset.criticality === 'HIGH') return 4;
  // Kritik öncelikli ama varlık kritik değil → 8 saat
  if (wo.priority === 'CRITICAL') return 8;
  // Yüksek öncelikli arıza → 24 saat
  if (wo.priority === 'HIGH' && wo.type === 'BREAKDOWN') return 24;

  // ⚠ Her yeni kural bu bloğun İÇİNE yazılmak zorunda; yani çalışan kodu
  //   her seferinde açıp değiştiriyorsun ve mevcut kuralları bozma riski
  //   alıyorsun.
}
```

İlk gün çalışır. Altıncı ayda sorun çıkar: her yeni kural bu bloğu **değiştirmeyi** gerektirir ve her değişiklik çalışan kuralları bozma riskidir. Ödev Ş7 bunu doğrudan işaret ediyor: *"Yeni bir SLA politikası eklendiğinde mevcut kodda mümkün olduğunca az değişiklik yapılmalıdır."*

Uygulanan tasarım

Her kural kendi sınıfında, ortak bir **arayüz** (E.0) altında:

```
// SÖZLEŞME: SLA hesaplayan her sınıf şu iki soruyu cevaplayabilmeli
interface SlaPolicy {
  supports(ctx: SlaContext): boolean;      // "bu iş emri bana uyuyor mu?"
  calculate(ctx: SlaContext): SlaPlan;     // bitiş + hatırlatma + yükseltme zamanı
}

// TEK BİR KURAL, kendi sınıfında. Başka kuralı bilmiyor, etkilemiyor.
@Injectable()
export class CriticalAssetBreakdownPolicy implements SlaPolicy {

  // Bu kural yalnızca "kritik varlıkta arıza" durumunda devreye giriyor
  supports(ctx: SlaContext) {
    return ctx.assetCriticality === 'HIGH' && ctx.type === 'BREAKDOWN';
  }

  // Uyduysa süreleri hesaplıyor
  calculate(ctx: SlaContext): SlaPlan {
    // Şu andan itibaren 4 saat. Saati Clock servisinden alıyoruz ki
    // testte "sen şu an şu andasın" diyebilelim.
    const due = this.clock.now().plus({ hours: 4 });

    return {
      dueAt:      due,                                // bitmesi gereken an
      remindAt:   due.minus({ hours: 1 }),             // 1 saat kala hatırlat
      escalateAt: due.minus({ minutes: 30 }),          // 30 dakika kala üst amire bildir
    };
  }
}
```

Factory, politikaları **enjeksiyonla** alır ve ilk uyanı seçer:


```

@Inject()
export class SlaPolicyFactory {
  // ★ Politikaların listesi DIŞARIDAN veriliyor – factory hiçbir yerden
  // kendisi arama yapmıyor. Bu ayrım, ödevin yasakladığı "Service Locator"
  // kullanımından kaçınmanın tam karşılığı.
  constructor(@Inject(SLA_POLICIES) private readonly policies: SlaPolicy[]) {}

  resolve(ctx: SlaContext): SlaPolicy {
    // Listeyi sırayla gez, "bu iş emri sana uyar mı?" diye sor, ilk uyanı al
    const match = this.policies.find(p => p.supports(ctx));

    // Hiçbiri uymadıysa sessizce varsayılan bir süre UYDURMUYORUZ.
    // Hata fırlatıyoruz ki eksik kural fark edilsin.
    if (!match) throw new NoSlaPolicyError(ctx);

    return match;
  }
}

```

Kayıt, modülde tek satır:

```

// Politikaların kaydedildiği tek yer. Yeni kural eklemek = bu listeye bir
// sınıf adı yazmak. Factory'ye ve diğer politikalara DOKUNULMUYOR.
// ★ Open/Closed prensibinin somut karşılığı burası.
// ⚠ Sıra önemli: özel kurallar üstte, genel kural altta olmalı. Genel kural
// öne geçerse özel olanlara hiç sıra gelmez ve hata sessiz kalır.
{ provide: SLA_POLICIES, useFactory: (...p: SlaPolicy[]) => p,
  inject: [CriticalAssetBreakdownPolicy, HighPriorityPolicy, DefaultPolicy] }

```

Yeni kural eklemek: yeni sınıf yaz, **inject** listesine ekle. Factory'ye, mevcut politikalara ve çağıran koda **dokunulmaz**. SOLID'in O harfi (E.2) budur.

Ödevin üç uyarısına verilen cevap

"Service Locator olmamalı." Service Locator, sınıfın ihtiyacını konteynerden **kendisinin aramasıdır** (`container.get('policy')`). Bağımlılıklar imzada görünmez, test edilmesi zorlaşır. Burada factory hiçbir şey aramaz; politika dizisi **dışarıdan verilir** ve constructor imzasında görünür.

"Göstermelik olmamalı." Factory gerçek bir seçim yapıyor ve sistemin başka hiçbir yerinde SLA seçme koşulu bulunmuyor — ödevin ifadesiyle "SLA seçme koşulları sistemin farklı bölümlerine dağılmamalıdır."

"Testlerle doğrulanmalı." Politikalar NestJS olmadan doğrudan `new` edilerek test edilebiliyor; factory testi ise "hangi bağlamda hangi politika seçiliyor" sorusunu doğruluyor. Sıra önemli olduğu için politika sırası da ayrı bir testle sabitlendi — aksi hâlde daha genel bir politika özel olanın önüne geçerse hata sessiz kalırdı.

E.5 Durum makinesi (§6)

Ne isteniyor: Durumların kontrolsüz değiştirilememesi, geçersiz geçişlerin **backend tarafında** engellenmesi, her değişiklikte geçmiş kaydı ve *"controller içerisinde dağınık koşul bloklarıyla yönetilmemesi."*

Geçiş tablosu

Durum makinesi, izin verilen geçişlerin tek bir yerde tanımlanmasıdır:

```
// Bu tablo şunu söylüyor: "soldaki durumdayken YALNIZCA sağdakilere geçilebilir."
// Tabloda olmayan her geçiş otomatik olarak reddediliyor.
const TRANSITIONS: Record<Status, Status[]> = {

  // Yeni açılmış iş emri: ya birine atanır ya iptal edilir
  OPEN:      ['ASSIGNED', 'CANCELLED'],

  // Atanmış iş: çalışmaya başlanır, geri alınır (OPEN) veya iptal edilir
  ASSIGNED:  ['IN_PROGRESS', 'OPEN', 'CANCELLED'],

  // Üzerinde çalışılıyor: parça beklemeye alınır, çözülür veya iptal edilir
  IN_PROGRESS: ['WAITING_PART', 'RESOLVED', 'CANCELLED'],

  // Parça bekliyor: parça gelince işe devam, gelmezse iptal
  WAITING_PART: ['IN_PROGRESS', 'CANCELLED'],

  // Çözüldü: kapatılır — ya da sorun tekrarlarsa yeniden işleme alınır
  RESOLVED:  ['CLOSED', 'IN_PROGRESS'],

  // Kapatıldı: SON DURAK. Buradan çıkış yok, liste boş.
  CLOSED:    [],

  // İptal edildi: SON DURAK. Buradan da çıkış yok.
  CANCELLED: [],
};
```

`CLOSED` ve `CANCELLED` **terminal** durumlarıdır — çıkışı olmayan düğümler. Ödevin *"kapatılmış iş emri normal güncelleme işlemleriyle değiştirilememelidir"* ve *"iptal edilmiş iş emri üzerinde işlem yapılamamalıdır"* maddeleri tablodan doğrudan okunur.

Tabloya ek olarak duran koşullu kurallar

Bazı kurallar sadece "hangi durumdan hangisine" sorusuyla ifade edilemez; geçişe **koşul** eklenir:

Geçiş	Ek koşul
OPEN → ASSIGNED	Atanan kullanıcı aktif ve rolü teknik personel olmalı
ASSIGNED → IN_PROGRESS	İş emri atanmış olmalı (atanmamış iş işleme alınamaz)
IN_PROGRESS → RESOLVED	Çözüm açıklaması zorunlu
herhangi → CANCELLED	İptal gerekçesi zorunlu

Neden dağınık **if** yerine tablo

Kontroller uç noktalara dağıtıldığında, yeni bir durum eklendiğinde hepsinin tek tek bulunması gerekir. Biri atlanır ve **sessizce açık kalır** — üstelik bu açık ancak o yoldan geçildiğinde fark edilir.

Tabloda toplandığında: yeni durum eklemek tabloya satır eklemektir, kontrol kodu değişmez. Ayrıca tablo okunabilir olduğu için iş birimine gösterilip doğrulatabilir.

Geçiş kontrolü domain katmanında yaşar; Prisma'yı, HTTP'yi ve NestJS'i bilmez (E.1). Bu yüzden testleri veritabanı gerektirmez.

Geçiş ile geçmiş kaydının atomikliği

Ödev §6 şunu şart koşuyor: "*Durum değişikliği ile geçmiş kaydı aynı işlem bütünlüğü içerisinde gerçekleştirilmelidir.*"

```
// $transaction = "ya ikisi birden olur ya hiçbiri".
// Arada elektrik kesilse bile yarım kayıt kalmaz.
await prisma.$transaction(async (tx) => {

  // 1) İş emrinin durumunu değiştir.
  //   "where" içindeki version, başkası bu arada değiştirdiyse yakalamak için.
  const updated = await tx.workOrder.update({
    where: { id, version },
    data: { status: next, version: { increment: 1 } }, // sürümü bir artır
  });

  // 2) Aynı işlemde geçmiş satırını da yaz: kim, ne zaman, nereden nereye.
  //   Bu olmadan durum değişir ama "kim değiştirdi" kaydı olmaz.
  await tx.workOrderHistory.create({
    data: { workOrderId: id, from: current, to: next, byUserId, at: clock.now() },
  });
});
```

Geçiş yazılıp geçmiş yazılmazsa sistem doğru görünür ama **izi olmaz** — denetim açısından kaydın hiç olmamasından kötüdür.

Geçersiz geçişin sonucu

Geçersiz bir geçiş denendiğinde domain katmanı tipli bir hata fırlatır; merkezî filtre bunu **409 Conflict** cevabına çevirir (E.7). Uç noktada `try/catch` bulunmaz.

E.6 Mapping kararı: neden bir mapping kütüphanesi yok (§13)

⚠ "Mapping" bu belgede **İKİ AYRI ŞEYİ** karşılıyor — karıştırma

Hangi mapping	Ne yapar	Nerede
Nesne dönüşümü (bu bölüm)	Veritabanı kaydını dışarı dönen cevaba çevirir; hangi alan gider, hangisi gizlenir	E.6
Ad eşlemesi <code>@map</code>	Aynı alanın kodda ve veritabanında farklı adla durmasını sağlar (<code>dueAt</code> ↔ <code>due_at</code>)	E.9

🚫 **Reddedtiğimiz şey birincisi.** AutoMapper gibi bir **kütüphane** kullanmıyoruz; dönüşümü Zod şeması + Prisma `select` yapıyor.

★ `@map` **reddedilen şey değil** — kütüphane değil, Prisma şemasında tek kelimelik bir nitelik. Nesne dönüştürmez, yalnızca **ada bakar**.

Ne isteniyor: Ödev AutoMapper veya Mapster kullanımını zorunlu tutuyor. Stack'i serbest bırakılan bu projede **karşılığı kullanılmayan tek maddedir**, bu yüzden ayrıca gerekçelendiriliyor.

Mapping neyi çözer

Veritabanı kaydı (entity) ile dışarı dönen cevap (DTO) aynı şey değildir. Entity'de `passwordHash`, `deletedAt`, iç notlar gibi alanlar bulunur; hiçbirisi dışarı çıkmamalıdır. Mapping, entity'yi DTO'ya dönüştürme işidir. C# tarafında bu dönüşüm elle yazıldığında çok kod tutar; AutoMapper bunu refleksiyonla otomatikleştirir.

Ölçüm: iki aday da elendi

Aday	Haftalık indirme	Son yayın	Elenme sebebi
<code>class-transformer</code>	~12M	2022-12 , hâlâ 0.x	Üç yılı aşkın süredir yeni sürüm yok
<code>@autmapper/core</code>	~106K	Güncel	Niş — devralacak geliştirici tanımayabilir

İkisi de **dekoratörlü sınıf** gerektiriyor. Prisma düz nesne (POJO) döndürdüğü için, yalnızca mapper'ı beslemek amacıyla her model bir de sınıf olarak yazılmalıydı. Ödev §12 tam bundan sakınmayı istiyor: *"Pattern kullanmış görünmek amacıyla eklenen yapılar olumlu değerlendirilmeyecektir."*

Yerine kurulan mekanizma

Şema **tek kaynak**, `select` ondan türetiliyor, çıkış şemadan geçiyor:

```
// Dosya: packages/contracts – liste ekranının cevabı BURADA tanımlı.
// Bu şema hem sunucuyu hem tarayıcıyı besliyor, tek kaynak.
export const WorkOrderListItem = z.object({
  id: z.number(), // kaydın numarası
  number: z.string(), // "IE-2026-000148"
  title: z.string(), // iş emri başlığı
  status: z.enum(STATUSES), // yalnızca tanımlı durumlardan biri
  priority: z.enum(PRIORITIES), // yalnızca tanımlı önceliklerden biri
  assigneeName: z.string().nullable(), // atanmamışsa boş olabilir
  slaDueAt: z.date(), // SLA bitiş zamanı
  // 🚫 Burada olmayan hiçbir alan dışarı çıkamaz – şifre özeti, iç not, audit
});

// Yukarıdaki şemadan TypeScript tipi otomatik üretiliyor.
// Yani tipi ayrıca elle yazmıyoruz; şema değişince tip de değişiyor.
export type WorkOrderListItem = z.infer<typeof WorkOrderListItem>;
```

```
// Veritabanı sorgusu. select listesi yukarıdaki şemadan üretiliyor,  
// yani veritabanı 25 kolonun tamamını değil yalnızca 7 tanesini çekiyor.  
const rows = await prisma.workOrder.findMany({  
  where, // filtreler (durum, lokasyon, tarih...)  
  skip, take, // sayfalama: kaçınıcı sayfa, kaç kayıt  
  select: selectFrom(WorkOrderListItem), // → { id: true, number: true, ... }  
});  
  
// ★ SON KONTROL: veri şemadan geçiriliyor. Şemada olmayan bir alan  
// yanlışlıkla sorguya girmiş olsa bile burada kırılıyor.  
// Koruma bir alışkanlık değil, mekanizma.  
return z.array(WorkOrderListItem).parse(rows);
```

Üçüncü adım kritiktir: `passwordHash` yanlışlıkla `select` 'e girse bile cevaba **çıkamaz**. Koruma konvansiyon değil, mekanizmadır.

Ödevin dört şartının karşılanması

Ödevin şartı	Karşılığı
Entity doğrudan response dönülmemeli	Cevap her zaman şemadan geçer
Mapping içinde iş kuralı çalıştırılmamalı	Şema yalnızca şekil doğrular
Hassas bilgi response'a taşınmamalı	Şemada olmayan alan kırılır
Listelerde bütün tablo belleğe alınmamalı	<code>select</code> ile DB seviyesinde projeksiyon
Mapping konfigürasyonu test edilmeli	Şema round-trip testleri

Savunma cümlesi

"AutoMapper'ın üç işi — projeksiyon, dönüşüm ve hassas alan gizleme — karşılanıyor. Fark şu: hatalar çalışma anında değil derleme zamanında yakalanıyor. AutoMapper C#'ta gereklidir çünkü `elle class`→`class dönüşüm` çok kod tutar; Prisma'da `select` zaten projeksiyonun kendisidir. Ayrıca ölçtüm: aday kütüphanelerden biri üç yıldır bakımsız, diğeri niş."

E.7 Merkezî hata yönetimi (§19)

Ne isteniyor: Merkezî hata yönetimi, uçlarda tekrar eden `try/catch` bulunmaması, standart hata biçimi ve hata türlerinin ayrıştırılması.

Problem

Gerçek hayat: Bir hastanede her poliklinik kendi hasta yönlendirme kâğıdını kendi tasarlıyor. Birinde "kat 3", diğesinde "3. kat", üçüncüsünde "üst kat" yazıyor. Hasta her kapıda yeniden anlamaya çalışıyor. Danışma tek bir standart koyduğunda sorun bitiyor.

Yazılımda: Her uçta `try/catch` yazılırsa üç sorun çıkar:

1. Aynı kod defalarca tekrarlanır (DRY ihlali — E.3)
2. Biri unutulur, o uçta uygulama **çöker**
3. Hata biçimi uçtan uca farklılaşır; arayüz her ucun hatasını ayrı ele almak zorunda kalır

Çözüm: tek hata filtresi

```
// 🚫 YANLIŞ: hata yakalama kodu her uç noktada yeniden yazılıyor
@Post('/:id/close')
async close(@Param('id') id: number, @Body() dto: CloseDto) {
  try {
    return await this.service.close(id, dto.note);
  } catch (e) {
    // Her hata türünü tek tek HTTP koduna çevirmek zorundasın
    if (e instanceof InvalidTransition) throw new ConflictException(e.message);
    if (e instanceof NotFound)         throw new NotFoundException();
    throw new InternalServerErrorException();
    // ...ve bu bloğu 40 uç noktada tekrarlıyorsun.
    // Birinde unutursan orada uygulama çöküyor.
  }
}

// ✅ DOĞRU: uç noktada hata yakalama YOK — sadece işi devrediyor
@Post('/:id/close')
close(@Param('id') id: number, @Body() dto: CloseDto) {
  // Servis hata fırlatırsa aşağıdaki merkezî filtre otomatik yakalıyor
  return this.service.close(id, dto.note);
}
```

Filtre tek yerde, tüm uygulamayı kapsar:

```
// @Catch() = "uygulamanın HER YERİNDEKİ hatayı yakala" demek.
// Tek bir yerde duruyor, 40 uç noktanın hepsini birden koruyor.
@Catch()
export class DomainExceptionFilter implements ExceptionFilter {
  catch(err: unknown, host: ArgumentsHost) {
    const res = host.switchToHttp().getResponse();

    // İsteğin başında üretilen takip numarasını al (C.16).
    // Aynı numara hem log'a hem kullanıcıya gidecek.
    const correlationId = this.cls.get('correlationId');

    // Hata türüne bakıp uygun HTTP kodunu belirle: 400 mü 409 mu 500 mü?
    const { status, code } = this.map(err);

    // Hatayı JSON olarak logla – sonradan aranabilir olsun diye (C.15)
    this.logger.error({ err, correlationId, code });

    // Kullanıcıya dönen cevap. Biçim her uçta AYNI, arayüz tek yerde ele alıyor.
    res.status(status).json({
      type:      `https://api.example/errors/${code}`,    // hatanın dokümanı
      title:     this.title(code),                        // kısa başlık
      status,    // HTTP kodu
      code,      // uygulama hata kodu
      correlationId, // kullanıcı ekranda bunu görür
      // Doğrulama hatasıysa hangi alanın hatalı olduğu da gönderilir
      errors:    err instanceof ValidationError ? err.fields : undefined,
    });
  }
}
```


Hata türleri ve HTTP kodları

Durum	Kod	Anlamı
Doğrulama hatası	400	Gönderilen veri biçimsel olarak hatalı
Kimlik doğrulanmadı	401	Giriş yapılmamış veya jeton (token) geçersiz
Yetkisiz işlem	403	Giriş var ama bu işleme yetki yok
Kayıt bulunamadı	404	—
Çakışma	409	Geçersiz durum geçişi veya eş zamanlı değişiklik (E.8)
İş kuralı ihlali	422	Biçim doğru ama kural izin vermiyor
Beklenmeyen hata	500	—

⚠ **401 ile 403 sık karıştırılır.** 401 "*seni tanımıyorum*", 403 "*seni tanıyorum ama bu işi yapamazsın*" demektir. Muhasebe personeli iş emri atamaya kalkarsa **403** alır — çünkü kim olduğu belli, yetkisi yok.

400 ile 422 farkı: "*başlık boş*" biçim hatasıdır → 400. "*pasif lokasyonda iş emri açılmaz*" iş kuralı ihlalidir → 422. Biçim doğru, kural izin vermiyor.

Cevabın içeriği ve correlation ID

Standart biçim kullanılıyor (RFC 9457 — "Problem Details"). En kritik alan **correlation ID**: kullanıcı ekranda bir kod görüyor, o kodu söylediğinde loglarda isteğin tüm yolculuğu tek aramayla bulunuyor (C.15).

🚫 **Canlıda hata detayı (stack trace) kullanıcıya gönderilmez** — sistemin iç yapısı hakkında bilgi sızdırır. Ödev §19 bunu ayrıca belirtiyor.

E.8 Transaction ve iyimser eşzamanlılık (§20)

Transaction'ın tanımı C.5'te. Burada ödevin ikinci şartı ele alınıyor: **aynı kaydın eş zamanlı güncellenmesi**.

Kayıp güncelleme (lost update) problemi

İki operasyon sorumlusu aynı iş emrini aynı anda açtı:

```
t0  A okur:  öncelik = NORMAL
t0  B okur:  öncelik = NORMAL
t1  A yazar: öncelik = HIGH
t2  B yazar: öncelik = LOW      ← A'nın değişikliği izsiz kayboldu
```

Hiçbir önlem yoksa son yazan kazanır ve **A hata görmez** — değişikliğinin uygulandığını sanır. Bu, veri kaybının en sinsi türüdür; log'a bile düşmez.

İki yaklaşım

Kötümser kilitleme (pessimistic). Kayıt okunurken satır kilitlenir (`SELECT ... FOR UPDATE`), diğerleri bekler. Kısa ömürlü ve sıcak çekişmeli işlemlerde doğrudur. Web formunda **yanlıştır**: kullanıcı sayfayı açık bırakıp ögle yemeğine giderse satır saatlerce kilitli kalır.

İyimser eşzamanlılık (optimistic) — bu projede kullanılan. Kilit yok; çakışma **yazma anında tespit** edilir.

Uygulama

Her mutasyona uğrayan tabloda bir `version` sütunu var:

```
model WorkOrder {
  id      Int @default(autoincrement()) // birincil anahtar, otomatik artıyor
  version Int @default(0)                // her güncellemede 1 artacak sayacı
  // Bu sayacı sayesinde "ben okuduktan sonra başkası değiştirmiş mi?"
  // sorusunu cevaplayabiliyoruz.
}
```

Güncelleme, okunan sürümü **koşul olarak** taşır:

```
// Güncelleme koşullu: "bu kaydı değiştir AMA sürümü hâlâ okuduğum sürümse."
const result = await tx.workOrder.updateMany({
  where: { id, version }, // ekranı açtığında okunan sürüm
  data: { priority, version: { increment: 1 } }, // değiştir ve sürümü artır
});

// ★ Bu arada başkası kaydı değiştirmişse sürüm artmış olur, koşul tutmaz
//   ve HİÇBİR satır güncellenmez. Etkilenen satır sayısı 0 gelir –
//   çakışmanın kanıtı bu.
if (result.count === 0) throw new ConcurrencyConflictError(id);
```

Üretilen SQL kabaca:

```
-- Yukarıdaki kodun veritabanına gönderdiği gerçek komut:
-- "Şu kaydın önceliğini değiştir ve sürümünü bir artır,
--   AMA yalnızca sürümü hâlâ benim okuduğum değerse."
UPDATE "WorkOrder" SET priority = $1, version = version + 1
WHERE id = $2 AND version = $3;

-- ★ Kontrol ve yazma TEK komutta. Ayı yapılsaydı ikisinin arasına başka bir
--   güncelleme girebilir ve kontrolün kendisi yarış koşulu üretti.
```

Başkası araya girmişse `version` artmıştır, `WHERE` tutmaz, **etkilenen satır sayısı 0** olur. Bu, çakışmanın kanıtıdır.

★ **Kritik ayrıntı:** kontrol ile yazma **tek bir ifadede** yapılıyor. "Önce oku, karşılaştı, sonra yaz" üç ayrı adım olsaydı adımların arasına başka bir güncelleme girebilirdi — yani kontrolün kendisi yarış koşulu üretti. `updateMany` + `count` kullanılmasının sebebi budur; `update` bulunamadığında `P2025` fırlatır ve bu hata "kayıt yok" ile "sürüm eskimiş" durumlarını ayırt ettirmez.

API davranışı

Çakışmada **409 Conflict** döner ve cevap gövdesinde çakışan alan ile güncel sürüm bulunur. Arayüz kullanıcıya "bu kayıt siz bakarken değişti" uyarısını gösterip yeniden yüklemeyi önerir.

Neden bu yaklaşım

Aynı iş emrine iki kişinin **aynı anda** dokunması nadirdir. Nadir bir durum için tüm kullanıcıları kilit kuyruğunda bekletmek yanlış takas olur; bunun yerine o nadir durumda kullanıcı bilgilendirilir. Ödev \$20 zaten "optimistic concurrency yaklaşımı uygulanmalıdır" diyor.

Transaction sınırları — nerede açılıyor, nerede açılmıyor

Ödev "her işlem için gereksiz manuel transaction açılmamalıdır" diye uyarıyor. Bu projede transaction yalnızca **birden fazla yazmanın atomik olması gereken** yerlerde açılıyor:

İşlem	Transaction	Gerekçe
Durum değişikliği + geçmiş kaydı	✓	İkisi ayrılırsa iz kaybolur
Atama + atama geçmişi	✓	Aynı
SLA ihlali işaretleme + bildirim	✓	İhlal yazılıp bildirim üretilmemesi
Refresh token yenileme + eskisini iptal	✓	İkisi ayrılırsa iki geçerli token kalır
Tek kayıt güncelleme	✗	Zaten tek ifade, kendiliğinden atomik
Listeleme / okuma	✗	Yazma yok

E.9 Veri modeli kararları (§11, §21)

★ İsimlendirme: iki dünya, iki standart, tek köprü

⚠ Bu, E.6'da reddettiğimiz "mapping" DEĞİLDİR

	E.6 — nesne dönüşümü	Burası — ad eşlemesi
Ne çevirir	Kayıt → cevap nesnesi	Alan adı → kolon adı
Nasıl	Zod şeması + Prisma select	Prisma @map niteliği
Kütüphane mi	🚫 Reddedildi (AutoMapper vb.)	★ Kütüphane değil, şema niteliği
Çalışma anı	Her istekte çalışır	🚫 Çalışmaz — derleme/sorgu üretimi anında

★ İkisi aynı anda geçerli: **@map** kolonun adını çevirir, **select** + Zod hangi alanların dışarı çıkacağını belirler. **Farklı işler, farklı katmanlar.**

Ödev §11 diyor ki: "veri tabanı tasarımı **kurum tarafından iletilen** veri tabanı geliştirme ve isimlendirme kurallarına uygun olmalıdır." ⚠ O doküman ödevin ekinde yok — sorulacak (**KURUMDAN-OGRENECEKLERİM** §1.1). Ama cevap ne olursa olsun **mimari hazır**, sebebi aşağıda.

Sorun: İki katmanın yerleşik standardı **birbirine uymaz.**

Katman	Yerleşik pratik	Örnek
TypeScript / NestJS	camelCase alan · tekil tip	dueAt , WorkOrder
PostgreSQL	snake_case kolon · çoğul tablo	due_at , work_orders

🚫 **Birini diğerine feda etmiyoruz.** Her katman kendi pratiğini koruyor; çeviriyi **Prisma** yapıyor.

Önce tek bir alanın üç ayrı yerdeki adı:

Nerede	Adı	Kim görür
TypeScript kodunda	dueAt	Sen, kod yazarken
Prisma şemasında	dueAt @map("due_at")	★ İkisini bağlayan satır
PostgreSQL tablosunda	due_at	Veritabanına bakan DevOps

Aynı veri, üç ayrı ad. Ortadaki satır **sözlük** görevi görüyor.

SEN YAZARSIN

PRİSMA ÇEVİRİR

VERİTABANINDA OLUŞUR

wo.dueAt	↔	dueAt @map("due_at")	↔	kolon: due_at
prisma.workOrder	↔	model WorkOrder	↔	tablo: work_orders
		@@map("work_orders")		

Şimdi şemanın kendisi:

```
// apps/api/prisma/schema.prisma
// Bu dosya veri modelinin TEK doğru kaynağı. Buradan hem veritabanı tabloları
// hem de TypeScript tipleri üretiliyor.

model WorkOrder {
  // — Bu satırlarda @map YOK, çünkü ad zaten iki dünyada da aynı —
  id    String @id @default(uuid()) // "id" hem TS'te hem SQL'de "id"
  code  String @unique               // "code" hem TS'te hem SQL'de "code"

  // — Bu satırlarda @map VAR, çünkü adlar ayrışıyor —
  //
  // dueAt    DateTime    @map("due_at")
  // —————
  //   ↑                ↑
  //   |                | — PostgreSQL'de oluşacak KOLON adı (snake_case)
  //   |                | — Kodda yazacağın ALAN adı (camelCase)
  //
  // Yani: sen "wo.dueAt" yazarsın, veritabanında "due_at" kolonu vardır.
  dueAt    DateTime @map("due_at")

  // Aynı mantık: kodda "createdAt", veritabanında "created_at"
  // @default(now()) → kayıt eklenirken o anki zamanı otomatik yazar
  createdAt DateTime @default(now()) @map("created_at")

  // — @@map (İki tane @) alanı değil, TABLONUN KENDİSİNİ eşler —
  // Kodda      : prisma.workOrder   (model adından türer, tekil camelCase)
  // Veritabanı : work_orders         (aşağıdaki satırın söylediği, çoğul)
  @@map("work_orders")
}
```

★ **Tek kural olarak özet:** `@map` bir alanı, `@@map` bir tabloyu eşler. Sol taraf **senin yazdığın**, sağ taraf **veritabanında duran**.

Kod tarafında hiç alt çizgi görmezsin:

```
// apps/api/src/work-orders/work-orders.service.ts
//
// NEREDEN : apps/web/.../talep-formu.tsx → form gövdesi, contracts şemasından geçmiş
// NE      : SLA'dan hesaplanan bitiş zamanıyla birlikte kayıt açılıyor
// NEREYE  : PostgreSQL "work_orders" tablosu
// SONUÇ   : Veritabanında yeni bir satır; ekranda listenin başında görünür

const created = await this.prisma.workOrder.create({
  data: {
    code, // ★ Temiz TypeScript adı – alt çizgi yok
    dueAt: plan.dueAt, // Prisma bunu "due_at"e çevirecek
  },
});

// ★ ARKA PLANDA PRISMA'NIN ÜRETTİĞİ SQL – PostgreSQL standardında:
// INSERT INTO "work_orders" ("id", "code", "due_at", "created_at")
// VALUES ('7b2d...', 'IE-2026-000148', '2026-08-26 20:00', '2026-08-26 16:00');
```

Okurken de aynı çeviri ters yönde çalışır:

```
const list = await this.prisma.workOrder.findMany();
// Prisma "due_at" kolonunu okur, NestJS'e teslim etmeden önce "dueAt"e çevirir.
// Next.js'e giden JSON tertemiz camelCase olur:
// [{ "id": "7b2d...", "code": "IE-2026-000148", "dueAt": "2026-08-26T20:00:00.000Z" }]
```

★ Bu, E.1'deki katman bağımsızlığının en somut karşılığı. Veritabanı adlandırması değişirse **yalnızca @map** satırları değişir; servis, controller ve ekran kodunun **tek satırı** bile dokunulmaz.

⚠ Kurumdan farklı bir kural gelirse (örn. tablo adları tekil, ya da **WorkOrder** PascalCase) yalnızca @map / @@map tarafı ona uydurulur. TypeScript tarafı **değişmez** — köprünün varlık sebebi tam olarak budur.

🚫 @map sonradan eklenmez. İlk migration'dan **önce** yazılır. Sonra eklenirse kolon yeniden adlandırma migration'ı gerekir ve canlıda veri taşıma riski doğar.

i Değerlendirmeci sorarsa

"İki katmanın isimlendirme standardı farklı. Uygulama katmanında TypeScript pratiğini, veri katmanında PostgreSQL pratiğini korudum; ikisi arasındaki dönüşümü Prisma'nın @map niteliğiyle soyutladım. Kurum kendi kuralını verirse yalnızca şema tarafı değişir, kod dokunulmaz."

Ödev, veri tabanı tasarımının tarafımızdan yapılmasını ve **kararların dokümante edilmesini** istiyor. Aşağıdakiler o kararlar ve gerekçeleri.

Soft delete (yumuşak silme)

Gerçek hayat: Bir kurumda emekli olan personelin kaydı **silinmez**; "pasif" işaretlenir. Silinseydi, o kişinin yıllar önce imzaladığı belgeler sahipsiz kalırdı.

Yazılımda: Kaydı gerçekten silmek yerine "silindi" olarak işaretlemek.

```
model Location {
    id          Int          @id @default(autoincrement()) // otomatik artan numara
    name        String
    // Soft delete: kayıt gerçekten silinmiyor, buraya silinme tarihi yazılıyor.
    // null  = kayıt aktif
    // dolu  = silinmiş sayılıyor ama veritabanında duruyor
    deletedAt   DateTime?
    // Her sorguda "silinmemiş olanlar" diye filtrelendiği için index gerekli
    @@index([deletedAt])
}
```

Bu projede neden gerekli: Bir lokasyon kapandı diye silinirse, o lokasyona bağlı geçmiş iş emirleri sahipsiz kalır. *"Bu iş emri hangi binadaydı?"* sorusu cevapsız kalır — denetim açısından kabul edilemez.

Nerede kullanılıyor: Lokasyon, varlık, kullanıcı — yani **başka kayıtların referans verdiği** tablolarda. İş emri geçmişisi gibi zaten değişmeyen kayıtlarda gerek yok.

⚠ **Bedeli:** Her sorguya "silinmemiş olanlar" koşulu eklemek gerekir. Unutulursa silinmiş kayıtlar listede görünür. Bu yüzden koşul tek tek yazılmıyor, merkezî bir filtreyle uygulanıyor:

```
// Her listeleme sorgusunun ilk koşulu: silinmiş kayıtları getirme.
// Kullanıcının seçtiği filtreler bunun ÜSTÜNE ekleniyor.
// ⚠ Bu koşul bir sorguda unutulursa silinmiş kayıtlar listede görünür –
//   hata vermez, sessizce yanlış veri gösterir.
const where = { deletedAt: null, ...userFilters };
```

Audit alanları

Her kayıтта dört alan: **kim oluşturdu, ne zaman, kim güncelledi, ne zaman.**

```
model WorkOrder {
  createdAt DateTime @default(now()) // kayıt açıldığı an, veritabanı doldurur
  createdBy Int // kim açtı — Prisma eklentisi doldurur
  updatedAt DateTime @updatedAt // son değişiklik anı, otomatik güncellenir
  updatedBy Int? // kim güncelledi. İlk kayıttaki boş olabilir
  // Bu dört alan servis kodunda ELLE doldurulmuyor; merkezî bir eklenti
  // aktif kullanıcıyı okuyup kendisi yazıyor (C.16). Unutma ihtimali yok.
}
```

Bunlar **elle doldurulmuyor** — Prisma eklentisi `nestjs-cls` bağlamından okuyup otomatik dolduruyor (C.16). Sebep: elle doldurulan bir alan er geç unutulur ve o kaydın izi kaybolur.

Enum'ların saklanma biçimi

Enum, sınırlı sayıda seçenekten oluşan alan: durum, öncelik, iş emri türü.

İki seçenek var: **sayı** olarak saklamak (0, 1, 2) veya **metin** olarak (`OPEN` , `ASSIGNED`). Bu projede **metin** tercih edildi.

```
-- Enum'u SAYI olarak saklarsak:
-- Veritabanına bakan kişi 3'ün ne demek olduğunu bilemez, kod dosyasını açar.
SELECT status FROM "WorkOrder"; -- sonuç: 3

-- Enum'u METİN olarak saklarsak:
-- Veritabanına bakan herkes ne olduğunu doğrudan anlar.
SELECT status FROM "WorkOrder"; -- sonuç: 'RESOLVED'
```

Neden: Veritabanına doğrudan bakan biri (DevOps, raporcu, denetçi) sayıyı anlamaz. Ayrıca araya yeni bir değer eklendiğinde sayılar kayar ve **eski kayıtlar yanlış anlam kazanır** — metinde bu risk yok.

Bedeli: Birkaç bayt daha fazla yer. Bu ölçekte önemsiz.

Tarih ve saat standardı

Tüm zamanlar veritabanında **UTC** (evrensel zaman) olarak saklanıyor; yalnızca kullanıcıya gösterilirken yerel saate çevriliyor.

Neden: Yaz saati uygulaması değiştiğinde veya sistem başka bir sunucuya taşındığında yerel saatle saklanmış veriler **kayar**. SLA hesabı saat farkına duyarlı olduğu için bu gerçek bir risk: bir iş emrinin "süresi geçti mi" sorusunun cevabı sunucunun saat dilimine göre değişmez.

Index kararları

Index nedir: Kitabın arkasındaki dizin — aranan satırı tüm tabloyu okumadan bulmayı sağlar.

⚠ **Index bedava değildir:** her yazma işleminde güncellenmesi gerekir. Bu yüzden "her kolona index" yanlış bir yaklaşımdır; index **fiilen kullanılan filtrele**re göre konur.

Index	Neden
<code>status</code>	Liste ekranının en sık filtresi
<code>assigneeId</code>	Teknik personel "bana atananlar" ekranı
<code>slaDueAt</code>	"SLA'sı yaklaşanlar" filtresi + arka plan taraması
<code>(locationId, status)</code> birleşik	İkisi birlikte filtreleneiyor
<code>title</code> GIN + <code>pg_trgm</code>	Metin araması (C.5)
<code>deletedAt</code>	Her sorguda kontrol ediliyor

★ **Birleşik (composite) index'te sıra önemlidir.** `(locationId, status)` index'i, yalnızca `locationId` ile yapılan aramada da çalışır; ama yalnızca `status` ile yapılan aramada **çalışmaz**. Bu yüzden sık kullanılan alan başa konur.

İş emri numarası üretimi

Ödev, iş emrinin **benzersiz ve okunabilir** bir numarası olmasını ve üretim yönteminin tarafımızdan tasarlanmasını istiyor.

Biçim: `IE-2026-000148` → sabit ön ek + yıl + sıfır dolgulu sıra numarası.

Neden bu biçim: Telefonda söylenebilir, gözle sıralanabilir, hangi yıla ait olduğu okunur. Kurumsal sistemlerde numara insanlar arasında konuşulan bir şeydir — `a3f9c2b1-...` gibi bir kimlik bu işi görmez.

Nasıl üretiliyor:

```
-- Veritabanının kendi sayacı. Her çağrıldığında bir sonraki sayıyı verir
-- ve iki istek aynı anda gelse bile ikisine AYNI sayıyı vermez.
CREATE SEQUENCE work_order_seq_2026 START 1;
```

```
// Veritabanı sayacından sıradaki numarayı iste (örn. 148)
const [{ nextval }] = await tx.$queryRaw`SELECT nextval('work_order_seq_2026')`;

// İnsanın okuyacağı biçime çevir: başına sıfır ekleyip ön ek ve yıl koy
// 148 → "IE-2026-000148"
const number = `IE-2026-${String(nextval).padStart(6, '0')}`;
```

🚫 Neden uygulama içinde "son numarayı bul, bir artır" yapılmıyor:

```
// 🚫 YANLIŞ YOL: "son numarayı bul, bir artır"
// Son kaydı oku
const son = await prisma.workOrder.findFirst({ orderBy: { id: 'desc' } });
// Bir artır
const number = `IE-2026-${son.seq + 1}`;

// ⚠️ İki talep aynı anda gelirse ikisi de AYNI son kaydı okur ve aynı
//   numarayı üretir. Buna yarış koşulu denir; tek kullanıcıli testte
//   asla görünmez, yük altında ortaya çıkar.
```

Veritabanı sayacı (sequence) bu yarış **yapısal olarak** engeller: iki eş zamanlı çağrı asla aynı sayıyı almaz. Üstelik transaction geri alınsa bile sayaç geri dönmez — yani numara "boşluklu" olabilir ama **asla tekrarlanmaz**.

Numaranın kendisi ayrıca `@unique` işaretli; ikinci savunma hattı.

Eşzamanlılık alanı

Her mutasyona uğrayan tabloda `version` sütunu var — gerekçesi ve çalışma biçimi E.8'de.

E.10 Listeleme, filtreleme ve sayfalama (§17)

Ne isteniyor: İş emri listesinde sunucu tarafı filtreleme, sıralama ve sayfalama; *"bütün kayıtlar belleğe alındıktan sonra filtreleme yapılmamalıdır."*

Sunucu tarafı ne demek, neden zorunlu

İki yaklaşım var:

	İstemci tarafı	Sunucu tarafı
Nasıl	Tüm kayıtlar tarayıcıya gider, filtre orada	Filtre + sayfa bilgisi sunucuya gider, DB yalnızca o sayfayı döner
200 kayıta	Çalışır	Çalışır
200.000 kayıta	Tarayıcı donar, ağ tıkanır	Fark etmez
Güvenlik	🚫 Kullanıcının görmemesi gereken kayıtlar da tarayıcıya iner	Yetki filtresi sorguya girer

Güvenlik maddesi tek başına belirleyicidir: istemci tarafı filtreleme, yetki kontrolünü tarayıcıya devretmek demektir.

Sorgu yapısı

```
// Kullanıcının ekrandan seçtiği filtreler burada tek bir koşula dönüşüyor.
// "&&" kalıbı şu demek: kullanıcı o filtreyi seçmediyse koşula hiç eklenmiyor.
const where: Prisma.WorkOrderWhereInput = {
  deletedAt: null, // silinmişler hiç gelmesin

  ...(status && { status }), // durum seçildiyse ekle
  ...(assignee && { assigneeId: assignee }), // personel seçildiyse ekle
  ...(locationId && { locationId }), // lokasyon seçildiyse ekle
  ...(slaBreached !== undefined && { slaBreached }), // "SLA ihlali" kutusu

  // Tarih aralığı: başlangıç girildiyse iki tarih arasını filtrele
  ...(createdFrom && { createdAt: { gte: createdFrom, lte: createdTo } }),

  // Arama kutusuna yazı girildiyse: başlıkta VEYA numarada ara.
  // mode:'insensitive' = büyük/küçük harf farkı gözetme
  ...(q && { OR: [
    { title: { contains: q, mode: 'insensitive' } },
    { number: { contains: q, mode: 'insensitive' } },
  ]}),
};

// İki sorgu birlikte çalışıyor: hem o sayfanın kayıtları hem toplam sayı.
// Aynı işlem içinde oldukları için arada yeni kayıt eklenip sayı ile
// liste tutarsız hâle gelemiyor.
const [rows, total] = await prisma.$transaction([
  prisma.workOrder.findMany({ where, orderBy, skip, take, select }), // sayfanın kayıtları
  prisma.workOrder.count({ where }), // kaç kayıt var
]);
```

Kayıtlar ve toplam sayı **aynı sorgu bağlamında** alınır; ayrı çalıştırılırsalardı arada bir kayıt eklenip toplam ile sayfa tutarsız olabilirdi.

Uygulanan kurallar

- **Varsayılan ve azami sayfa boyutu.** `?limit=999999` ile sunucu yorulamaz; değer azamiye kırılır.
- **Sıralama alanları beyaz listede.** Kullanıcının gönderdiği alan adı doğrudan `orderBy` 'a konmaz; izin verilen alanlar arasında değilse istek reddedilir. Aksi hâlde indexlenmemiş bir kolonla tam tablo taraması tetiklenebilir.
- **Filtreler URL'de taşınır.** Arayüz tarafında filtre durumu sorgu parametrelerine yazılır; böylece liste **paylaşılabılır ve yer imine eklenebilir** olur (ödev §22 bunu ayrıca istiyor).

- **Sayfalama kodu ortak.** Her uç için ayrı yazılmaz (DRY — E.3).

Cevap zarfı

Ödev §17 dönen bilgileri tek tek sayıyor:

```
{
  "items": [ ... ],           // bu sayfadaki kayıtlar (en fazla pageSize kadar)
  "page": 2,                 // kaçınıcı sayfadayız
  "pageSize": 20,            // sayfa başına kaç kayıt
  "totalCount": 431,         // filtreye uyan TOPLAM kayıt sayısı
  "totalPages": 22,          // toplam kaç sayfa var
  "hasNext": true,           // sonraki sayfa var mı – arayüz butonu buna bakıyor
  "hasPrevious": true        // önceki sayfa var mı
}
```

★ Offset yerine cursor ne zaman gerekir

İki sayfalama yöntemi var. Aralarındaki fark, **kullanıcı 5000. sayfaya gittiğinde ve liste sürekli akarken** ortaya çıkıyor.

Yöntem 1 — Offset (bu projede kullanılan)

"İlk 100 kaydı **atla**, sonraki 20'yi ver."

```
// Prisma karşılığı – 6. sayfa, sayfa boyutu 20
await prisma.workOrder.findMany({
  skip: 100,    // ← 5 sayfa × 20 kayıt = 100 kaydı ATLA (offset)
  take: 20,     // ← sonraki 20 kaydı al (limit)
  orderBy: { createdAt: 'desc' },
});
```

```
-- Üretilen SQL
SELECT * FROM "WorkOrder" ORDER BY "createdAt" DESC
LIMIT 20 OFFSET 100;
```

🚫 **Gizli maliyeti: OFFSET 100** demek, veritabanının o 100 satırı **gerçekten okuyup atması** demektir. Atlanan satırlar bedava değildir.

Sayfa	Atlanan satır	Veritabanı ne yapıyor
2	20	20 satır okuyup atıyor, 20 döndürüyor — hızlı
100	1.980	1.980 satır okuyup atıyor — fark edilir
5.000	99.980	99.980 satır okuyup atıyor, 20 döndürüyor — 🚫 saniyeler

Gerçek hayat benzetmesi

Bir kitabın 500. sayfasını bulmak için sayfaları **tek tek çevirerek** saymak. 5. sayfa için 5 çevirirsin, 500. sayfa için 500. Kitap kalınlaştıkça aynı iş katlanarak zorlaşır — oysa aradığın şey her seferinde tek bir sayfa.

⚠️ **İkinci sorun — kayma (drift):** Sen 2. sayfaya bakarken listenin başına **yeni bir kayıt eklenirse** her şey bir sıra kayar. 1. sayfanın son kaydını 2. sayfada **tekrar** görürsün; bir kayıt ise **hiç görünmez**.

```
10:00 Sayfa 1 → [K20, K19, K18 ... K1]          (en yeniden eskiye)
10:01 Yeni kayıt K21 eklendi
10:02 Sayfa 2 → [K1, ...] 🚫 K1'i 1. sayfada da görmüştün
      🚫 ve bir kayıt atlandı
```

Yöntem 2 — Cursor (imleç)

"**Şu kayıttan sonrakini ver.**" Atlama yok; veritabanı doğrudan o noktaya gidiyor.

```
// Kullanıcı en son K81'i gördü. Sonraki sayfa:
await prisma.workOrder.findMany({
  take: 20,
  cursor: { id: 'K81' }, // ★ "Bu kayıttan başla" – atlama yok
  skip: 1,               // K81'in kendisini tekrar gönderme
  orderBy: { createdAt: 'desc' },
});
```

```
-- Üretilen SQL'in özü: eşitlik değil, KARŞILAŞTIRMA
SELECT * FROM "WorkOrder"
WHERE ("createdAt", "id") < ('2026-08-24 10:00', 'K81')
--   └─ index bu noktaya DOĞRUDAN atlıyor, önündekileri saymıyor
ORDER BY "createdAt" DESC, "id" DESC
LIMIT 20;
```

★ **Kazanç:** 1. sayfa ile 5.000. sayfa **aynı hızda** çalışır — çünkü ikisi de "şu noktadan sonraki 20 kayıt" sorusudur. Araya yeni kayıt girse de kayma olmaz; imleç bir kaydı işaret ediyor, bir sayı değil.

🚫 **Bedeli:** "7. sayfaya git" diyemezsin. Cursor yalnızca **ileri/geri** gider, sayfa numarası kavramı yoktur. Toplam sayfa sayısı da gösteremezsin.

Karar tablosu — hangisi ne zaman

Durum	Yöntem	Neden
Kullanıcı sayfa numarasına tıklıyor ("Sayfa 7")	Offset	Cursor sayfa numarası veremez
Toplam sayfa sayısı gösteriliyor ("1/48")	Offset	Cursor toplamı bilmez
Kullanıcı filtreleyerek daraltıyor , derine inmiyor	Offset	Derin sayfa hiç oluşmuyor; offset daha basit
Sonsuz kaydırma (aşağı indikçe yükleniyor)	Cursor	Doğal olarak "sonrakini ver" akışı
Liste sürekli akıyor (bildirim, olay günlüğü)	Cursor	🚫 Offset'te kayma kaçınılmaz
Çok büyük tablo + derin sayfalama gerçekten oluyor	Cursor	Offset saniyelere çıkar
Dışa aktarma / toplu okuma (tüm kayıtları gez)	Cursor	Sabit hızda ilerler

★ Bu projedeki karar ve gerekçesi

İş emri listesi → **offset**. Üç sebep:

- Ekranda **sayfa numaraları** var ve "48 kayıttan 1–20 arası" yazıyor — ikisi de cursor'ın veremediği şeyler
- Kullanıcı lokasyon, durum ve tarihe göre **filtreleyerek daraltıyor**; 5.000. sayfa pratikte oluşmuyor
- Filtreler adres çubuğunda tutuluyor (BÖLÜM G → 3. adım); **?sayfa=3** paylaşılabılır bir adres, **?cursor=K81** değil

Bildirim listesi → **cursor** (eklendiğinde). Sürekli akıyor ve sonsuz kaydırmayla gösterilecek; offset'te kullanıcı aynı bildirimi iki kez görürdü.

⚠️ **Koruma — offset'in sınırı zorlanmasın:** azami sayfa boyutu **100** ile sınırlı ve **sayfa** parametresi doğrulanıyor. Biri adres çubuğuna **?sayfa=999999** yazarsa istek **reddediliyor**, veritabanı 20 milyon satır taramaya kalkmıyor.

i Değerlendirmeci bunu sorarsa

"Offset seçtim çünkü bu ekran sayfa numarası ve toplam sayısı gösteriyor; cursor ikisini de veremez. Derin sayfalama riskini biliyorum — azami sayfa boyutu ve parametre doğrulamasıyla sınırladım. Bildirim listesi gibi sürekli akan bir yerde cursor kullanırdım, çünkü orada kayma sorunu gerçekten ortaya çıkar."

E.11 Git akışı ve CI (§26, §27)

Git nedir

Git, kodun değişiklik geçmişini tutan programdır. Bilgisayarda çalışır, internet gerektirmez. Her kayıt (**commit**) bir fotoğraf gibidir: o an kodun hâli ve neden değiştiği yazılıdır.

GitHub / GitLab ise bu geçmişin bir kopyasını internette veya kurumun kendi sunucusunda tutan **barındırma servisleridir**. Git'in vermediklerini eklerler: kod inceleme ekranı, değişiklik önerisi (Pull Request / Merge Request), otomatik test çalıştırma.

Benzetme: Git = belgeyi yazan ve sürümlerini tutan program. GitHub/GitLab = belgeyi sakladığın, paylaştığın ve üzerine yorum aldığın yer.

Bu projedeki dal akışı

```
main'den yeni dal aç → çalış, commit at → uzağa gönder  
→ değişiklik önerisi aç → testler yeşil → main'e birleştir → dal silinir
```

★ GitHub Flow — adım adım, İngilizce karşılıklarıyla

Yukarıdaki akışın piyasadaki adı **GitHub Flow**'dur. Mülakatta ve ekip içinde İngilizce adlarıyla anılır; ikisini de bilmen gerekiyor.

#	Adım	İngilizcesi	Ne oluyor
1	<code>main</code> 'den yeni dal aç	Create branch	<code>main</code> 'in bir kopyası üzerinde çalışırsın; ana dal bozulmaz. Ad: <code>feature/is-emri-atama</code>
2	Çalış, commit at	Commit changes	Her commit tek bir işi anlatır. Commit = o anki kodun fotoğrafı + neden değiştiği
3	Uzağa gönder	Push to remote	Dalın GitHub/GitLab'a çıkar. Buraya kadar kimse görmüyordu
4	Değişiklik önerisi aç	Open Pull Request (PR)	" <i>Şu dalı <code>main</code> 'e almak istiyorum</i> " teklifi. ★ Geliştirici burada PR açıklaması yazar: ne değişti, neden, nasıl test edilir
5	Kod gözden geçirme	Code Review	Ekipteki başka bir geliştirici (peer) veya kıdemli (senior) satır satır inceler, yorum bırakır, değişiklik ister
6	Testler yeşil	CI Pass	Otomatik denetimler koşar; biri bile kırmızıysa birleştirilemez
7	<code>main</code> 'e birleştir	Merge into main	Değişiklik ana dala girer
8	Canlıya çıkış	Deploy to production	<code>main</code> yayına alınır (bu projede: DevOps ekibi)
9	Dal silinir	Delete branch	İşi biten dal temizlenir; geçmişi commit'lerde zaten duruyor

★ 4. adımdaki PR açıklaması neden önemli: Kod *ne* yaptığını gösterir, PR açıklaması *neden* yaptığını. İnceleyen kişi bunu okumadan koda bakarsa her satırı sorgulamak zorunda kalır. Ödev §26'nın "*commit geçmişisi geliştirme sürecini göstermeli*" şartı bununla da karşılanıyor.

6. adımda tam olarak ne koşuyor — bu projede

"Testler yeşil" tek bir şey değil; **yedi ayrı kapı**:

#	Kapı	Araç	Neyi yakalar
1	Biçim ve kural	ESLint + Prettier	Kullanılmayan değişken, tehlikeli kalıp, biçim bozukluğu
2	Tip denetimi	TypeScript (<code>tsc --noEmit</code>)	Olmayan alana erişim, yanlış tip
3	Birim testler	Vitest	İş kuralları — SLA hesabı, durum makinesi
4	Entegrasyon testleri	Testcontainers + Vitest	Gerçek PostgreSQL'e karşı: transaction, kısıtlar, eşzamanlılık
5	Mimari testi	dependency-cruiser	Domain katmanına Prisma sızmış mı (C.8)
6	Uçtan uca	Playwright	Gerçek tarayıcıda gerçek tıklama
7	Derleme	<code>next build</code> · <code>nest build</code> · <code>docker build</code>	Derlenmiyorsa yayına çıkamaz

Bunların hepsi `pnpm ci:verify` içinde toplanır — ★ **aynı komutu kendi bilgisayarında da çalıştırabilirsin**. CI dosyası yalnızca bu komutu çağıran ince bir sarmalayıcıdır (E.11 → "*CI platforma bağımlı yazılmaz*").

"Başka test aracı var mı" — dürüst cevap

Araç	Kullanıyor muyuz	Neden
GitHub Actions	✅ Evet	Yukarıdaki yedi kapıyı çalıştıran motor
GitLab CI	✅ Hazır	<code>.gitlab-ci.yml</code> aynı <code>npm ci:verify</code> 'ı çağırıyor — kurum GitLab'a geçince ek iş yok
Renovate	✅ Evet	Bağımlılık güncellemelerini PR olarak açar (C.23)
SonarQube	⚠️ Hayır	Kod kalitesi ve teknik borç ölçeği ayrı bir platform. Kurumsal ortamda yaygın; ayrı sunucu ister ve bizim yedi kapımızın çoğunu ESLint + TypeScript zaten yapıyor. ★ Kurum kullanıyorsa CI'a bir adım olarak eklenir — mimariyi değiştirmez
CodeQL	⚠️ Hayır	GitHub'ın güvenlik tarayıcısı. Ücretsiz ve tek satır — eklenmesi kolay, ama ödevde istenmedi. Teknik borç listesine yazıldı
Codecov	⚠️ Hayır	Test kapsamı raporlama servisi. Kapsamı Vitest zaten ölçüyor; dışarı veri göndermek KVKK açısından gereksiz risk

GitHub ile GitLab farkı — akış aynı, isimler farklı

★ İş akışının kendisi birebir aynıdır. Değişen tek şey isimler ve dosya adları:

Konu	GitHub	GitLab
Değişiklik önerisi	Pull Request (PR)	Merge Request (MR)
CI tanım dosyası	<code>.github/workflows/ci.yml</code>	<code>.gitlab-ci.yml</code>
CI'ı çalıştıran	GitHub Actions	GitLab CI/CD
Komut satırı aracı	<code>gh</code>	<code>glab</code>
Otomatik güncelleme botu	Dependabot veya Renovate	🚫 Dependabot yok → Renovate
Güvenlik taraması	CodeQL	Yerleşik SAST (Ultimate sürümde)
Kurumun kendi sunucusuna kurulabilir mi	Enterprise Server (ücretli)	✅ Community Edition ücretsiz

★ **Son satır belediyeler için belirleyicidir:** GitLab kurumun kendi sunucusuna ücretsiz kurulabildiği için **kod kurum dışına hiç çıkmaz**. Kamu tarafında yaygın olmasının sebebi teknik üstünlük değil, budur.

⚠️ **Bu yüzden Renovate seçildi, Dependabot değil** (C.23): Dependabot yalnızca GitHub'da çalışır. Kurum GitLab'a geçtiğinde bot da taşınabilsin diye.

- `main` her zaman çalışır durumdadır ve doğrudan commit edilmez
- Her commit **tek bir işi** anlatır; ilgisiz değişiklikler aynı commit'te birleştirilmez

- Ödev §26 "projenin tamamının tek commit ile gönderilmemesi" ve "commit geçmişinin geliştirme sürecini göstermesi" şartını koyuyor — yani geçmişin kendisi de teslimin parçası

🚫 **Gizli değerler (şifreler, anahtarlar) asla commit edilmez.** `.env` dosyası gönderilmez; yalnızca hangi değişkenlerin gerektiğini gösteren `.env.example` gönderilir.

CI nedir

CI (Continuous Integration — sürekli entegrasyon), her değişiklik önerisinde testlerin **otomatik** çalışmasıdır.

Neden gerekli: "bende çalışıyordum" cümlesini ortadan kaldırır. Kod, kimsenin bilgisayarına bağlı olmayan temiz bir ortamda derlenir ve test edilir.

Bu projedeki zincir: bağımlılıkları kur → tip kontrolü → birim testler → entegrasyon testleri → mimari testler → arayüz derlemesi → Docker imajı. Herhangi biri kırmızı yanarsa değişiklik birleştirilemez.

Taşınabilirlik notu: Adımların kendisi proje içindeki bir komut dosyasında yaşar; CI dosyaları yalnızca o komutu çağırır. Böylece hem GitHub Actions hem GitLab CI aynı işi yapar ve platform değiştiğinde yeniden yazılmaz.

E.12 Dokümantasyon: ADR ve AI_USAGE (§28, §29)

ADR — mimari karar kaydı

Nedir: *Architecture Decision Record*. Önemli bir teknik kararın **neden** alındığını yazan kısa belge. Ödev en az üç tane istiyor.

Bu projede yazılacak ADR'ler

No	Konu	Rehberde gerekçesi
ADR-001	Neden bu stack — .NET yerine JS ailesi	Giriş · BÖLÜM A
ADR-002	Neden ayrı NestJS API (kitin varsayılanı Next tek başına)	E.1
ADR-003	Neden Zod, <code>class-validator</code> değil (Nest'in resmî yolu)	C.4
ADR-004	Mapping kütüphanesi kullanılmaması	E.6
ADR-005	Repository Pattern eklenmemesi	E.13
ADR-006	Redis'in eklenmesi (kuyruk + hız sınırı + kilit)	C.6
ADR-007	Next.js seçimi — Vite + React Router yerine	C.2
ADR-008	SLA politikası ve karma takvim kararı	E.4

⚠ **Liste kapalı değil.** Yapım sırasında ölçüyle verilen her sapma yeni bir ADR doğurur. 🚫 Gerekçesiz sapma yasak; gerekçeli sapma **ADR'ye yazılır.**

Gerçek hayat: Bir binada kolonun ortada durduğunu görürsünüz ve "burası kapı olsaydı daha iyiydi" dersiniz. Projeyi çizen mühendis o kolonu oraya **zemin etüdü** yüzünden koymuştur — ama bunu kimse yazmamışsa, biri gelip kolonu kaldırır.

Yazılımda: Kod *ne* yapıldığını gösterir, **neden** yapıldığını göstermez. Bir yıl sonra biri "*burası neden böyle?*" diye sorduğunda cevap ya ADR'dedir ya da kaybolmuştur. Kaybolduğunda o karar genellikle bozulur ve aynı duvara yeniden toslanır.

Biçimi:

```
# ADR-004 – Mapping kütüphanesi kullanılmaması

## Durum
<!-- Karar hangi aşamada: Önerildi / Kabul edildi / Geçersiz kılındı -->
Kabul edildi – 2026-08-21

## Bağlam
<!-- Hangi sorunla karşılaşıldı, hangi kısıtlar vardı -->
Ödev AutoMapper veya Mapster kullanımını zorunlu tutuyor. JS tarafındaki
adaylar ölçüldü: class-transformer (12M/hafta ama son yayın 2022),
@autmapper/core (bakımda ama 106K/hafta).

## Karar
<!-- Ne yapılmasına karar verildi -->
Mapping kütüphanesi kullanılmayacak. Response şekli Zod şemasında tanımlanacak,
Prisma select ondan türetilecek, çıkışta schema.parse() ile fazla alan kırılacak.

## Sonuçlar
<!-- Artılar VE eksiler birlikte. Eksileri yazmak kararın düşünüldüğünün kanıtı -->
+ Hatalar derleme zamanında yakalanıyor
+ Hassas alan sızması yapısal olarak engelleniyor
- Ödevin harfi harfine istediği araç kullanılmadı; sunumda gerekçelendirilecek
```

Bu projedeki ADR'ler: mimari yaklaşım seçimi · ayrı backend kararı · Repository Pattern kullanılmaması · mapping yaklaşımı · eşzamanlılık yaklaşımı · enum saklama biçimi.

★ **En değerli kısmı "Sonuçlar" bölümündeki eksi maddeler.** Bir kararın bedelini yazmak, kararın **düşünülmüş** olduğunun kanıtıdır. Yalnızca artıları yazan ADR, karar kaydı değil savunma metnidir.

AI_USAGE.md

Ne isteniyor: Ödev §28 yapay zekâ kullanımını serbest bırakıyor ancak **belgelenmesini** şart koşuyor: hangi araçlar, hangi aşamalarda, üretilen kodda yapılan önemli değişiklikler, **aracın yanlış veya eksik ürettiği en az bir örnek**, çıktıların nasıl doğrulandığı.

Ödevin kendi ifadesiyle: "'Agent böyle oluşturdu' açıklaması teknik kararların sorumluluğunu ortadan kaldırmaz."

Neden bu dosya aslında bir avantaj: Aracın hatasını **fark edebilmek**, aracı kullanabilmekten daha değerli bir yetkinliktir. Dürüstçe yazılmış bir AI_USAGE dosyası, kodu sahiplendiğinizi gösterir.

Bu projede yazılacak gerçek örneklerden biri:

Araç, veri modeli dokümanını üretmek için `prisma-dbml-generator` paketini önerdi. Paket kontrol edildiğinde son yayınının 2024 başında olduğu ve Prisma 7 ile çalışmayacağı görüldü. Yerine `@dbml/cli` ile migration'ın ürettiği gerçek şemadan üretme yöntemi kuruldu ve CI'a bir tutarlılık kontrolü eklendi.

Bu tür bir kayıt iki şeyi birden gösteriyor: aracın önerisinin **doğrulandığını** ve yerine konan çözümün **daha sağlam** olduğunu.

Zorunlu doküman listesi

Ödev §29 şunları zorunlu tutuyor:

Dosya	İçeriği
<code>README.md</code>	Kurulum, Docker'lı ve Docker'sız çalıştırma, migration, testler, ortam değişkenleri, geliştirme kullanıcıları, bilinen eksikler, varsayımlar
<code>AI_USAGE.md</code>	Yukarıdaki içerik
<code>docs/database.dbml</code>	Veri modeli (C.21 — otomatik üretiliyor)
<code>docs/database-decisions.md</code>	E.9'daki kararlar
<code>docs/architecture.md</code>	Katmanlar, bağımlılık yönleri, istek yaşam döngüsü, transaction sınırları
<code>docs/api.md</code>	Uç (<i>endpoint</i>) listesi ve sözleşmeler (<i>API Contract — aşağıda</i>)
<code>docs/testing.md</code>	Test stratejisi ve nasıl koşulacağı
<code>docs/lifecycle.md</code>	Servis yaşam döngüsü tablosu (C.1 §4)
<code>docs/background-jobs.md</code>	Dört iş, zamanlamaları, idempotency yaklaşımı
<code>docs/decisions/</code>	ADR'ler

i "Sözleşme" ne demek — İngilizce karşılığı **API Contract

Bir API'nin, kendisini kullananlara verdiği **yazılı söz**: "*bana şu şekilde istek gönderirsen, sana şu şekilde cevap dönerim.*"

Gerçek hayat: Kargo firmasının taahhüdü. "*Şu bilgileri şu formatta ver (alıcı adı, adres, telefon); ben sana şu bilgileri döneyim (takip numarası, tahmini teslim).*" İki taraf da bu söze göre kendi işini planlar.

Sözleşme neleri kapsar:

Parça	İngilizcesi	Bu projede nerede
İsteğin şekli	Request schema	<code>packages/contracts</code> → Zod şeması
Cevabın şekli	Response schema / DTO	Aynı paket, <code>select</code> ile eşleşir (E.6)
Hata biçimi	Error format	RFC 9457 Problem Details (E.7)
Adres ve fiil	Endpoint + method	<code>POST /api/v1/work-orders</code>
Kurallar	Constraints	"başlık en az 5 karakter", "sayfa boyutu en çok 100"

Yakın terimler — hangisi ne zaman kullanılır:

Terim	Ne demek
API Contract	Sözün kendisi — iki tarafın uyacağı kural
API Specification	O sözün belgelenmiş hâli (bu projede OpenAPI/Swagger)
Data Schema	Tek bir veri yapısının tanımı — sözleşmenin bir parçası

🚫 **Sözleşme neden önemli:** Değiştirdiğinde onu kullanan **herkesi** bozarsın. Bu yüzden `/api/v1` sürümlenmesi var (C.13): sözleşmeyi değiştirmek gerekirse eskisi çalışmaya devam eder, yenisi `/api/v2` olur. Mobil uygulamada bu **zorunludur** — kullanıcının telefonundaki sürümü sen güncelleyemezsin.

★ **README'nin en çok atlanan iki maddesi:** "*bilinen eksikler*" ve "*varsayımlar*". Ödev ikisini de açıkça istiyor. Bunları yazmak zayıflık değil, **projeye hâkimiyet** göstergesidir — neyin eksik olduğunu bilmek, eksik olmadığını iddia etmekten daha güvenilirdir.

E.13 Değerlendirilip seçilmeyen alternatifler

Teknik incelemede en sık gelen sorulardan biri "*şunu düşündün mü?*" olur. Aşağıdakiler düşünüldü, ölçüldü ve **bilerek** seçilmedi.

Neden Express, Fastify değil (NestJS'in altındaki HTTP katmanı)

NestJS bir HTTP sunucusu değil; altına bir adaptör takılıyor. İki seçenek var: **Express** (varsayılan) ve **Fastify** (daha hızlı).

Darboğaz = zincirin en yavaş halkası. Yalnızca onu hızlandırmanın anlamı var; diğerlerini hızlandırmak toplam süreyi değiştirmez.

Bir iş emri listesi isteğinin süresi kabaca şöyle dağılıyor:

Aşama	Süre
HTTP katmanı (Express/Fastify)	~0.3 ms
Doğrulama + yetki kontrolü	~0.5 ms
Veritabanı sorgusu	20–80 ms
Cevabı JSON'a çevirme	~1 ms

Fastify HTTP katmanında yaklaşık iki kat hızlı: 0.3 ms → 0.15 ms. **60 ms'lik bir istekte kazanç %0.25** — ölçüm aleti zor görür.

Aynı emeği doğru bir index'e harcamak 80 ms'yi 5 ms'ye indiriyor: **16 kat**.

i "Emeği index'lere harcadım" tam olarak ne demek

Index nedir: Kitabın arkasındaki **dizin**. Bir kelimenin hangi sayfada geçtiğini bulmak için kitabı baştan sona okumazsın — dizine bakarsın. Veritabanı index'i de aynı şey: "*durumu AÇIK olan iş emirleri*" sorusunun cevabını, 500 bin satırı tek tek okumadan bulmayı sağlayan yardımcı bir yapı.

"Emek" burada ne: Bir mühendisin sınırlı zamanı. O zamanı iki yere harcayabilirsin:

Nereye harcarsan	Ne kadar iş	Ne kazanırsın
Express yerine Fastify'a geçmek	Adaptörü değiştir, uyumsuz eklentileri düzelt, ekibe yeni aracı öğret	0.15 ms
Doğru index'i eklemek	Yavaş sorguyu bul, EXPLAIN ile plana bak, tek satırlık index tanımı yaz	75 ms

Cümlelerin anlamı: "*Hızlanmak istiyorsan darboğazın olduğu yeri hızlandır.*" İsteğin süresinin %95'i veritabanında geçiyorsa, HTTP katmanını iyileştirmek ölçülemeyen bir kazanç verir. Aynı emek veritabanına harcadığında 500 kat büyük bir kazanç veriyor.

★ Değerlendirmeci bunu sorduğunda anlatılacak şey teknoloji değil, **öncelik muhakemesi:** hangi optimizasyonun ölçülebilir karşılığı var.

★ **Savunma cümlesi:** "*Fastify daha hızlı, doğru. Ama isteğimizin süresinin %95'i veritabanında geçiyor; HTTP katmanını iki katına çıkarmak toplamda ölçülemeyen bir kazanç veriyor. Emeği index'lere harcadım. Nest'te adaptör tek satır — ölçüp gerekirse geçeriz, bu karar bizi bağlamıyor.*"

Ayrıca yaygınlık farkı büyük: Nest'in Express adaptörü haftada ~6.6M, Fastify adaptörü ~1.3M indiriliyor.

Neden REST, GraphQL değil

Önce yaygın bir yanılgıyı düzeltelim: "*REST tüm veriyi getirir, GraphQL sadece isteneni*" — bu, **kötü** tasarlanmış REST'in belirtisi, kuralı değil.

Bu projedeki REST zaten sadece gerekeni döndürüyor: liste ekranı için 7 alan, detay ekranı için 25 alan, ve veritabanından da yalnızca o alanlar çekiliyor (E.6). Yani mobil veri kotası argümanı burada geçerliliğini kaybediyor.

	REST	GraphQL
HTTP önbelleği	Kendiliğinden çalışır (CDN, tarayıcı, proxy)	Çalışmaz; kendi cache katmanını kurarsın
İzleme	Her uç ayrı adres — hangisi yavaş, hangisi hata veriyor doğrudan görünür	Tek adres (/graphql); izleme aracı hepsini aynı görür
Yetkilendirme	Uç bazında	Alan bazında — çok daha karmaşık
N+1 sorgu riski	Düşük	Yüksek; ek çözüm (DataLoader) gerekir
Nest'te yaygınlık	Varsayılan	@nestjs/graphql ~846K/hafta ile azınlıkta

★ **Belirleyici olan izleme maddesi:** Bu sistemi canlıda sen izlemeyeceksin, kurumun DevOps ekibi izleyecek. GraphQL'de "*hangi uç (endpoint) yavaşladı*" sorusu izleme aracında görünmez; kurumsal ortamda bu ciddi bir eksiktir.

Tablodaki "N+1 sorgu riski" satırı ne demek

Bu terim mülakatta da sorulur, gerçek sistemleri de gerçekten yavaşlatır.

Gerçek hayat: 50 kişilik bir sınıfın velilerinin telefonlarını topluyorsun. İki yöntem var:

Yöntem	Kaç kez arşive gidersin
Her öğrenci için ayrı ayrı dosya çekmek	1 (sınıf listesi) + 50 (her öğrenci) = 51
Sınıf listesini alıp " <i>bu 50 kişinin dosyasını birden ver</i> " demek	1 + 1 = 2

İlk yöntem **N+1**'dir: 1 ana sorgu + N tane ek sorgu. Sonuç aynı, maliyet 25 kat.

Yazılımdaki hâli: "*İş emirlerini listele, her birinin atandığı personelin adını da göster.*"

```
// 🚫 N+1 — bu kod ÇALIŞIR ve test ortamında HIZLI görünür
const workOrders = await prisma.workOrder.findMany(); // 1 sorgu: 50 iş emri geldi
for (const wo of workOrders) { // sonra 50 kez döngü
  wo.assignee = await prisma.user.findUnique({ // 🚫 her tur AYRI sorgu → 50 sorgu
    where: { id: wo.assignedToId },
  });
}
// Toplam: 51 veritabanı gidiş-gelişi. 10 kayıtla fark edilmez, 500 kayıtla ekran donar.
```

```
// ✔ Tek sorgu – Prisma ilişkiiyi aynı sorguda getiriyor
const workOrders = await prisma.workOrder.findMany({
  include: { assignedTo: { select: { id: true, fullName: true } } },
  //           ↑ "personeli de getir"           ↑ sadece bu iki kolonu (E.6)
});
// Toplam: 1 veritabanı gidiş-gelişi.
```

REST'te risk neden düşük: Cevabın şeklini **sen** yazıyorsun. Yukarıdaki `include` 'u bir kez doğru yazarsın, o uç hep tek sorgu atar.

GraphQL'de risk neden yüksek: Cevabın şeklini **istemci** belirliyor. İstemci "*iş emirlerini ver, her birinin personeli ver, her personelin de bağlı olduğu lokasyonu ver*" diye sorabilir. GraphQL bu isteği alan alan çözer ve **her alan için ayrı çağrı** yapar — kimse yanlış kod yazmadığı hâlde N+1 kendiliğinden oluşur.

Çözümü — DataLoader: Aynı turda istenen tekil kayıtları **biriktirip tek sorguda** getiren küçük bir kütüphane. "50 kez tek tek sor" yerine "50'sini bir kerede sor"a çevirir; yukarıdaki tablodaki 51 → 2 dönüşümünün kod karşılığıdır.

★ **2026 Ağustos itibarıyla ölçüldü:** `dataloader` haftada **13.6M** indiriliyor ve GraphQL dünyasında bunun yerini alan bir alternatif çıkmadı. Son yayını 2024-12 — ama bu terk edilmişlik değil, **tamamlanmışlık**: kütüphane tek bir iş yapıyor ve o işi bitirmiş durumda. Sunucu tarafında `graphql-yoga` (1.7M/hafta) ve `@apollo/server` (2.9M/hafta) ikisi de aktif ve ikisi de DataLoader'ı öneriyor.

⚠ **Bu satırın maliyeti:** GraphQL'e geçmek yalnızca "yeni bir kapı açmak" değil; yanında **DataLoader** kurmayı ve her ilişki için ayrı loader yazmayı da getirir. REST'te bu iş `include` satırıyla bitiyor.

GraphQL gerçekten ne zaman kazanır — örnekle

Kazandığı tek durum şu: **cevabın şeklini senin belirleyemediğin, sayısını bilmediğin istemciler.**

Bu projede DEĞİL:

İstemci	Kim yazıyor	Ne isteyeceğini biliyor muyum
Web arayüzü (UI)	Sen	Evet — listede 7 alan, detayda 25 alan
Mobil uygulama	Sen	Evet — aynı uçları çağırarak

İki istemci de senin. Yeni bir alan gerekirse uca eklersin, bir hafta sonra mobilde de kullanırsın. Ortada çözülecek bir problem yok.

Kazandığı durum — somut senaryo: Diyelim İzmir Büyükşehir bu iş emri verisini **kurum dışına** açtı:

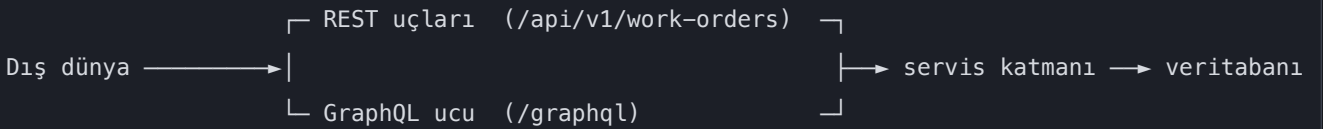
Tüketici	Ne istiyor
İlçe belediyesi paneli	Yalnızca kendi ilçesindeki iş emirlerinin sayısı ve durumu
Yüklenici firma	Kendisine atanmış iş emirlerinin tamamı + geçmişi + fotoğrafları
Merkezî 153 sistemi	Yalnızca vatandaş talebinin durumu ve tahmini bitiş saati
Açık veri portalı	Kişisel veri hariç aylık toplamlar
Bir üniversitenin araştırma ekibi	Coğrafi dağılım + kapanma süreleri

Beş tüketici, beş farklı alan kombinasyonu — ve **hiçbirini sen yazmıyorsun**. REST'te bunun iki kötü çözümü var: ya beş ayrı uç yazarsın (her yeni tüketicide bir tane daha), ya tek uçtan herkese her şeyi verirsın (fazla veri + gereksiz kişisel veri riski). GraphQL'de her tüketici kendi alanlarını seçer.

★ **Ayrım cümlesi:** "İstemci sayısı sabit ve hepsi bende ise REST; istemci sayısı açık uçlu ve ihtiyaçlarını ben bilmiyorsam GraphQL." Bu projede birinci durum geçerli. Yarın kurum dışına açılırsa karar yeniden değerlendirilir — mimari buna hazır (aşağıdaki bölüm).

İkisi AYNI ANDA kullanılabilir mi — evet

Bu, "birini seç" sorusu değil. İkisi aynı sistemde yan yana çalışabilir, çünkü ikisi de **yalnızca giriş kapısıdır**; arkalarındaki iş kuralları ortaktır.



i Bunu mümkün kılan şey ne — mimarinin somut getirisi

İkinci bir kapı açmak, ancak arkadaki kod o kapıyı tanımıyorsa mümkün olur.

Bu projede iş kuralları **HTTP'yi bilmiyor**: `WorkOrder.close()` metodu bir isteğin nereden geldiğini, hangi biçimde geldiğini, hatta bir istek olup olmadığını bilmiyor. Yalnızca kuralı biliyor (E.1 → Clean Architecture).

Sonuç: giriş kapısı **değiştirilebilir veya çoğaltılabilir**. REST'in yanına GraphQL eklendiğinde servis katmanı, doğrulama, yetki kontrolü ve veritabanı erişimi **tek satır değişmez**.

Aynı özellik başka kapılar için de geçerli: yarın bir mesaj kuyruğu tüketicisi veya bir komut satırı aracı eklense, onlar da aynı servisleri çağırır.

🚫 Bunun bozulduğu tek durum: iş kuralının controller'ın içine yazılması. O an kural HTTP'ye yapışır ve ikinci kapı açmak "aynı kuralı bir daha yazmak" hâline gelir.

Pratik sonuç: **bugün REST yazmak, yarın GraphQL eklemeyi engellemiyor**. Mevcut REST tüketicileri de çalışmaya devam ediyor.

"Başka kurumlar tüketirse" — belediyede somut karşılığı

Bugün bu API'yi yalnızca **senin yazdığın** web arayüzü çağırıyor. İhtiyacı olan alanları biliyorsun, uçları ona göre tasarlıyorsun.

Şu durumlar ortaya çıkarsa tablo değişir:

Senaryo	Neden farklı
Başka bir müdürlüğün panosu iş emri verilerini göstermek istiyor	Onların ihtiyacı senin liste ekranıyla aynı değil
İzmirim Kart uygulaması vatandaşın açtığı talebin durumunu göstermek istiyor	Yalnızca 3–4 alan istiyor; senin 7 alanlık listen fazla
Yüklenici firma portalı kendi ekibine atanmış işleri çekiyor	Kendi alan kümesi var, üstelik onların kodunu sen yazmıyorsun
Merkezî bir sistem entegrasyon istiyor	Alan adları ve biçimleri onların şartnamesine göre
Açık veri portalı anonim istatistik yayınlıyor	Kişisel veri içermeyen bambaşka bir kesit

Ortak nokta: **her tüketici farklı alan kombinasyonu istiyor ve sen o istemcileri yazmıyorsun**.

REST'te bu durumda iki kötü seçenek doğar:

- Her tüketici için ayrı uç açmak — `/work-orders/for-izmirim-kart`, `/work-orders/for-yuklenici` ... Uç sayısı tüketici sayısıyla birlikte artar, her biri ayrı bakım ister
- Herkese her şeyi döndürmek — gereksiz veri, gereksiz yük ve  **ihtiyacı olmayan tarafa kişisel veri göndermek**

GraphQL'in kazandığı yer tam burasıdır: her istemci kendi alanlarını sorar, sen yeni uç yazmazsın.

Karar bugün nasıl veriliyor

Dört soru sorulur; **hepsine "hayır" ise REST tek başına yeterlidir**:

Soru	Bu projede
API'yi senin yazmadığın istemciler tüketecek mi?	Hayır — web ve ileride mobil, ikisi de senin
Tüketicilerin veri ihtiyaçları birbirinden belirgin farklı mı?	Hayır — aynı ekranların aynı alanları
Tüketicileri sen güncelleyemiyor musun?	Hayır — ikisini de sen yayınlıyorsun
İzleme ve ön bellek kurumun altyapısına bağlı mı?	Evet → bu, GraphQL'in aleyhine bir cevap

Dördüncü soru bu projede belirleyici: sistemi canlıda **DevOps ekibi** izleyecek. GraphQL'de tüm istekler tek adrese (`/graphql`) gittiği için "hangi uç yavaşladı" sorusu izleme aracında görünmez.

⚠ **Bugün GraphQL eklemenin bedeli** — hiçbir karşılığı yokken ödenirdi: HTTP ön belleğinin devre dışı kalması, izlemenin körleşmesi, yetkinin alan bazına inmesi ve N+1 sorgu riski.

★ **Savunma cümlesi:** "REST ile GraphQL birbirini dışlamıyor; ikisi de giriş kapısı ve arkalarındaki iş kuralları ortak. Bugün tek tüketici biziz, bu yüzden REST yeterli ve izlenebilir. Yarın kontrol etmediğimiz istemciler bağlanmak isterse, servis katmanına dokunmadan GraphQL kapısı eklenebilir — bunu baştan mümkün kılmak için iş kurallarını HTTP'den bağımsız tuttum."

Neden Repository Pattern kullanılmadı

Ödev §12 bunu **zorunlu tutmuyor**, ama kullanılırsa gerekçe istiyor — kullanılmaması da açıklanmalı.

Repository Pattern nedir: Veritabanı erişimini bir ara katmanın arkasına saklamak. Amacı, iş kodunun veritabanı aracını doğrudan tanımaması.

Neden bu projede eklenmedi:

1. **Prisma zaten o soyutlamadır.** Prisma Client, SQL'i saklayan ve tip güvenli bir arayüz sunan katmanın kendisi. Üstüne ikinci bir katman koymak, aynı işi iki kez yapmak olur.
2. **select ve include yeteneklerini kısıtlar.** Repository arkasına saklanınca, her ekranın ihtiyaç duyduğu farklı alan kümesi için ya ayrı metot yazılır ya da her şey döndürülür. İkisi de E.6'daki projeksiyon kazancını yok eder.
3. **Ödevin uyardığı tuzağa götürür:** "Her entity için birbirinin aynısı generic CRUD servisleri oluşturulmamalıdır." Repository katmanı çoğu projede tam olarak buna dönüşür.

Peki iş kuralları veritabanına nasıl bağımsız kalıyor: Repository ile değil, **bağımlılığın tersine çevrilmesiyle** (E.2 → D harfi). Domain katmanı ihtiyaç duyduğu soruyu kendi arayüzü olarak tanımlıyor; onu Prisma ile cevaplayan sınıf altyapı katmanında duruyor.

★ **Savunma cümlesi:** "Repository eklemedim çünkü Prisma'nın kendisi o soyutlama. İkinci bir katman **select** yeteneklerini kısıtlar ve ödevin uyardığı generic CRUD tuzağına götürür. Domain'in bağımsızlığını Repository ile değil, bağımlılığı tersine çevirerek sağladım — ve bunu **dependency-cruiser** testiyle zorunlu kıldım."

Neden BullMQ, pg-boss değil

pg-boss işleri Redis yerine **PostgreSQL'de** tutuyor. Ödevin "Hangfire verileri PostgreSQL üzerinde saklanmalıdır" maddesine daha yakındı ve dördüncü bir konteyner gerektirmiyordu.

Seçilmeme sebebi **yaygınlık**: BullMQ ~7.9M indirme/hafta, pg-boss ~1.35M. Uzun ömürlü ve devredilecek bir kurum sisteminde daha az bilinen aracı seçmek, projeyi devralacak geliştiriciyi zorlar.

⚠ Redis'in eklenmesi tek amaçlı değil: iş kuyruğunun yanında **hız sınırı sayacı** ve **dağıtık kilit** için de kullanılıyor. Yani dördüncü konteynerin karşılığı yalnızca kuyruk değil.

BÖLÜM F — Bir iş emrinin hayatı (sunucu tarafı, uçtan uca akış)

Önceki bölümler parçaları tek tek anlattı. Bu bölüm hepsini **tek bir isteğin yolculuğunda** birleştiriyor: bir talep açıldığı andan iş emri kapandığı ana kadar hangi katman devreye giriyor, hangi kural nerede çalışıyor.

🕒 *Kod yazıldıkça gerçek dosya adları ve satır numaralarıyla doldurulacak.*

1. **Talep açılıyor** — Kullanıcı formu doldurur → Next form doğrulaması (`packages/contracts` şeması) → API'ye gider → **aynı şema** sunucuda tekrar doğrular (*istemciye asla güvenilmez*) — ayrıntısı hemen aşağıda
2. **İş kuralları** — Lokasyon pasif mi? Varlık kullanım dışı mı? Bu kontroller `packages/domain` içinde, veritabanı bilmeden
3. **SLA hesabı** — `SlaPolicyFactory` devreye girer (§6.1)
4. **Kayıt** — İş emri + ilk geçmiş kaydı **tek transaction** içinde
5. **Job planlanıyor** — BullMQ'ya gecikmeli hatırlatma bırakılır
6. **Atama** — Yalnızca teknik personele; atama + geçmiş kaydı yine tek transaction
7. **Durum değişikliği** — Durum makinesi geçerli mi diye bakar; geçersizse domain hatası → merkezî filtre → **409**
8. **Eş zamanlı güncelleme** — İki kişi aynı anda değiştirirse `version` kolonu çakışmayı yakalar → **409 Conflict** — `version` kolonunun ne olduğu aşağıda
9. **SLA aşımı** — Worker tarar, işaretler, bildirim üretir — **iki kez çalışsa da tek bildirim** (unique index + job anahtarı)
10. **Kapanış** — Çözüm açıklaması zorunlu; kapalı kayıt normal güncellemeyle değiştirilemez

★ 1. adımın açılımı: "aynı şema sunucuda tekrar doğrular" tam olarak nerede

Bu cümle en çok soru alan yer, çünkü **iki ayrı bilgisayardan** bahsediyor.

Şema nerede duruyor

`packages/contracts` bir **paylaşılan paket**. Ne web'e ne API'ye ait — ikisi de onu içeri alıyor:

```
packages/contracts/work-order.ts    ← ŞEMA BURADA, TEK KOPYA
|
├─> apps/web  içeri alır (kullanıcının tarayıcısında çalışır)
└─> apps/api  içeri alır (belediyenin sunucusunda çalışır)
```

```
// packages/contracts/work-order.ts – TEK tanım, iki yerde kullanılıyor
import { z } from 'zod';

export const talepOlusturSemasi = z.object({
  baslik:      z.string().min(5).max(200),    // en az 5, en çok 200 karakter
  aciklama:    z.string().min(10),            // en az 10 karakter
  lokasyonId:  z.uuid(),                      // geçerli bir kimlik biçimi olmalı
  oncelik:     z.enum(['DUSUK', 'ORTA', 'YUKSEK', 'KRITIK']), // yalnızca bu dört değer
});
```

Akış — hangi satır hangi bilgisayarda çalışıyor

#	Nerede	Ne oluyor	Hangi dosya
1	Tarayıcı (kullanıcının bilgisayarı)	Kullanıcı formu doldurur, "Gönder"e basar	<code>apps/web/.../talep-formu.tsx</code>
2	Tarayıcı	Şema burada çalışır. Başlık 3 harfse form hiç gönderilmez , kullanıcı anında uyarı görür	aynı dosya — <code>zodResolver(talepOlusturSemasi)</code>
3	Ağ	Veri JSON olarak internete çıkar: <code>POST /api/v1/work-orders</code>	—
4	Sunucu (belediyenin makinesi)	İstek NestJS'e ulaşır. Controller metodu henüz çalışmaz	<code>apps/api/.../work-orders.controller.ts</code>
5	Sunucu	★ Şema İKİNCİ KEZ burada çalışır — controller'a girmeden önce, <code>ZodValidationPipe</code> içinde	<code>nestjs-zod</code> · <code>app.useGlobalPipes(...)</code>
6	Sunucu	Geçersizse controller hiç çağrılmaz → 400 Bad Request döner	merkezî hata filtresi (E.7)
7	Sunucu	Geçerliyse controller çağrılır, servise iletir, iş kuralları çalışır	<code>packages/domain</code>


```
// apps/api/.../work-orders.controller.ts
import { talepOlusturSemasi } from '@bakim/contracts'; // ★ AYNI şema, web ile aynı dosya
import { ZodValidationPipe } from 'nestjs-zod';

@Post()
@UsePipes(new ZodValidationPipe(talepOlusturSemasi))
// † Bu satır "kapıda bekçi" gibidir: gövde şemaya uymuyorsa
// aşağıdaki metot HİÇ çalışmaz, istek 400 ile geri döner.
async talepOlustur(@Body() gövde: TalepOlusturDto) {
  // Buraya ulaşan veri şemadan GEÇMİŞTİR – burada tekrar kontrol etmeye gerek yok.
  return this.servis.olustur(gövde);
}
```

🚫 Neden iki kez — "zaten tarayıcıda kontrol ettik" yeterli değil

Tarayıcıdaki doğrulama bir güvenlik önlemi değildir, bir nezakettir.

Kullanıcı senin ekranını kullanmak zorunda değil. **F12** ile geliştirici araçlarını açıp, ya da hiç tarayıcı kullanmadan tek satır komutla isteği doğrudan gönderebilir:

```
# Ekranı hiç açmadan, senin form doğrulamandan geçmeden gönderilen istek:
curl -X POST https://bakim.izmir.bel.tr/api/v1/work-orders -H "Content-Type: application/json"
```

Sunucudaki şema olmasaydı bu istek veritabanına düşerdi. **Şema orada olduğu için 400 ile geri döner.**

Doğrulama	Amacı	Atlanabilir mi
Tarayıcıda (adım 2)	Kullanıcıya anında geri bildirim; boşuna ağ isteği atılmasın	✅ Evet — kullanıcı isterse atlar
Sunucuda (adım 5)	Güvenlik ve veri bütünlüğü	🚫 Hayır — son kapı burasıdır

★ **Kazanç şu:** iki doğrulama var ama **tek şema** var. Bir kural değişince ("*başlık en az 10 karakter olsun*") **packages/contracts** içinde tek satır değişir; iki taraf da aynı anda güncellenir. İki ayrı yere yazsaydın biri güncellenir, diğeri unutulurdu — bu, **E.3 (DRY)**'in somut karşılığı.

★ 8. adımın açılımı: **version** kolonu tam olarak nedir

Kısa cevap: veritabanı tablosunda sıradan bir sayı kolonu. Kütüphane değil, teknoloji değil, araç değil — kendi elinle koyduğun bir sütun. Değeri olan şey kolonun kendisi değil, **onunla kurulan kural**.

Gerçek hayat

İki kişi aynı Word belgesini indirip düzeltiyor. İkisi de kaydediyor. İkincinin kaydı, birincinin düzeltmelerinin **üstüne yazıyor** ve birinci kişi bunu hiç öğrenmiyor. Buna **kayıp güncelleme** denir.

Çözüm: belgenin üstüne bir **baskı numarası** yaz. Kaydederken "*ben 7. baskıyı düzelttim*" dersin. Arşivdeki belge hâlâ 7. baskıysa kabul edilir ve 8 olur. Başkası araya girip 8 yaptıysa **senin kaydın reddedilir** — sen de yeni hâli okuyup tekrar düzeltirsin.

Nerede duruyor

```
// apps/api/prisma/schema.prisma – veri modelinin tanımlandığı dosya
model WorkOrder {
  id          String   @id @default(uuid())
  baslik      String
  durum       Durum
  version     Int       @default(0) // ★ İŞTE BU. Sıradan bir tam sayı kolonu.
  //          Her başarılı güncellemede 1 artar.
}
```

prisma migrate çalıştığında PostgreSQL'de şu kolon oluşur:

```
ALTER TABLE "WorkOrder" ADD COLUMN "version" INTEGER NOT NULL DEFAULT 0;
```

Soru	Cevap
Hangi teknoloji?	Hiçbiri. PostgreSQL'de INTEGER kolon, Prisma şemasında Int alan
Kim üretir?	Sen — schema.prisma 'ya elle yazarsın
Kim artırır?	Güncelleme kodu, her başarılı yazmada version + 1
Kullanıcı görür mü?	Hayır. Cevapta gider ama ekranda gösterilmez; tarayıcı onu geri gönderir
Adı version olmak zorunda mı?	Hayır. rowVersion , revision de olur. Yaygın adı version

Nasıl çalışıyor

```
// 1) Kullanıcı iş emrini açar. API cevabında version da gider.
//   { id: "abc", baslik: "Pompa arızası", durum: "ACIK", version: 7 }

// 2) Kullanıcı "Kaydet"e basar. Tarayıcı OKUDUĞU version'ı geri yollar: 7

// 3) Sunucu güncellemeyi ŞARTLI yapar:
const sonuc = await prisma.workOrder.updateMany({
  where: {
    id:      girdi.id,
    version: girdi.version,  // ★ ŞART: kayıt HÂLÂ 7. sürümde mi?
  },
  data: {
    durum:  girdi.durum,
    version: { increment: 1 },  // başarılıysa 7 → 8
  },
});

// 4) updateMany kaç satır etkilediğini SAYI olarak döner.
if (sonuc.count === 0) {
  // 🚫 Hiçbir satır güncellenmedi = kayıt artık 7. sürümde DEĞİL.
  //   Demek ki aramızda başka biri kaydetti. Üstüne yazMIYORUZ.
  throw new ConflictException('Bu kayıt siz bakarken değiştirildi. Yenileyip tekrar deneyin.');
```

// → merkezî hata filtresi bunu 409 Conflict'e çevirir (E.7)

```
}
```

Üretilen SQL tek satır ve **atomik** — PostgreSQL bu işlemi bölmez:

```
UPDATE "WorkOrder" SET durum = 'DEVAM', version = version + 1
WHERE id = 'abc' AND version = 7;
-- Sonuç 1 satır → kabul.  Sonuç 0 satır → çakışma, 409.
```

İki kişi aynı anda kaydederse

Zaman	Ayşe	Mehmet	Veritabanı
10:00	Kaydı açar, <code>version=7</code> okur		<code>version=7</code>
10:01		Kaydı açar, <code>version=7</code> okur	<code>version=7</code>
10:05	Kaydeder → <code>WHERE version=7</code> eşleşir → 1 satır		<code>version=8</code>
10:06		Kaydeder → <code>WHERE version=7</code> eşleşmez → 0 satır	<code>version=8</code>
10:06		409 Conflict alır, ekranda uyarı görür	değişmez

❌ **version** kolonu olmasaydı: Mehmet'in kaydı 10:06'da Ayşe'nin değişikliğinin üstüne yazardı. Kimse hata görmezdi, sistem "başarılı" derdi ve Ayşe'nin yazdığı çözüm açıklaması **sessizce kaybolurdu**.

⚠️ **Bu hata tek kullanıcı testte HİÇ görünmez.** Ancak iki kullanıcı aynı kaydı aynı dakikada açtığı anda ortaya çıkar — yani canlıda. Bu yüzden ödev §20'de açıkça isteniyor.

i Adı neden "iyimser" (optimistic)

İki yaklaşım var:

	Kötümser kilit	İyimser kilit (bizim seçimimiz)
Mantığı	"Çakışma olacak, kaydı baştan kilitle"	"Çakışma nadirdir, kaydederken kontrol et"
Nasıl	Ayşe açınca satır kilitlenir, Mehmet bekler	Kimse beklemez, ikisi de çalışır
Maliyeti	Ayşe kahve içmeye giderse Mehmet 20 dakika bekler	Nadiren biri 409 alır ve tekrar dener
Ne zaman doğru	Çakışma sık ise (bilet/koltuk satışı)	Çakışma seyrek ise

İş emri sisteminde aynı kaydı aynı dakikada iki kişinin düzenlemesi seyrek — bu yüzden iyimser doğru seçim.

Bu akışta hangi karar nerede görünüyor

Adım	İlgili bölüm
Aynı şemanın iki tarafta doğrulaması	E.3 — DRY
İş kurallarının veritabanı bilmemesi	E.1 — Clean Architecture
SLA politikasının seçilmesi	E.4 — Factory Pattern
İş emri + geçmiş kaydının tek işlemde yazılması	E.5 ve E.8 — transaction
Gecikmeli hatırlatma işi	C.6 — BullMQ
Geçersiz durum geçişinin engellenmesi	E.5 — durum makinesi
Eş zamanlı güncellemenin yakalanması	E.8 — iyimser eşzamanlılık
Mükerrer bildirimin engellenmesi	C.5 ve C.6 — benzersiz index + iş anahtarı
Hataların 409'a çevrilmesi	E.7 — merkezî hata yönetimi

★ **Sunumda en çok işe yarayacak bölüm budur.** "Mimariyi anlat" dendiğinde katman isimlerini saymak yerine bu yolculuğu anlatmak, mimariyi **anlatmadan göstermek** demektir.

BÖLÜM G — Bir ekranın hayatı (arayüz / UI tarafı)

BÖLÜM F **sunucu tarafını** anlattı: bir isteğin ağdan girip veritabanına ulaşana kadarki yolculuğu. Bu bölüm aynı şeyi **arayüz tarafı** için yapıyor: kullanıcı bir adrese gittiği andan ekrandaki tabloyu gördüğü ana kadar hangi teknoloji nerede devreye giriyor.

i Terim: "arayüz" bu bölümde ekran demek

Bu belgede *arayüz* kelimesi iki farklı şeyi karşılıyor ve karıştırmamak gerekiyor (E.2 → I harfindeki uyarının aynısı):

Bu bölümde	Nedir	İngilizcesi
✅ Kullanıcı arayüzü	Ekrandaki sayfa, buton, tablo, form	UI
❌ Kod arayüzü	Sınıfların birbirine verdiği metod sözü	<i>interface</i>

Bundan sonra **arayüz (UI)** yazacağım — hangisi olduğu tereddütsüz belli olsun diye.

G.0 Önce sıra sorusu: arka uç (backend) mu önce, arayüz (UI) mü

Bu projedeki cevap: önce arka uç (backend), sonra arayüz (UI)

Yapım planında (BÖLÜM H) arayüz (UI) ekranları **Adım 10'da** başlıyor; ondan öncesindeki dokuz adım arka uç (backend). Sebebi üç madde:

#	Sebep	Somut karşılığı
1	Ekran, veri şeklini bilmeden çizilemez	İş emri listesinde hangi kolonlar var? <code>sla_bitis</code> bir tarih mi, kalan dakika mı? Bu cevap veri modelinden (Adım 3) gelir
2	Ekran yeni yetenek eklemeyi, var olanı insana açar	"İş emrini kapat" düğmesi, arkada <code>kapat()</code> kuralı yoksa hiçbir şey yapmaz
3	Arka uç (backend) bittiğinde ekran GERÇEK veriyle geliştirilebilir	Uydurma (mock) veriyle geliştirilen ekran, gerçek veri gelince hep bir yerinden patlar: boş liste, çok uzun metin, <code>null</code> alan

🚫 Bu plan neyi kabul etmiyor

"Önce ekranı yapalım, arkasını sonra bağlarız."

Yaygın ama pahalı. Ekran, **henüz var olmayan** bir veri şekline göre tasarlanır; arka uç (backend) yazılınca şekil değişir ve ekran baştan yazılır. İki kez iş yapılır.

⚠️ Bunun sinsi tarafı şu: ilk hafta **çok verimli görünür**. Ortada tıklanabilir ekranlar vardır, herkes memnundur. Maliyet üçüncü haftada, gerçek veri bağlanırken çıkar.

⚠️ Ama bu kural EVRENSEL DEĞİL — üç durumda tersine döner

Bu, "her projede önce backend" demek değildir. **Kuralın gerçek hâli şu:**

★ Sıra "önce arka uç (backend)" değil — **"önce VERİ ŞEKLİ kesinleşsin"**. Arka uç (backend) yazmak, veri şeklini kesinleştirmenin bir yoludur. Zaten kesinleşmişse o iş bitmiştir, doğrudan arayüzden başlanır.

Durum	Doğru sıra	Neden
Arka uç (backend) zaten var — API'ler yazılmış, veritabanı ayakta, yalnızca yüz yenileniyor	Önce arayüz (UI)	Veri şekli sabit ve değişmeyecek. Bekleyecek bir şey yok
Yalnızca arayüz değişiyor — arka uca hiç dokunulmuyor (yeniden tasarım)	Sadece arayüz (UI)	Zaten tek taraflı iş
Ürün belirsiz — ne isteneceği bilinmiyor, önce görülmesi gerekiyor	Tıklanabilir taslak → arka uç (backend) → gerçek arayüz	Taslak <i>atılmak üzere</i> yapılır; içine iş kuralı yazılmaz. Kararı hızlandırır, koda dönüşmez

★ **İzmir Büyükşehir'de karşılaşacağın en olası durum birincisidir:** kurumda API'ler ve veritabanı çoktan yazılmış olur, senden istenen yeni bir arayüz (UI) olur. O zaman bu plandaki Adım 1–9 **atlanır**, Adım 10'dan başlanır — ama öncesinde bir iş vardır: **mevcut API'nin sözleşmesini** **packages/contracts** içine **Zod şeması olarak yazmak**. Böylece G.2'deki akış aynen kurulur.

🚫 **Bu karar proje BAŞINDA verilir, ortasında değil.** Kurulumda **/yeni-proje** bunu soruyor; cevap **docs/project/teknoloji-ve-plan.md** içine yazılıyor. Yarisında sıra değiştirmek, iki yaklaşımın maliyetini birden ödemek demektir.

G.1 Arayüz (UI) tarafında hangi teknoloji nerede duruyor

Arka uçta olduğu gibi burada da her aracın **tek bir işi** var. Üst üste binmiyorlar; biri diğerinin bıraktığı yerden alıyor.

TypeScript	Dil. Aşağıdaki HER dosya .ts / .tsx
└ React	"Veriye göre ekranı çiz" motoru
└ Next.js	React'i çalıştıran çatı: adres, sunucu, derleme
└ Tailwind CSS	Görünüm: renk, boşluk, yerleşim
└ shadcn/ui	Hazır parçalar: buton, tablo...
└ TanStack Query	Sunucudaki veriyi getir/tazele
└ React Hook Form	Form durumu ve gönderimi
└ Zod (contracts)	Doğrulama — backend ile AYNI şema

Her biri **hangi soruyu** cevaplıyor:

Soru	Cevaplayan	Kart
Bu deęiřkenin tipi ne, yanlış alan adı yazdım mı?	TypeScript	C.3
Veri deęiřti, ekranın hangi parçası yeniden çizilecek?	React	C.2
<code>/is-emirleri/42</code> adresi hangi dosyaya karşılık geliyor? (ařaęıda açıldı)	Next.js (App Router)	C.2
Bu tablo nasıl görünecek — kenarlık, boşluk, renk?	Tailwind CSS	C.20
Açılır menüyü, modalı, tabloyu sıfırdan mı yazacaęım?	shadcn/ui	C.20
Veriyi ne zaman çekeyim, önbellekte tutayım mı, ne zaman tazeleyeyim?	TanStack Query	C.19
Formdaki 12 alanın deęerini ve hata mesajlarını kim tutuyor? (ařaęıda açıldı)	React Hook Form	C.20
Bu form deęeri geçerli mi?	Zod — <code>packages/contracts</code>	C.4


```
// apps/web/app/(protected)/is-emirleri/page.tsx
// Dosyanın başında "use client" YOK → bu bileşen SUNUCUDA çalışır.
// Faydası: ilk açılış hızlı, arama motoru içeriği görür,
//           ve buradaki kod tarayıcıya HİÇ inmez.
export default async function IsEmirleriSayfasi() { ... }
```

```
// apps/web/.../is-emri-filtresi.tsx
'use client'; // ★ Bu satır sınırdır: buradan itibaren TARAYICIDA çalışır.
// Neden gerekli: kullanıcı etkileşimi (tıklama, yazma, durum tutma)
// yalnızca tarayıcıda mümkün.
```

🚫 Kural: `'use client'` mümkün olan EN AŞAĞI yazılır

Ne demek: Bu satırı bir dosyanın başına koyduğunda, o dosya **ve içine koyduğu her şey** tarayıcıya iner. Yani etkisi tek dosyayla sınırlı değil, **aşağı doğru bulaşıcıdır**.

Gerçek hayat benzetmesi: Bir binanın elektriğini şalttan kesiyorsun. Şalteri **en üst kata** koyarsan altındaki bütün katlar etkilenir. **İhtiyacı olan dairenin kapısına** koyarsan yalnızca o daire etkilenir.

Somut örnek — aynı sayfa, iki kurgu:

🚫 YANLIŞ — `'use client'` sayfanın en üstünde

app/is-emirleri/page.tsx	<code>'use client'</code> ← şalter en üstte
├ <Baslik/>	→ tarayıcıya İNDİ (oysa sabit metin, gerek yok)
├ <IstatistikKartlari/>	→ tarayıcıya İNDİ (oysa sunucuda hesaplanabilirdi)
├ <IsEmriTablosu/>	→ tarayıcıya İNDİ (oysa veriyi sunucu çekebilirdi)
└ <FiltrePaneli/>	→ tarayıcıya İNDİ ✅ bunun inmesi GEREKİYORDU

✅ DOĞRU — yalnızca ihtiyacı olan bileşende

app/is-emirleri/page.tsx	(satır yok) ← sunucu bileşeni
├ <Baslik/>	→ sunucuda kaldı
├ <IstatistikKartlari/>	→ sunucuda kaldı
├ <IsEmriTablosu/>	→ sunucuda kaldı
└ FiltrePaneli.tsx	<code>'use client'</code> ← şalter yalnızca burada

Kaybedilen ne — üç somut şey:

Sunucuda kalırsa	Tarayıcıya inerse
O kodun JavaScript'i hiç indirilmez → sayfa daha hızlı açılır	Kullanıcı o kodu indirmek zorunda kalır
Veritabanına doğrudan erişebilir, ekstra API isteği gerekmez	Veriyi API üzerinden istemek zorundadır → bir gidiş-geliş daha
Kodun içi görünmez — iş mantığı kullanıcıya açılmaz	⚠ Kaynak kod tarayıcıda okunabilir hâle gelir

⚠ Bu yüzden "çalışıyor ama yavaş" durumu buradan doğar. Sayfa iki kurguda da doğru çalışır; fark yalnızca **ölçütüğünde** görünür: indirilen JavaScript boyutu ve ilk açılış süresi.

Ne zaman gerçekten gerekir: Bileşen kullanıcı etkileşimi içeriyorsa — tıklama, yazma, açılır menü, kendi içinde durum tutma (`useState`), tarayıcı API'si kullanma. Bunlar **yalnızca tarayıcıda** mümkündür.

★ **Pratik yöntem:** Etkileşimli parçayı **kendi dosyasına ayır** ve `'use client'` ı oraya koy. Sayfa sunucu bileşeni olarak kalsın, o küçük parçayı içine alsın.

G.2 Bir ekranın hayatı — on adım

Örnek ekran: **iş emri listesi** (`/is-emirleri`). Kullanıcı adrese gidiyor, filtreliyor, bir iş emrinin durumunu değiştiriyor.

1 — Kullanıcı adrese gidiyor

Tarayıcı → `https://bakim.izmir.bel.tr/is-emirleri?durum=ACIK&sayfa=2`

Next.js adresi **dosya yoluna** çevirir. Ayrı bir rota tablosu yoktur — klasör yapısının kendisi rotadır:

```
apps/web/app/
├── (protected)/          ← Parantezli klasör ADRESTE GÖRÜNMEZ.
│   │                   Sadece gruplar: altındakiler aynı korumadan geçer.
│   └── is-emirleri/
│       ├── page.tsx      ← /is-emirleri
│       └── [id]/
│           └── page.tsx   ← /is-emirleri/42 ([id] = değişken parça)
```

2 — Oturum kontrolü (arayüz / UI tarafındaki)

(`protected`) grubunun ortak `layout.tsx` 'i oturumu kontrol eder; yoksa giriş sayfasına yönlendirir.

🚫 **Bu güvenlik değildir.** Asıl kontrol sunucuda, API tarafında (BÖLÜM F, Adım 4). Buradaki kontrol yalnızca **kullanıcı deneyimi**: girişsiz birinin boş bir ekranla karşılaşmasını engeller. Kullanıcı bu kontrolü atlasabilir API isteği **401** döner.

3 — Adres çubuğu filtrelerin tek doğru kaynağı

Filtreler bileşen içinde bir değişkende değil, **adreste** tutuluyor:

```
// Sunucu bileşeni – filtreleri doğrudan adresten okuyor
export default async function IsEmirleriSayfasi({
  searchParams, // ← Next.js adresteki ?durum=ACIK&sayfa=2 kısmını buraya verir
}: { searchParams: Promise<Record<string, string>> }) {
  const filtre = filtreSeması.parse(await searchParams);
  //          ↑ ★ Adresten gelen veri de DOĞRULANIYOR.
  //          Kullanıcı adres çubuğuna ?sayfa=-5 yazabilir; şema bunu keser.
  return <IsEmirListesi filtre={filtre} />;
}
```

★ **Kazancı somut:** Kullanıcı filtreli listenin adresini kopyalayıp WhatsApp'ta gönderdiğinde karşı taraf **aynı listeyi** açar. Geri tuşu çalışır. Sayfa yenilendiğinde filtreler kaybolmaz. Filtreler bileşen içinde tutulsaydı üçü de olmazdı.

4 — Veri çekiliyor: TanStack Query

```
'use client';

import { useQuery } from '@tanstack/react-query';
import type { IsEmriListesi } from '@bakim/contracts'; // ★ Tip backend'le AYNI paketten

export function IsEmriListesi({ filtre }: { filtre: Filtre }) {
  const { data, isPending, isError, error } = useQuery({
    queryKey: ['is-emirleri', filtre],
    // ↑ ★ ÖNBELLEK ANAHTARI. filtre değişince anahtar değişir → yeni istek atılır.
    // Aynı filtreye geri dönülürse istek atILMAZ, önbellekten gelir.
    queryFn: () => apiGet<IsEmriListesi>('/api/v1/work-orders', filtre),
    // ↑ Ekran doğrudan fetch ÇAĞIRMAZ. Ortak katmandan geçer (adım 5).
    staleTime: 30_000,
    // ↑ 30 saniye boyunca veri "taze" sayılır; kullanıcı sekmeler arasında
    // gidip gelirse boşuna istek atılmaz.
  });

  if (isPending) return <TabloIskeleti />; // ★ ÜÇ DURUM da ele alınıyor:
  if (isError) return <HataKutusu hata={error} />; // yükleniyor / hata / boş
  if (data.items.length === 0) return <BosDurum />; // (kit kuralı – C.19)

  return <Tablo satirlar={data.items} />;
}
```

⚠ **Üç durumun üçü de zorunlu.** Yalnızca mutlu yol yazılırsa kullanıcı hata anında **boş beyaz ekran** görür ve neyin yanlış gittiğini anlamaz.

5 — Ortak API katmanı: hiçbir ekran doğrudan istek atmaz

```
// apps/web/lib/api.ts – TEK giriş noktası
export async function apiGet<T>(yol: string, sorgu?: Record<string, unknown>) {
  const cevap = await fetch(url(yol, sorgu), {
    credentials: 'include',
    // ↑ 🚫 Oturum çerezini gönderir. httpOnly olduğu için JS onu OKUYAMAZ –
    // XSS saldırısında jeton çalınamaz (C.15).
  });

  if (cevap.status === 401) return oturumuYenileVeTekrarDene(yol, sorgu);
  // ↑ ★ Jeton süresi dolduğunda kullanıcı bunu HİÇ görmez: sessizce yenilenir.
  // Bu mantık tek yerde durduğu için 12 ekranın hiçbirini bunu bilmek zorunda değil.

  if (!cevap.ok) throw new ApiHatasi(await cevap.json());
  // ↑ Backend'in RFC 9457 Problem Details cevabını (E.7) nesneye çevirir.
  return cevap.json() as Promise<T>;
}
```

★ **E.3 (DRY)'ın arayüz tarafındaki karşılığı.** Jeton (token) yenileme, hata biçimi ve temel adres tek dosyada. Değişirse tek yerde değişir.

6 — Görünüm: Tailwind + shadcn/ui

```
<Table>                                     {/* shadcn/ui – hazır tablo bileşeni */}
  <TableRow className="hover:bg-muted/50"> {/* Tailwind – üstüne gelince arka plan */}
    <TableCell className="font-medium">{satir.baslik}</TableCell>
    <TableCell>
      <DurumRozeti durum={satir.durum} />  {/* Kendi bileşenimiz – tek yerde tanımlı */}
    </TableCell>
  </TableRow>
</Table>
```

Araç	Ne veriyor	Neden bu
Tailwind	Hazır görünüm sınıfları (<code>font-medium</code> , <code>hover:bg-*</code>)	Ayrı CSS dosyası ve isim uydurma derdi yok; kullanılmayan stil derlemede atılır
shadcn/ui	Tablo/modal/menü gibi parçaların kodu senin depona kopyalanır	★ Bağımlılık değil, senin kodun . Tasarım değişince kütüphanenin izin vermesini beklemezsin

7 — Kullanıcı filtreyi değiştiriyor

```
function durumDegisti(yeniDurum: string) {
  const p = new URLSearchParams(searchParams);
  p.set('durum', yeniDurum);
  p.set('sayfa', '1'); // ⚠️ Filtre değişince sayfa 1'e döner.
  // Yoksa kullanıcı "sonuç yok" sanır — aslında 7. sayfadadır.
  router.push(`/is-emirleri?${p}`);
  // ↑ Adres değişti → adım 3'ten itibaren akış YENİDEN işler.
  // TanStack Query'nin queryKey'i de değişti → yeni istek atıldı.
}
```

🚫 **Filtreleme sunucuda yapılıyor** — 500 bin kaydı tarayıcıya indirip orada süzmek değil. [?](#)
`durum=ACIK&sayfa=2` API'ye gider, veritabanı yalnızca 20 satır döner (ödev §17 · E.10).

8 — Kullanıcı durum değiştiriyor: form + doğrulama

```
const form = useForm({
  resolver: zodResolver(durumDegistirSemasi),
  // ↑ ★ packages/contracts'tan gelen AYNI şema. Backend de bunu kullanıyor
  // (BÖLÜM F, 1. adımın açılımı). Kural tek yerde.
});

const mutation = useMutation({
  mutationFn: (girdi) => apiPatch(`/api/v1/work-orders/${id}/status`, {
    ...girdi,
    version: isEmri.version, // ★ Okuduğumuz sürüm geri gönderiliyor.
    // Çakışma kontrolü bununla yapılıyor (BÖLÜM F, 8. adım).
  }),
  onSuccess: () => {
    queryClient.invalidateQueries({ queryKey: ['is-emirleri'] });
    // ↑ ★ "Bu önbellek artık geçersiz" der. TanStack Query listeyi kendiliğinden
    // tazeler — sayfa yenilenmeden liste güncel hâle gelir.
  },
});
```

9 — Hata ekrana nasıl geliyor

Backend geçersiz bir geçişte **409** ve RFC 9457 gövdesi döner (E.7). Ekran o gövdeyi doğrudan gösterir:

```
{mutation.isError && (  
  <Uyari tur="hata">  
    {mutation.error.detail}  
    {/* ↑ Backend'in yazdığı açıklama: "Kapalı iş emri tekrar açılmaz."  
      ★ Mesaj arayüzde UYDURULMUYOR – kuralın sahibi backend, metni de o veriyor.  
      Yoksa kural değiştiğinde iki yerde güncelleme gerekirdi (E.3). */}  
  </Uyari>  
)}
```

Durum kodu	Kullanıcı ne görür
400	Alanın altında hata metni (form doğrulaması)
401	Hiçbir şey — jeton (token) sessizce yenilenir (adım 5)
403	"Bu işlem için yetkiniz yok"
409	"Bu kayıt siz bakarken değiştirildi. Yenileyip tekrar deneyin."
500	Genel hata kutusu + izleme kimliği (correlation ID)

★ **İzleme kimliği neden gösteriliyor:** Kullanıcı destek hattını aradığında o kodu söyler; DevOps ekibi log'larda **tam o isteği** bulur. Bu bağ olmadan "*dün bir hata aldım*" şikâyeti aranabilir değildir (C.16 · C.22).

10 — Tarayıcıda fiilen doğrulanıyor

Ekran "çalışıyor" diye kabul edilmez, **ölçülür:** `chrome-devtools-mcp` ile sayfa gerçekten açılır, tıklanır ve şunlar kontrol edilir — konsolda hata yok, başarısız ağ isteği yok, yükleniyor/hata/boş durumlarının üçü de görülüyor.

G.3 Bu akışta hangi karar nerede görünüyor

Adım	İlgili bölüm
Klasör yapısının adres olması	C.2 — Next.js App Router
Arayüzdeki oturum kontrolünün güvenlik olmaması	C.15 · BÖLÜM F → Adım 4
Filtrelerin adres çubuğunda tutulması	E.10 — listeleme ve sayfalama
Aynı Zod şemasının iki tarafta çalışması	E.3 — DRY · BÖLÜM F → 1. adımın açılımı
Önbellek ve tazeleme	C.19 — TanStack Query
Ortak API katmanı, sessiz jeton (token) yenileme	C.15 — kimlik doğrulama
<code>version</code> alanının geri gönderilmesi	E.8 · BÖLÜM F → 8. adımın açılımı
Hata metninin backend'den gelmesi	E.7 — merkezî hata yönetimi
Sunucuda filtreleme	§17 · E.10
shadcn kodunun depoya kopyalanması	C.20

★ **Sunumda kullanımı:** "*Frontend'i anlat*" dendiğinde teknoloji listesi saymak yerine bu on adımı anlatmak, BÖLÜM F'nin arka uç (backend) için yaptığı işi arayüz için yapar — **anlatmadan gösterir.**

BÖLÜM H — Yapım planı: adım adım ne, neyle, nereye

Önceki bölümler **neyi neden** seçtiğimizi anlattı. Bu bölüm **hangi sırayla yapılacağını** gösteriyor: her adımda ne üretiliyor, hangi teknolojiyle, hangi klasöre yazılıyor ve neye bağlanıyor.

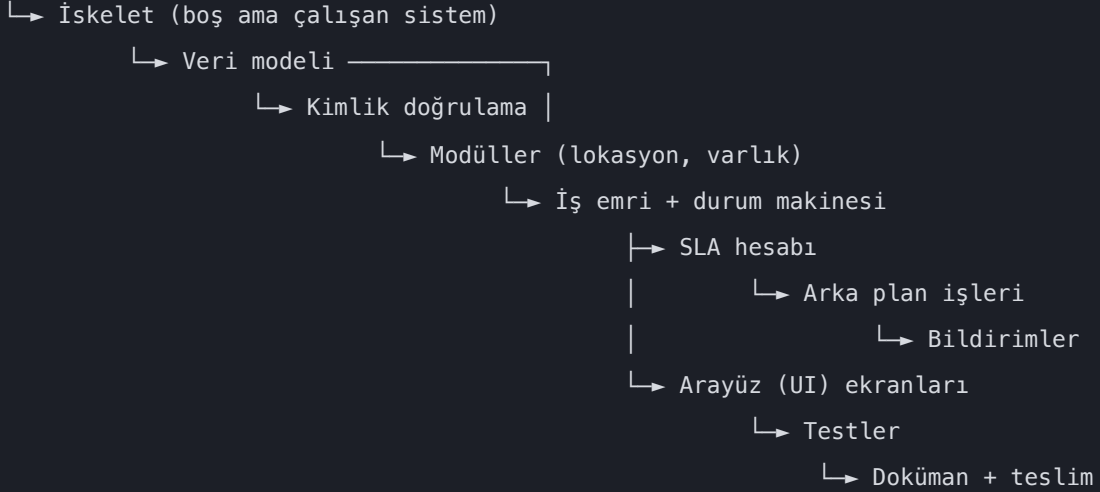
Bu plan nasıl kuruldu

Sıra keyfi değil. Dört kurala göre dizildi; her adımın neden orada olduğu bu kurallarla açıklanabiliyor.

Kural 1 — Bağımlılık: bir şey, dayandığı şeyden sonra gelir

En bariz olanı. Bir adım, kendinden önce var olması gereken şeyi bekler:

Ortam kurulumu



Neden bu yönde: Veri modeli olmadan API yazılamaz — hangi tabloya yazacağını bilmez. API olmadan ekran veri çekemez — çağıracak bir şey yoktur. Durum makinesi olmadan SLA hesaplanamaz — hangi iş emrinin hâlâ açık olduğu bilinmez.

Gerçek hayattan karşılığı

İnşaatta temel atılmadan duvar örülmez, duvar olmadan çatı kurulmaz. Kimse "çatıyı önce yapalım, temeli sonra atarız" demez — çünkü çatının duracağı bir yer yoktur.

Yazılımda bu bağımlılık **görünmez** olduğu için atlanabiliyor: kod yazılır, derlenir, hatta çalışır gibi görünür — ta ki dayandığı şeyin olmadığı ortaya çıkana kadar.

Kural 2 — Yatay kesen işler erken yapılır

Bazı şeyler tek bir modüle ait değildir; **her modülü kesip geçer**. Kimlik doğrulama, hata yönetimi, sayfalama, audit alanları böyledir.

Bunlar sonraya bırakılırsa, yazılmış her modüle **geri dönmek** gerekir.

Yatay kesen iş	Sonraya bırakılırsa
Kimlik doğrulama (Adım 4)	Yazılmış her uç noktaya geri dönüp yetki kontrolü eklenir
Merkezî hata yönetimi (Adım 5)	Her uçtaki try/catch tek tek sökülür
Sayfalama kalıbı (Adım 5)	Her listeleme ucu yeniden yazılır
Audit alanları (Adım 4)	Yazılmış her kayıt işlemine elle alan doldurma eklenir

Gerçek hayattan karşılığı

Binada elektrik tesisatı **sıva atılmadan önce** döşenir. Sonra döşenecekse duvarların kırılması gerekir.

Kimlik doğrulama yazılımın tesisatıdır: her odaya uğrar. Sıvadan sonra eklemek, her odayı yeniden açmak demektir.

Kural 3 — En riskli parça erken denenir

Projede belirsizliği en yüksek, hata ihtimali en fazla olan parça **öne alınır**. Sebebi şu: geç kalırsa, sürpriz çıktığında geri dönüş pahalıdır.

Bu projede o parçalar **durum makinesi (Adım 6)** ve **SLA Factory (Adım 7)**. İkisi de ödevin en çok puan verdiği yerler ve ikisi de yanlış tasarlanırsa üstüne kurulan her şey etkilenir.

 Ekranlar ise **düşük riskli**: ne yapacakları bellidir, sürpriz çıkmaz. Bu yüzden sona bırakıldılar — erken yapılsalardı, arka uçtaki bir tasarım değişikliği hepsini yeniden yazdırırdı.

Gerçek hayattan karşılığı

Bir yemeği ilk kez pişirirken en zor adımı (hamurun tutması, sosun kesilmemesi) önce dener, sonra süslemeye geçersiniz. Süslemeyi önce yapıp hamurun tutmadığını görmek, iki işi birden çöpe atmak demektir.

Kural 4 — Kalıp, basit yerde oturtulur

Aynı işi birçok kez yapacaksanız, **ilk yaptığınız yer en basit olan** seçilir. Orada oturttuğunuz kalıbı sonrakiler tekrarlar.

Bu projede **lokasyon ve varlık modülleri (Adım 5)** bu yüzden iş emrinden önce geliyor. İkisi de basit CRUD; karmaşık kural yok. Ama orada kurulan şeyler sonraki her modülde kullanılıyor:

- Zod şeması nasıl tanımlanır ve nereye konur
- Sayfalama yardımcısı nasıl çağrılır
- Hata filtresi hangi hatayı hangi koda çevirir
- Soft delete filtresi nereye yazılır

★ İş emri modülü doğrudan yazılsaydı, hem karmaşık iş kurallarıyla hem de henüz oturmamış kalıplarla aynı anda uğraşılırdı — ve çıkan hatanın hangisinden geldiği ayırt edilemezdi.

Sıranın dört evresi

Yukarıdaki kurallar birleşince plan dört evreye ayrılıyor:

Evre	Adımlar	Ne üretiliyor	Bittiğinde ne kanıtlanmış olur
1. Temel	0–2	Çalışan boş sistem	Altyapı ayakta: dört servis birbirini görüyor
2. Çekirdek	3–9	Veri, kurallar, arka plan	Sistem kullanıcısız çalışıyor: API üzerinden her şey yapılabilir
3. Yüzey	10–12	Ekranlar	İnsan kullanabilir
4. Teslim	13–16	Testler, doküman, paket	Başkası çalıştırabilir ve devralabilir

★ **2. evrenin sonunda sistem tamamdır — ekran olmadan.** Bu bilinçli: iş kuralları arayüzden bağımsız olduğu için (E.1), doğruluğu ekran yazılmadan kanıtlanabilir. Ekran yalnızca **var olan** bir yeteneği insana açıyor.

Bu plan neyi kabul etmiyor

- ❌ "Önce ekranı yapalım, arkasını sonra bağlarız." Yaygın ama pahalı: ekran, henüz var olmayan bir veri şekline göre tasarlanır; arka uç (backend) yazılınca şekil değişir ve ekran baştan yazılır.
- ❌ "Testleri sona bırakalım." Testler Adım 13'te *tamamlanıyor* ama her adımda **yazılıyor**. Sona bırakılan test, yazıldığında çoktan çalışan bir kodu onaylamaktan ibaret kalır — hata bulmaz.
- ❌ "Dokümanı en sonda yazarız." Adım 14 derleme adımdır, yazma adımı değil. Her adımın kararı o adım biterken yazılır; gerekçe, kararı verirken en net hatırlanır.

Kutucuklar: ☐ yapılmadı · ☒ bitti. Her adım tamamlandığında kutucuk işaretlenir; böylece projeye ara verilip dönüldüğünde nerede kalındığı tek bakışta görülür.

Adım 0 — Ortam kurulumu

Amaç: Geliştirme yapılabilir bir makine.

Teknoloji	Node.js 24, pnpm, Docker Desktop, Git, VS Code, Claude Code eklentileri
Nereye	Makinenin kendisi — projeye dosya yazılmıyor
Neye bağlanıyor	Sonraki her adım buna dayanıyor
Bitti sayılır	<code>node -v</code> , <code>pnpm -v</code> , <code>docker -v</code> , <code>gh --version</code> hepsi sürüm veriyor
Rehberde	C.10 Docker → port ve konteyner planı

Adım 1 — PRD: sistemin ne yapacağını yazılı hâli

Amaç: Ödevde yazmayan onlarca kararı netleştirmek. SLA süreleri kaç saat, iş emri numarası hangi biçimde, hangi rol neyi görebilir, kapsam dışı ne.

Teknoloji	Yok — bu bir görüşme adımı
Nereye	<code>docs/project/PRD.md</code>
Neye bağlanıyor	Veri modeli (Adım 3) ve iş kuralları (Adım 6-7) buradan türüyor
Bitti sayılır	PRD'de "açık soru" kalmadı
Rehberde	BÖLÜM B — sistemin yapması istenen 12 şey

Bu adım atılamaz. Cevabı bilinmeyen bir kural kodlanırsa, yanlış varsayım tüm katmanlara yayılır ve geri dönüşü pahalı olur.

i Neden en başta

"SLA süresi kaç saat" sorusunun cevabı veri modelini (hangi kolonlar), iş kurallarını (hangi eşik) ve arka plan işlerini (ne zaman tetiklenecek) birden etkiliyor. Yani bu tek cevap **üç ayrı adımı** belirliyor.

Kod yazıldıktan sonra öğrenilirse, üçünü birden değiştirmek gerekir. Terzi ölçü almadan kumaş kesmez; kesildikten sonra ölçü öğrenmek kumaşı çöpe atmaktır.

Adım 2 — İskelet: boş ama çalışan sistem

Amaç: Dört parçanın (web, api, worker, veritabanı) birbirini görerek ayağa kalkması. Henüz hiçbir özellik yok — sadece iskele.

Teknoloji	pnpm workspaces + Turborepo · NestJS · Next.js · Docker Compose
Nereye	apps/web , apps/api , apps/worker , packages/contracts , docker-compose.yml
Neye bağlanıyor	Web → API'ye "merhaba" isteği atıyor, API → veritabanına bağlanabiliyor
Bitti sayılır	docker compose up --build dörtünü birden ayağa kaldırıyor · /health/ready yeşil
Rehberde	C.1 NestJS · C.2 Next.js · C.10 Docker · C.14 Turborepo · C.17 Terminus

Neden önce boş iskelet

Bir özellik yazıp sonra "acaba Docker'da çalışır mı" diye bakmak, iki sorunu aynı anda çözmek demektir: hem özelliğin hatası hem altyapının hatası aynı anda karşına çıkar, hangisi olduğunu ayıramazsın.

Boş iskelet ayağa kalktığında altyapı **kanıtlanmış** olur; sonraki her hata yazdığın koda aittir.

Adım 3 — Veri modeli

Amaç: Tablolar, ilişkiler, index'ler ve ilk örnek veri.

Teknoloji	Prisma (şema + migration) · PostgreSQL · @dbml/cli
Nereye	apps/api/prisma/schema.prisma · prisma/migrations/ · docs/database.dbml
Neye bağlanıyor	Migration veritabanına tabloları kuruyor; DBML dosyası aynı şemadan otomatik üretiliyor
Bitti sayılır	Migration boş veritabanına uygulanıyor · seed verisi yükleniyor · CI'daki DBML kontrolü yeşil
Rehberde	C.3 Prisma · C.5 PostgreSQL · C.21 @dbml/cli · E.9 veri modeli kararları

Bu adımda verilen kararlar: soft delete hangi tablolarda, enum metin mi sayı mı, tarihler UTC mi, hangi kolonlara index, iş emri numarası nasıl üretiliyor. Hepsinin gerekçesi E.9'da.

Adım 4 — Kimlik doğrulama ve yetkilendirme

Amaç: Giriş, oturum yenileme, rol bazlı yetki, pasif kullanıcı engeli.

Teknoloji	@nestjsjs/jwt · argon2 · NestJS Guard · nestjs-cls
Nereye	apps/api/src/modules/auth/ · Guard'lar apps/api/src/common/guards/
Neye bağlanıyor	Giriş → jeton (token) üretiliyor → sonraki her istekte Guard jetonu (token) çözüp kullanıcıyı nestjs-cls bağlamına koyuyor → audit alanları oradan doluyor
Bitti sayılır	Yanlış şifre 401 · yetkisiz rol 403 · pasif kullanıcı giremiyor · jeton (token) yenileme eskisini iptal ediyor
Rehberde	C.13 JWT ve argon2 · C.16 nestjs-cls

⚠ **Neden bu kadar erken — Kural 2 (yatay kesen iş):** Sonraki her modülün yetki kontrolüne ihtiyacı var. Sonra eklenirse yazılmış her uç noktaya geri dönüp Guard takmak gerekir.

Ayrıca **audit alanları buna bağlı:** "kim oluşturdu" bilgisi ancak aktif kullanıcı bilinirse doldurulabiliyor. Kimlik doğrulama olmadan yazılan her kayıt, sahibi belirsiz kayıt olur.

Gerçek hayattan karşılığı

Bir kuruma giriş kartı sistemi, odalar döşenmeden önce kurulur. Sonra kurulacaksa her kapının yeniden sökülmesi gerekir — ve o ana kadar içeri kimin girip çıktığının kaydı yoktur.

Adım 5 — Lokasyon ve varlık modülleri

Amaç: İlk gerçek CRUD ekranlarının arka ucu. Basit oldukları için **kalıbı burada oturtuyoruz**; sonraki modüller bu kalıbı tekrarlayacak.

Teknoloji	NestJS modülü · Zod (packages/contracts) · Prisma · Exception Filter
Nereye	apps/api/src/modules/locations/ , ../assets/ · şemalar packages/contracts/
Neye bağlanıyor	İstek → Zod doğrulaması → servis → Prisma → veritabanı. Cevap dönerken şemadan geçip fazla alanlar kırılıyor
Bitti sayılır	Liste, detay, oluştur, güncelle, pasife al çalışıyor · pasif lokasyonda yeni varlık açamıyor
Rehberde	C.4 Zod · E.6 mapping · E.7 hata yönetimi · E.10 sayfalama

Burada kurulan ve tekrar kullanılacak parçalar: ortak sayfalama yardımcısı, merkezî hata filtresi, response şeması kalıbı, soft delete filtresi.

Neden iş emrinden önce — Kural 4 (kalıp basit yerde oturur)

Lokasyon ve varlık basit modüller: karmaşık iş kuralı yok, durum geçişi yok, eşzamanlılık sorunu yok. Kalıbı burada oturtmak, yalnızca **kalıpla** uğraşmak demek.

Doğrudan iş emriyle başlansaydı, hem oturmamış kalıpla hem karmaşık kuralla aynı anda uğraşılırdı — ve çıkan hatanın hangisinden geldiği ayırt edilemezdi. Yeni bir tarifi önce basit malzemeye denemek gibi.

Adım 6 — İş emri çekirdeği ve durum makinesi

Amaç: Sistemin kalbi. Talep açma, iş emrine dönüştürme, atama, durum değiştirme ve her değişikliğin geçmişe yazılması.

Teknoloji	Saf TypeScript (domain katmanı) · <code>prisma.\$transaction</code> · <code>version</code> kolonu
Nereye	Kurallar <code>packages/domain/work-order/</code> · akış <code>apps/api/src/modules/work-orders/</code>
Neye bağlanıyor	Durum değişikliği + geçmiş kaydı tek transaction içinde yazılıyor; eş zamanlı güncelleme <code>version</code> ile yakalanıp 409 dönüyor
Bitti sayılır	Geçersiz geçiş reddediliyor · kapalı iş emri güncellenemiyor · iki kişi aynı anda değiştirince biri 409 alıyor
Rehberde	E.5 durum makinesi · E.8 transaction ve eşzamanlılık · E.1 katmanlar

 Kurallar `packages/domain` içinde, Prisma'yı tanımadan yazılıyor. Bu, `dependency-cruiser` ile CI'da zorlanıyor (C.8).

Neden bu kadar erken — Kural 3 (riskli parça öne alınır)

Durum makinesi projenin en belirsiz parçası: hangi geçişe izin verileceği, hangi koşulun aranacağı, geçmişin nasıl tutulacağı burada kararlaşıyor. Yanlış tasarlanırsa **üstüne kurulan her şey** etkileniyor — SLA hesabı, bildirimler, ekranlar.

Ekranlar ise düşük riskli; ne yapacakları belli. Bu yüzden riskli olan başa, belirli olan sona alındı.

Adım 7 — SLA hesabı ve Factory Pattern

Amaç: İş emrine uygulanacak süre politikasının seçilmesi ve üç zamanın (bitiş, hatırlatma, yükseltme) hesaplanması.

Teknoloji	NestJS bağımlılık enjeksiyonu (çoklu sağlayıcı) · Factory Pattern · Clock soyutlaması
Nereye	<code>packages/domain/sla/</code> — her politika ayrı dosya, factory bir dosya
Neye bağlanıyor	İş emri oluşturulurken factory çağırılıyor → hesaplanan zamanlar iş emriyle aynı satıra yazılıyor → hatırlatma işi kuyruğa bırakılıyor
Bitti sayılır	Her politika ayrı ayrı test edilmiş · factory'nin doğru politikayı seçtiği test edilmiş · yeni politika eklemek mevcut kodu değiştirmiyor
Rehberde	E.4 Factory Pattern · E.0 → SLA · E.2 → O harfi

Adım 8 — Arka plan işleri

Amaç: Kimse ekranı açmasa da çalışan dört iş: SLA hatırlatma, ihlal taraması, günlük özet, arşiv adayı belirleme.

Teknoloji	BullMQ + Redis · ayrı worker süreci (<code>apps/worker</code>)
Nereye	İş tanımları <code>apps/worker/src/jobs/</code> , kuyruğa bırakma <code>apps/api</code> içinden
Neye bağlanıyor	API işi kuyruğa bırakıyor → Redis'te bekliyor → worker zamanı gelince alıp çalıştırıyor → sonuç veritabanına yazılıyor
Bitti sayılır	İş iki kez çalışsa da tek bildirim üretiyor · kapalı iş emrinde hiçbir şey yapmıyor · hatada yeniden deniyor
Rehberde	C.6 BullMQ + Redis · C.16 → worker bağlamı

⚠ **İşler yalnızca kimlik numarası alıyor**, nesnenin tamamını değil — kuyrukta beklerken veri eskiyebilir.

i Neden SLA'dan sonra

Arka plan işlerinin çoğu SLA zamanlarına bakıyor: "süresi yaklaşan" ve "süresi geçen" iş emirlerini buluyor. O zamanlar hesaplanmadan bu işler neye bakacağını bilemez.

Sıra tersine çevrilseydi, işler yazılır ama test edilemezdi — kontrol edecekleri alan henüz dolmuyor olurdu.

Adım 9 — Bildirimler ve yönetim istatistikleri

Amaç: Kullanıcıya sistem içi bildirim, yöneticiye özet sayılar.

Teknoloji	Prisma benzersiz index · Prisma toplama sorguları
Nereye	<code>apps/api/src/modules/notifications/</code> , <code>.../dashboard/</code>
Neye bağlanıyor	Durum değişikliği ve arka plan işleri bildirim üretiyor; benzersiz index mükerrerini engelliyor
Bitti sayılır	Aynı olay iki kez işlense de tek bildirim var · sayılar veritabanında hesaplanıyor, bellekte değil
Rehberde	C.5 → unique constraint · E.10 → sayma

Adım 10 — Arayüz (UI) temeli

Amaç: Giriş ekranı, oturum yönetimi, korumalı sayfalar, ortak API katmanı.

Teknoloji	Next.js App Router · TanStack Query · Tailwind + shadcn/ui
Nereye	<code>apps/web/app/(auth)/</code> , <code>apps/web/app/(protected)/</code> , <code>apps/web/hooks/</code>
Neye bağlanıyor	Ekran → <code>hooks/</code> içindeki ortak katman → API. Hiçbir ekran doğrudan istek atmıyor
Bitti sayılır	Girişsiz kullanıcı korumalı sayfaya giremiyor · jeton (token) süresi dolunca sessizce yenileniyor · rol bazlı butonlar gizleniyor
Rehberde	★ BÖLÜM G — bir ekranın hayatı (arayüz tarafının tamamı) · C.2 Next.js · C.19 TanStack Query · C.20 Tailwind/shadcn

★ **Bu adımdan itibaren arayüz (UI) tarafındasın.** Adım 1–9 arka uçtu; bundan sonraki üç adımın teknolojileri, katman sırası ve uçtan uca akışı **BÖLÜM G**'de ayrı ayrı anlatılıyor — hangi araç hangi soruyu cevaplıyor (G.1), bir ekranın on adımlık yolculuğu (G.2), hangi kararın nerede görüldüğü (G.3).

⚠ **Ekranlarda buton gizlemek güvenlik değildir** — asıl kontrol sunucuda (Adım 4). Butonu gizlemek kullanıcıya kolaylıktır; yetkisi olmayan biri isteği elle de gönderebilir ve sunucu onu reddetmek zorundadır.

i Neden ekranlar bu kadar geç

Bu noktada arka uç (backend) **tamamen çalışıyor**: her işlem API üzerinden yapılabilir, kurallar test edilmiş durumda. Ekran, var olan bir yeteneği insana açıyor — yeni bir yetenek eklemiyor.

Ekranlar önce yazılıysaydı, henüz var olmayan bir veri şekline göre tasarlanır ve arka uç (backend) yazılınca baştan elden geçirilirdi.

⚠ **Bu sıra evrensel değil.** Arka uç (backend) zaten yazılmışsa (kurumda sık olan durum) doğru sıra tersine döner ve doğrudan bu adımdan başlanır. Üç istisna ve kuralın gerçek hâli **G.0**'da.

Adım 11 — İş emri listesi

Amaç: Sunucu tarafı filtreleme, arama, sıralama, sayfalama ve filtrelerin adres çubuğuyla senkronu.

Teknoloji	Prisma <code>where / skip / take</code> · <code>pg_trgm</code> + GIN index · Next.js <code>searchParams</code>
Nereye	Uç <code>apps/api/.../work-orders/</code> , ekran <code>apps/web/app/(protected)/is-emirleri/</code>
Neye bağlanıyor	Kullanıcı filtre seçiyor → adres çubuğuna yazılıyor → API'ye gidiyor → veritabanı yalnızca o sayfayı dönüyor
Bitti sayılır	Filtreli liste adresi paylaşıldığında aynı sonucu açıyor · azami sayfa boyutu sınırı çalışıyor
Rehberde	E.10 listeleme ve sayfalama · C.5 → metin araması

Adım 12 — İş emri detayı ve kalan ekranlar

Amaç: 12 ekranın tamamlanması: detay, düzenleme, durum değiştirme, atama, yorum, geçmiş, bildirimler, hata ekranları.

Teknoloji	React Hook Form + Zod resolver · TanStack Query mutation
Nereye	<code>apps/web/app/(protected)/**</code>
Neye bağlanıyor	Form doğrulaması backend ile aynı Zod şemasını kullanıyor; işlem başarılı olunca liste otomatik tazeleniyor
Bitti sayılır	12 ekranın hepsi çalışıyor · backend hata mesajları kullanıcıya gösteriliyor
Rehberde	C.20 React Hook Form · C.19 → <code>invalidateQueries</code>

Adım 13 — Test tamamlama

Amaç: Üç test türünün de yeşil olması.

Teknoloji	Vitest (birim) · Testcontainers (entegrasyon) · dependency-cruiser (mimari) · Playwright (uçtan uca)
Nereye	Her modülün yanında <code>*.spec.ts</code> , entegrasyon <code>apps/api/test/</code>
Neye bağlanıyor	CI bu dört zinciri sırayla koşuyor; biri kırmızıysa kod ana dala giremiyor
Bitti sayılır	Ödev §23'teki senaryoların tamamı test edilmiş · koruma testleri geçici kaldırma yöntemiyle kanıtlanmış
Rehberde	C.7 Testcontainers · C.11 Vitest · C.8 dependency-cruiser · C.12 Playwright

Adım 14 — Dokümantasyon ve sunum

Amaç: Ödevin istediği sekiz doküman, ADR'ler, `AI_USAGE.md` ve sunumun derlenmesi.

Teknoloji	Yok — yazı işi
Nereye	<code>README.md</code> , <code>AI_USAGE.md</code> , <code>docs/**</code> , <code>docs/decisions/</code>
Neye bağlanıyor	Her session'da alınan kararlar buraya birikmiş oluyor
Bitti sayılır	Sekiz dosyanın hepsi var · en az üç ADR yazılmış · bilinen eksikler listelenmiş
Rehberde	E.12 ADR ve AI_USAGE

⚠ Bu adım **en sona bırakılmaz**. Her session'ın kararları o session biterken yazılır; gerekçe, kararı verirken en net hatırlanır.

Adım 15 — Teslim paketi

Amaç: DevOps'un sorunsuz çalıştırabileceği paket.

Teknoloji	Çok aşamalı Dockerfile · Docker Compose · GitLab CI
Nereye	<code>Dockerfile</code> , <code>docker-compose.yml</code> , <code>.env.example</code> , <code>.gitlab-ci.yml</code>
Neye bağlanıyor	Tek komut dört servisi ayağa kaldırıyor; migration açılışa uygulanıyor
Bitti sayılır	Temiz bir makinede <code>docker compose up --build</code> çalışıyor · <code>.env.example</code> eksiksiz
Rehberde	C.10 Docker · E.11 CI · BÖLÜM 0 → sistem nasıl çalışıyor

Adım 16 — Kite geri yazma

Amaç: Bu projede öğrenilen ve **stack'ten bağımsız** olan kuralların bir sonraki projeye taşınması.

Teknoloji	<code>/kit-senkron</code>
Nereye	<code>proje-kiti</code> eklentisi
Neye bağlanıyor	Kural kite yazılınca sonraki her projede otomatik geçerli oluyor
Bitti sayılır	Öğrenilen kurallar kite işlendi ve sürüm yayınlandı
Rehberde	—

Her adımın sonunda yapılacaklar

Bir adım bittiğinde, **oturum kapatılmadan önce** sırayla:

1. Testler yeşil mi (`pnpm ci:verify`)
2. Bu adımın kararları ilgili dokümana yazıldı mı
3. Commit atıldı ve değişiklik önerisi açıldı mı
4. Yukarıdaki kutucuk  →  yapıldı mı
5. Bir sonraki adımı tarif eden not güncellendi mi

 **4. madde atlanmaz.** Projeye bir hafta ara verilip dönüldüğünde, nerede kalındığını hatırlamanın tek güvenilir yolu bu liste.

KAPANIŞ

Bilinen teknik borçlar

Ödev §31 bunu açıkça soruyor ve **sorulmadan söylenmesi** olumlu değerlendirilir: neyin eksik olduğunu bilmek, eksik olmadığını iddia etmekten daha güvenilirdir.

 *Proje ilerledikçe doldurulacak.*

Bu stack'in tek cümlelik özeti

Kullanıcı **Next.js** ile yazılmış ekranı açar. Ekran, **NestJS** ile yazılmış API'ye istek atar. API gelen veriyi **Zod** ile doğrular, yetkiyi **JWT** ile kontrol eder, iş kurallarını uygular ve **Prisma** üzerinden **PostgreSQL**'e gider. Sonuç JSON olarak döner. Kullanıcıyı bekletmemesi gereken işler **BullMQ** ile kuyruğa alınır; ayrı bir **worker** süreci onları yapar. Tümü **Docker** ile paketlenir ve tek komutla ayağa kalkar.

Değişmeyen ilke

Stack baştan sona değişti, ancak ödevin sorduğu hiçbir yetenek düşmedi:

Ödevin sorduğu yetenek	Karşılandığı yer
Factory Pattern	SLA politikaları + NestJS bağımlılık enjeksiyonu
Servis yaşam döngüleri	NestJS DI (DEFAULT / REQUEST / TRANSIENT)
Transaction	prisma.\$transaction
İyimser eşzamanlılık	version kolonu + koşullu güncelleme
Mimari testler	dependency-cruiser
DBML şema dokümanı	@dbml/cli + CI kontrolü
Gerçek veritabanıyla entegrasyon testi	Testcontainers
Zamanlanmış görevler	BullMQ + worker
Merkezî hata yönetimi	NestJS Exception Filter
Merkezî audit	Prisma eklentisi + istek bağlamı

Araç değişti, beklenti değişmedi.